



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu

Plataforma Web para Gestão e Cooperação entre Produtores Agrícolas

Sérgio Daniel Carvalho da Costa

Trabalho de Projeto

Licenciatura em

Engenharia Informática



**Politécnico
de Viseu**

Escola Superior
de Tecnologia
e Gestão de Viseu



Plataforma Web para Gestão e Cooperação entre Produtores Agrícolas

Sérgio Daniel Carvalho da Costa

Realizado na empresa Visionary Devotion Unipessoal Lda

Relatório de Projeto

Licenciatura em

Engenharia Informática

Trabalho efetuado sob a orientação de
Professor Mauro Lima (ESTGV)
Eng.º Rafael Marques (Visionary Devotion Unipessoal
Lda)

junho de 2026

Agradecimentos

Ao longo deste percurso, várias pessoas e entidades contribuíram, de forma direta ou indireta, para a concretização deste projeto e para a conclusão desta etapa académica.

Em primeiro lugar, agradece-se ao Professor **Mauro Lima**, pela orientação, disponibilidade, espírito crítico e acompanhamento prestado ao longo do estágio e da elaboração do presente relatório. O seu apoio foi determinante para a consolidação do trabalho desenvolvido e para a reflexão sobre as opções técnicas e metodológicas adotadas.

Ao orientador da empresa, **Rafael Marques**, é devido um agradecimento especial pela confiança depositada, pelo acompanhamento próximo do projeto, pela partilha de conhecimento e pela clarificação contínua dos requisitos e objetivos do sistema. A sua colaboração revelou-se essencial para alinhar o desenvolvimento técnico com as necessidades reais do contexto empresarial.

À **Visionary Devotion Unipessoal Lda.**, agradece-se a oportunidade de realização do estágio curricular, bem como o contexto profissional proporcionado para aplicação prática dos conhecimentos adquiridos ao longo da licenciatura.

Aos meus pais e à minha família, pelo apoio incondicional, pela paciência, pelo incentivo constante e pela força transmitida ao longo de todo este percurso académico.

Aos meus colegas e amigos, especialmente a Sr. Diogo Caçador e Jaime Ribeiro, pela entreaajuda, motivação e companheirismo, que contribuíram para tornar este caminho mais suportável e enriquecedor.

A todos os que, de alguma forma, acompanharam este trajeto e contribuíram para a sua concretização, fica expresso o meu sincero agradecimento.

Resumo

A crescente necessidade de melhorar a gestão de recursos no setor agrícola, reduzir desperdícios e facilitar a articulação entre produtores tem impulsionado o desenvolvimento de soluções digitais orientadas para contextos cooperativos. Neste enquadramento, o presente relatório descreve o trabalho desenvolvido no âmbito do estágio curricular da Licenciatura em Engenharia Informática da Escola Superior de Tecnologia e Gestão de Viseu, realizado na empresa Visionary Devotion Unipessoal Lda., no período de 21 de fevereiro de 2026 a 5 de junho de 2026.

O projeto teve como principal objetivo o desenvolvimento de uma plataforma web para apoio a uma cooperativa agrícola, permitindo integrar a gestão de produtos, stocks, necessidades, excedentes e transações internas entre produtores. A solução foi concebida para apoiar os agricultores na identificação de oportunidades de compra e venda dentro da rede cooperativa, promovendo uma melhor redistribuição de recursos e contribuindo para a redução do desperdício agrícola.

No seu estado atual, a plataforma inclui módulos de autenticação e gestão de utilizadores, perfil do produtor, catálogo de produtos, gestão de stocks, necessidades e previsões de produção, marketplace interno com anúncios de stock atual e pré-venda, recomendações de apoio à decisão, encomendas, alertas automáticos, mensagens em tempo real e suporte ao utilizador.

A aplicação foi desenvolvida com recurso à framework Django, utilizando a linguagem Python e uma base de dados PostgreSQL. A interface web foi construída com HTML, CSS, Bootstrap e HTMX. A arquitetura adotada apresenta uma organização modular, assente num modelo relacional próprio, com comunicação em tempo real através de Django Channels, Redis e Daphne, armazenamento de ficheiros media em Cloudinary e colocação em produção na plataforma Railway.

Os resultados obtidos demonstram que a solução desenvolvida responde aos objetivos definidos para o estágio, disponibilizando uma base funcional sólida para a gestão cooperativa de recursos e para a articulação comercial entre produtores. O projeto permitiu ainda consolidar competências técnicas nas áreas de desenvolvimento web, modelação de bases de dados, arquitetura aplicacional, integração de serviços e implementação de funcionalidades orientadas a um contexto real de utilização.

Palavras-Chave: Cooperativa agrícola; Marketplace interno; Django; gestão de stocks; Sistema de recomendações; Plataforma web.

Abstract

The growing need to improve resource management in the agricultural sector, reduce waste, and enhance coordination between producers has driven the development of digital solutions tailored to cooperative environments. In this context, this report describes the work carried out within the scope of the curricular internship of the Bachelor's Degree in Computer Engineering at the School of Technology and Management of Viseu, conducted at Visionary Devotion Unipessoal Lda., from February 21, 2026 to June 5, 2026.

The main objective of the project was the development of a web platform to support an agricultural cooperative, integrating functionalities for managing products, stock levels, needs, surpluses, and internal transactions between producers. The solution was designed to assist farmers in identifying buying and selling opportunities within the cooperative network, promoting better resource redistribution and contributing to the reduction of agricultural waste.

In its current state, the platform includes modules for user authentication and management, producer profiles, product catalog, stock management, needs and production forecasts, an internal marketplace with both available stock and pre-sale listings, decision-support recommendations, order management, automated alerts, real-time messaging, and user support.

The application was developed using the Django framework with Python and a PostgreSQL database. The web interface was built with HTML, CSS, Bootstrap, and HTMX. The adopted architecture follows a modular structure based on a custom relational model, with real-time communication supported by Django Channels, Redis, and Daphne, media storage handled by Cloudinary, and deployment carried out on the Railway platform.

The results obtained demonstrate that the developed solution meets the objectives defined for the internship, providing a solid functional foundation for cooperative resource management and commercial interaction between producers. The project also contributed to the consolidation of technical skills in web development, database modeling, application architecture, service integration, and the implementation of features tailored to real-world usage contexts.

Keywords: agricultural cooperative; Web platform; internal marketplace; inventory management; recommendation system; Django.

Índice

ÍNDICE DE FIGURAS	XIV
ÍNDICE DE TABELAS	XVI
LISTA DE SIGLAS / ABREVIATURAS.....	XVII
GLOSSÁRIO.....	XX
1 INTRODUÇÃO	1
1.1 ENTIDADE DE ACOLHIMENTO	2
1.2 OBJETIVOS E RESULTADOS ESPERADOS	3
1.3 PLANO DE TRABALHO	4
1.4 ESTRUTURA DO RELATÓRIO	5
2 ESTADO DA ARTE	7
2.1 COOPERATIVAS AGRÍCOLAS E A TRANSFORMAÇÃO DIGITAL.....	8
2.2 PLATAFORMAS DE GESTÃO E COMERCIALIZAÇÃO AGRÍCOLA.....	9
2.2.1 AGRIVI.....	10
2.2.2 Farmbrite	11
2.2.3 Outras Soluções Identificadas.....	11
2.2.4 Análise Comparativa.....	13
2.3 SISTEMAS DE APOIO À DECISÃO NA AGRICULTURA	14
2.4 MODELOS DE PREVISÃO APLICADOS À AGRICULTURA	15
2.5 SÍNTESE CRÍTICA DO ESTADO DA ARTE	16
3 METODOLOGIAS	17
3.1 ESTUDO INICIAL DO PROBLEMA E CONTEXTO DA EMPRESA.....	18
3.2 ESTUDO DE MERCADO E ANÁLISE DE SOLUÇÕES EXISTENTES.....	18
3.3 LEVANTAMENTO E ANÁLISE DE REQUISITOS	19
3.4 PROTOTIPAGEM E VALIDAÇÃO INICIAL DA INTERFACE	20
3.5 ORGANIZAÇÃO DO DESENVOLVIMENTO E VALIDAÇÃO PROGRESSIVA.....	20

4	ARQUITETURA GERAL DA SOLUÇÃO	22
4.1	VISÃO GERAL DA SOLUÇÃO	22
4.2	APLICAÇÃO MONOLÍTICA MODULAR E ORGANIZAÇÃO EM MÓDULOS	24
4.3	PADRÃO MVT DO DJANGO	25
4.4	BASE DE DADOS COMO FONTE DE VERDADE	26
4.5	MODELO DE DADOS E PRINCIPAIS ENTIDADES	27
4.6	COMUNICAÇÃO EM TEMPO REAL E INFRAESTRUTURA	28
4.7	SEGURANÇA, FICHEIROS E CONFIGURAÇÃO.....	30
5	TECNOLOGIAS E FERRAMENTAS UTILIZADAS	31
5.1	AMBIENTE DE DESENVOLVIMENTO E CONTROLO DE VERSÕES	31
5.2	PROTOTIPAGEM DE ALTA FIDELIDADE.....	33
5.3	PYTHON E DJANGO — <i>BACKEND</i> DA APLICAÇÃO.....	34
5.4	POSTGRESQL E PGADMIN 4 — BASE DE DADOS E MODELAÇÃO RELACIONAL.....	34
5.5	DJANGO TEMPLATES, HTML, CSS, BOOTSTRAP, HTMX E CHART.JS — INTERFACE E INTERATIVIDADE	36
5.6	DJANGO CHANNELS, REDIS E DAPHNE — COMUNICAÇÃO EM TEMPO REAL	38
5.7	SERVIÇOS EXTERNOS, FICHEIROS, CONFIGURAÇÃO E EMAIL.....	38
5.8	RAILWAY — COLOCAÇÃO EM PRODUÇÃO	40
5.9	JUSTIFICAÇÃO GERAL DAS ESCOLHAS TECNOLÓGICAS	41
6	ATIVIDADES DESENVOLVIDAS	44
6.1	MODELAÇÃO DO SISTEMA E ESTRUTURA DE DADOS	44
6.1.1	<i>Implementação do esquema relacional.....</i>	45
6.1.2	<i>Modelo relacional, entidades principais e Diagrama Entidade-Relação</i>	46
6.2	GESTÃO DE CONTA, AUTENTICAÇÃO E SEGURANÇA	47
6.2.1	<i>Registo, confirmação de conta e recuperação de palavra-passe</i>	47
6.2.2	<i>Gestão de sessões, permissões e proteção de acessos</i>	49
6.2.3	<i>Auditoria, rate limiting e validação de operações</i>	49
6.3	GESTÃO DE PRODUTOS, PERFIS E INVENTÁRIO.....	50
6.3.1	<i>Catálogo global de produtos e categorias</i>	51
6.3.2	<i>Perfil do produtor e associação de produtos</i>	51
6.3.3	<i>Gestão de stocks, movimentos e previsões de produção.....</i>	52
6.3.4	<i>Cálculo de disponibilidade, stock crítico e excedentes.....</i>	53
6.4	MARKETPLACE COOPERATIVO E NECESSIDADES	54
6.4.1	<i>Publicação de anúncios e pré-venda.....</i>	55
6.4.2	<i>Gestão de necessidades e respostas privadas</i>	55
6.4.3	<i>Listagem, pesquisa, filtros e detalhe de anúncios</i>	56

6.4.4	<i>Cobertura de necessidades e regras operacionais</i>	57
6.5	SISTEMA DE RECOMENDAÇÕES E APOIO À DECISÃO	58
6.5.1	<i>Lógica geral do sistema de recomendações</i>	59
6.5.2	<i>Recomendações de compra e venda</i>	60
6.5.3	<i>Combinação de ofertas e divisão por produtores</i>	62
6.5.4	<i>Criação automática de fluxos operacionais</i>	63
6.6	GESTÃO DE ENCOMENDAS E FLUXOS OPERACIONAIS	64
6.6.1	<i>Criação de encomendas</i>	64
6.6.2	<i>Grupos de encomendas e relações entre entidades</i>	65
6.6.3	<i>Workflow de estados e timeline operacional</i>	66
6.6.4	<i>Reservas, receção e atualização de stock</i>	67
6.7	ALERTAS, NOTIFICAÇÕES E COMUNICAÇÃO	68
6.7.1	<i>Alertas operacionais e deduplicação</i>	68
6.7.2	<i>Notificações informativas</i>	70
6.7.3	<i>Mensagens entre produtores e comunicação em tempo real</i>	70
6.7.4	<i>Gestão de anexos e conversas</i>	72
6.8	SUORTE, ADMINISTRAÇÃO E MONITORIZAÇÃO	73
6.8.1	<i>Sistema de suporte ao utilizador</i>	73
6.8.2	<i>Dashboard do produtor e painel administrativo</i>	75
6.8.3	<i>Auditoria e registo técnico de operações</i>	77
6.9	COLOCAÇÃO EM PRODUÇÃO E OPERAÇÃO DA PLATAFORMA	78
6.9.1	<i>Organização da infraestrutura</i>	78
6.9.2	<i>Deploy da aplicação e serviços associados</i>	80
6.9.3	<i>Configuração operacional e execução em produção</i>	80
7	RESULTADOS OBTIDOS	82
7.1	SÍNTESE DAS FUNCIONALIDADES IMPLEMENTADAS	82
7.2	RESULTADOS TÉCNICOS OBTIDOS	83
7.3	TESTES E VALIDAÇÃO FUNCIONAL	84
7.4	FEEDBACK RECOLHIDO E AJUSTES REALIZADOS	85
7.5	ANÁLISE CRÍTICA FACE AOS OBJETIVOS PROPOSTOS	87
8	CONCLUSÃO	89
8.1	PRINCIPAIS RESULTADOS E CONTRIBUIÇÕES	89
8.2	DESAFIOS ENFRENTADOS E LIMITAÇÕES DA SOLUÇÃO	90
8.3	TRABALHO FUTURO E REFLEXÃO FINAL	92
9	REFERÊNCIAS BIBLIOGRÁFICAS	94

ANEXO A	— REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS	97
A.1	REQUISITOS FUNCIONAIS	97
A.1.1	Autenticação, registo e gestão de conta	97
A.1.2	Perfil do produtor e produtos produzidos	98
A.1.3	Catálogo de produtos	98
A.1.4	Gestão de stocks, necessidades e previsões	98
A.1.5	Pedidos externos de clientes	99
A.1.6	Marketplace interno	100
A.1.7	Ofertas dirigidas a necessidades e privacidade no marketplace	101
A.1.8	Recomendações e apoio à decisão	101
A.1.9	Encomendas	103
A.1.10	Alertas e notificações	104
A.1.11	Mensagens entre produtores	104
A.1.12	Suporte ao utilizador	105
A.1.13	Administração, auditoria e integração	106
A.2	REQUISITOS NÃO FUNCIONAIS	106
A.2.1	Segurança e controlo de acesso	106
A.2.2	Privacidade	107
A.2.3	Usabilidade e adequação ao utilizador	108
A.2.4	Fiabilidade e disponibilidade	108
A.2.5	Rastreabilidade e auditoria	109
A.2.6	Manutenibilidade e modularidade	109
A.2.7	Integração e interoperabilidade	110
A.2.8	Comunicação em tempo real e desempenho funcional	110
A.2.9	Armazenamento e gestão de ficheiros	111
A.2.10	Operação em ambiente de produção	111
A.3	SÍNTESE	112
ANEXO B	— DIAGRAMA ENTIDADE-RELAÇÃO	112
ANEXO C	— TABELAS	117

Índice de Figuras

FIGURA 1.1 — CRONOGRAMA RESUMIDO DO PLANO DE TRABALHO DO ESTÁGIO.....	5
FIGURA 2.1 — EXEMPLO DE INTERFACE DA PLATAFORMA AGRIVI 360 FARM ENTERPRISE.....	10
FIGURA 2.2 — EXEMPLO DE PÁGINA DE PLANOS E FUNCIONALIDADES DA FARMBRITE	11
FIGURA 2.3 — EXEMPLO DE SOLUÇÃO ERP AGRÍCOLA LEVRIDGE BASEADA EM MICROSOFT DYNAMICS 365 FONTE: LEVRIDGE, N.D. / MICROSOFT APPSOURCE, N.D.	12
FIGURA 4.1 — ARQUITETURA GERAL DA PLATAFORMA VISION4FARMS: COMPONENTES, MÓDULOS FUNCIONAIS E SERVIÇOS EXTERNOS	23
FIGURA 4.2 — FLUXO DE UM PEDIDO HTTP NO PADRÃO MVT DO DJANGO, COM PERCURSO PELAS CAMADAS DE ENCAMINHAMENTO, LÓGICA, DADOS E APRESENTAÇÃO.....	26
FIGURA 4.3 — ORGANIZAÇÃO DO MODELO DE DADOS EM DOMÍNIOS BASE E DOMÍNIOS OPERACIONAIS	28
FIGURA 4.4 — PERCURSO DE UMA MENSAGEM EM TEMPO REAL: WEBSOCKET, DJANGO CHANNELS, REDIS E ENTREGA AO DESTINATÁRIO	29
FIGURA 5.1 — AMBIENTE DE DESENVOLVIMENTO NO VISUAL STUDIO CODE.....	32
FIGURA 5.2 — REPOSITÓRIO GITHUB DO PROJETO	33
FIGURA 5.3 — PROTÓTIPO DE ALTA FIDELIDADE CRIADO COM APOIO DO GOOGLE AI STUDIO.....	33
FIGURA 5.4 — MODELO RELACIONAL DA PLATAFORMA NO PGADMIN 4	35
FIGURA 5.5 — EXEMPLO DE INTERFACE DESENVOLVIDA COM DJANGO TEMPLATES, BOOTSTRAP E HTMX.....	37
FIGURA 5.6 — EXEMPLO DE INTERFACE DESENVOLVIDA COM CHART.JS.....	37
FIGURA 5.7 — EXEMPLO DE EMAIL TRANSACIONAL DE RECUPERAÇÃO DE PALAVRA-PASSE ENVIADO ATRAVÉS DO RESEND.	40
FIGURA 5.8 — CONFIGURAÇÃO DOS SERVIÇOS DE MEDIA, ESTÁTICOS E VARIÁVEIS DE AMBIENTE	40
FIGURA 5.9 — SERVIÇOS CONFIGURADOS NO RAILWAY PARA EXECUÇÃO DA PLATAFORMA.....	41
FIGURA 6.1 — VISÃO GERAL DO MODELO DE DADOS DA PLATAFORMA, ORGANIZADO POR DOMÍNIOS FUNCIONAIS	47
FIGURA 6.2 — PÁGINA DE REGISTO DE CONTA DE PRODUTOR	48
FIGURA 6.3 — REGISTOS DE AUDITORIA NO PAINEL ADMINISTRATIVO	50
FIGURA 6.4 — DETALHE DE UM PRODUTO NO INVENTÁRIO: INDICADORES DE STOCK, HISTÓRICO DE MOVIMENTOS E AÇÕES RÁPIDAS.....	53
FIGURA 6.5 — ESTADO DO INVENTÁRIO DO PRODUTOR COM INDICADORES DE DISPONIBILIDADE, RESERVAS E COMPROMISSOS EXTERNOS	54
FIGURA 6.6 — FEED DO MARKETPLACE COOPERATIVO COM ANÚNCIOS DE VENDA E PROCURAS AGREGADAS	57

FIGURA 6.7 — RECOMENDAÇÃO DE COMPRA APRESENTADA AO PRODUTOR COM BASE EM DÉFICES E EXCEDENTES DO INVENTÁRIO	59
FIGURA 6.8 — FLUXO DE DECISÃO DO SISTEMA DE RECOMENDAÇÕES: DO ESTADO TEMPORAL DO INVENTÁRIO À CRIAÇÃO DO FLUXO OPERACIONAL.....	60
FIGURA 6.9 — RECOMENDAÇÃO DE COMPRA APRESENTADA AO PRODUTOR COM BASE NO DÉFICE TEMPORAL DO INVENTÁRIO	62
FIGURA 6.10 — DETALHE DE UMA ENCOMENDA E TIMELINE DE ESTADOS	67
FIGURA 6.11 — PÁGINA DE ALERTAS OPERACIONAIS COM SEPARADORES E SECÇÕES TEMÁTICAS	69
FIGURA 6.12 — FLUXO DE ENVIO DE MENSAGEM EM TEMPO REAL.....	71
FIGURA 6.13 — CONVERSA ENTRE PRODUTORES COM COMUNICAÇÃO EM TEMPO REAL E ENVIO DE ANEXOS	73
FIGURA 6.14 — GESTÃO ADMINISTRATIVA/CONVERSA DE UM TICKET DE SUPORTE COM HISTÓRICO DE CONVERSA	74
FIGURA 6.15 — PAINEL DO PRODUTOR COM ALERTAS, INDICADORES DE STOCK, CLIMA E AÇÕES RECOMENDADAS	76
FIGURA 6.16 — PAINEL ADMINISTRATIVO COM MÉTRICAS GLOBAIS, SUPORTE ATIVO E ATIVIDADE COMERCIAL.....	77
FIGURA 6.17 — INFRAESTRUTURA DE PRODUÇÃO NO RAILWAY: SERVIÇOS INTERNOS, DEPLOY AUTOMÁTICO A PARTIR DO GITHUB E INTEGRAÇÕES EXTERNAS	79
FIGURA B.1 — VISÃO GERAL DO MODELO RELACIONAL	113
FIGURA B.2 — RELAÇÕES ENTRE CONTAS, PERFIS, CATÁLOGO, INVENTÁRIO E NECESSIDADES	114
FIGURA B.3 — RELAÇÕES ENTRE MARKETPLACE, RECOMENDAÇÕES, ENCOMENDAS, ALERTAS E NOTIFICAÇÕES	115
FIGURA B.4 — RELAÇÕES DOS MÓDULOS DE COMUNICAÇÃO, SUPORTE, AUDITORIA E INTEGRAÇÃO	116

Índice de Tabelas

TABELA C.1 — COMPARAÇÃO ENTRE SOLUÇÕES DIGITAIS AGRÍCOLAS ANALISADAS.	117
TABELA C.2 — SÍNTESE DAS FUNCIONALIDADES IMPLEMENTADAS	119
TABELA C.3 — PRINCIPAIS RESULTADOS TÉCNICOS OBTIDOS	120
TABELA C.4 — SÍNTESE DOS FLUXOS FUNCIONAIS VALIDADOS.....	121
TABELA C.5 — FEEDBACK CONSIDERADO E AJUSTES REALIZADOS.....	122

Lista de siglas / abreviaturas

Sigla / Acrónimo	Significado
API	Application Programming Interface — Interface de Programação de Aplicações
ASGI	Asynchronous Server Gateway Interface — Interface assíncrona entre servidor web e aplicação Python
B2B	Business to Business — Relação comercial entre empresas ou entidades profissionais
BD	Base de Dados
CDN	Content Delivery Network — Rede de distribuição de conteúdos
CRUD	Create, Read, Update, Delete — Criar, Ler, Atualizar, Eliminar
CSRF	Cross-Site Request Forgery — Tipo de ataque que força ações não autorizadas num utilizador autenticado
CSS	Cascading Style Sheets — Folhas de Estilo em Cascata
DER	Diagrama de Entidade-Relação
ERD	Entity Relationship Diagram — Diagrama de Entidade-Relação
ERP	Enterprise Resource Planning

HTML	HyperText Markup Language — Linguagem de Marcação de Hipertexto
HTTP	HyperText Transfer Protocol — Protocolo de Transferência de Hipertexto
HTMX	Biblioteca utilizada para adicionar interações dinâmicas em páginas HTML através de pedidos HTTP
IA	Inteligência Artificial
IDE	Integrated Development Environment — Ambiente Integrado de Desenvolvimento
JSON	JavaScript Object Notation — Notação de Objetos JavaScript
MVP	Minimum Viable Product — Produto Mínimo Viável
MVC	Model-View-Controller — Modelo-Vista-Controlador
MVT	Model-View-Template — Modelo-Vista-Template, padrão utilizado pelo Django
NIF	Número de Identificação Fiscal
ORM	Object-Relational Mapping — Mapeamento Objeto-Relacional
RF	Requisito Funcional
RNF	Requisito Não Funcional
SaaS	Software as a Service — Software como Serviço
SMTP	Simple Mail Transfer Protocol — Protocolo Simples de Transferência de Correio Eletrônico

SQL	Structured Query Language — Linguagem de Consulta Estruturada
TLS	Transport Layer Security — Protocolo de segurança para comunicações cifradas
UI	User Interface — Interface do Utilizador
UML	Unified Modeling Language — Linguagem de Modelação Unificada
URL	Uniform Resource Locator — Localizador Uniforme de Recursos
UUID	Universally Unique Identifier — Identificador Único Universal
UX	User Experience — Experiência do Utilizador
WebSocket	Protocolo de comunicação bidirecional entre cliente e servidor em tempo real
WSGI	Web Server Gateway Interface — Interface tradicional entre servidor web e aplicação Python

Glossário

Alerta — Notificação gerada automaticamente pelo sistema com base em condições operacionais, como stock crítico, excedente disponível, necessidade sem cobertura ou oportunidade de pré-venda.

Anúncio — Publicação criada por um produtor no marketplace, podendo estar associada a stock atual, pré-venda ou oferta dirigida a uma necessidade específica.

Cobertura de necessidade — Parte da quantidade pretendida numa necessidade que já se encontra assegurada através de propostas, recomendações ou encomendas associadas. Permite classificar uma necessidade como aberta, parcialmente coberta ou coberta.

Compromisso externo — Quantidade de um produto que um produtor assumiu entregar aos seus próprios clientes, com base em pedidos externos registados na plataforma. É considerada nos cálculos de disponibilidade e de stock crítico.

Défice temporal — Situação em que o stock disponível e a produção futura útil de um produtor não são suficientes para cumprir, dentro do prazo, os compromissos externos assumidos para um produto.

Encomenda — Registo de uma transação comercial efetuada entre produtores, contendo informação sobre produtos, quantidades, preços, estados e dados logísticos.

Excedente — Quantidade de produto disponível que ultrapassa o nível mínimo operacional definido para um produtor, podendo ser disponibilizada para venda ou redistribuição.

Grupo de encomendas — Estrutura que agrega várias sub-encomendas associadas ao mesmo comprador, normalmente resultantes de uma recomendação dividida por vários vendedores ou origens.

Marketplace interno — Ambiente digital integrado na plataforma que permite a compra e venda de produtos entre produtores pertencentes à mesma rede cooperativa.

Mensagem em tempo real — Comunicação enviada entre produtores com atualização imediata da interface através de WebSocket, podendo incluir texto, eventos do sistema ou ficheiros anexos.

Movimento de stock — Registo histórico de alterações na quantidade de um produto, resultantes de entradas, saídas, reservas, ajustes ou cobertura de necessidades.

Necessidade — Situação em que um produtor não possui quantidade suficiente de determinado produto, declarando uma procura que pode vir a ser coberta por recomendações, anúncios privados ou encomendas.

Oferta dirigida — Anúncio privado criado em resposta a uma necessidade específica, visível apenas para o produtor que o criou e para o produtor dono dessa necessidade.

Pedido externo de cliente — Pedido registado por um produtor referente a um cliente seu, externo à cooperativa. O conjunto destes pedidos representa a procura real do produtor e pode dar origem a uma necessidade agregada quando existe défice temporal.

Pré-venda — Modalidade de anúncio baseada em previsão de produção futura, permitindo disponibilizar no marketplace quantidades ainda não presentes no stock atual.

Produtor — Utilizador da plataforma associado a um perfil funcional, responsável pela gestão de produtos, stocks, necessidades, anúncios e operações comerciais.

Recomendação — Sugestão automática gerada pelo sistema com base em necessidades, stocks, previsões e anúncios disponíveis, com o objetivo de apoiar a tomada de decisão do utilizador.

Stock — Quantidade atual de determinado produto registada para um produtor, podendo incluir valores disponíveis, reservados e limiares operacionais.

Stock disponível — Quantidade efetivamente utilizável de um produto, obtida pela diferença entre a quantidade atual e a quantidade reservada.

Suporte — Módulo da plataforma que permite ao utilizador criar pedidos de ajuda ou esclarecimento, acompanhados através de tickets geridos pela administração.

Ticket de suporte — Pedido de apoio criado por um utilizador para reportar uma dúvida, problema ou necessidade de assistência dentro da plataforma.

WebSocket — Tecnologia de comunicação bidirecional persistente entre cliente e servidor, utilizada para entrega em tempo real de mensagens e atualização imediata de informação na interface.

1 Introdução

O presente capítulo apresenta o enquadramento geral do estágio curricular realizado no âmbito da Licenciatura em Engenharia Informática da Escola Superior de Tecnologia e Gestão de Viseu. Ao longo deste capítulo são apresentados o contexto que motivou o desenvolvimento do projeto, a entidade de acolhimento, os objetivos definidos, o plano de trabalho seguido e a forma como o relatório se encontra organizado.

A Engenharia Informática desempenha um papel crescente na resolução de problemas operacionais em diversos setores. No caso do setor agrícola, a partilha de informação entre produtores pode ter um impacto significativo na gestão de recursos. Numa cooperativa agrícola, diferentes produtores podem ter produtos disponíveis em excesso, enquanto outros apresentam necessidades por satisfazer. Quando esta informação não se encontra centralizada, torna-se difícil identificar oportunidades de colaboração, responder a necessidades em tempo útil e aproveitar os recursos existentes. Foi a partir deste problema que se definiu a necessidade de criar uma solução digital capaz de apoiar a articulação entre produtores.

O estágio decorreu no ano letivo de 2025/2026, na empresa Visionary Devotion Unipessoal Lda., entre 21 de fevereiro de 2026 e 5 de junho de 2026. O trabalho realizado teve como objetivo principal o desenvolvimento de uma plataforma web para apoio a uma cooperativa agrícola. Esta plataforma permite gerir produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e pedidos de suporte, reunindo num único sistema os principais fluxos operacionais da cooperativa. Os objetivos académicos e técnicos do estágio são apresentados em detalhe na Secção 1.2.

A solução desenvolvida enquadra-se na digitalização de processos agrícolas e cooperativos, procurando transformar um problema operacional numa aplicação web funcional e compreensível para os seus utilizadores. A plataforma foi concebida de forma modular, respondendo aos requisitos

definidos para o âmbito do estágio e mantendo margem para evolução futura em função das necessidades da cooperativa.

1.1 Entidade de Acolhimento

O estágio curricular foi realizado na empresa **Visionary Devotion Unipessoal Lda.**, entidade que desenvolve a sua atividade comercial sob a marca **Vision4You**. Trata-se de uma empresa tecnológica sediada em Vale de Cambra, constituída em 1 de setembro de 2017, cuja atividade se enquadra na área da consultoria e implementação de soluções digitais adaptadas às necessidades das organizações. (Racius, s.d.).

De acordo com a informação disponibilizada pela entidade de acolhimento, a Vision4You foca-se na implementação de soluções digitais ajustadas a cada organização, procurando promover eficiência, crescimento sustentável e novos modelos de negócio assentes na tecnologia. A empresa apresenta ainda como ambição contribuir para uma sociedade melhor através da tecnologia digital, incentivando redes colaborativas de inovação, criatividade e talento.

A atividade da Vision4You organiza-se em três áreas principais. A primeira é a inovação de negócios, associada à definição de estratégias de Tecnologias de Informação e à implementação de recursos digitais capazes de impulsionar novos modelos de negócio. A segunda é a inovação tecnológica, que inclui a aplicação estratégica de inteligência artificial, a valorização da decisão baseada em dados, a otimização de processos e a articulação entre automação e fator humano. A terceira é a inovação para a sustentabilidade, centrada em projetos de inovação social e colaborativa com impacto positivo no negócio, nas pessoas e no planeta.

No âmbito deste estágio, a empresa propôs o desenvolvimento de uma plataforma *web* de apoio a uma cooperativa agrícola. A proposta surgiu da necessidade de criar um sistema capaz de auxiliar produtores na gestão dos seus produtos, *stocks*, necessidades e excedentes, bem como nas transações internas, promovendo uma melhor organização da informação e uma articulação mais eficiente entre os intervenientes.

Ao longo do período de estágio, a empresa acompanhou ativamente o projeto. Através de reuniões e momentos de validação, foram clarificados requisitos, ajustadas prioridades e avaliadas as funcionalidades implementadas. Este acompanhamento permitiu manter o desenvolvimento alinhado com as necessidades reais do sistema e com os objetivos definidos.

Importa ainda referir que a Visionary Devotion Unipessoal Lda. não assumiu apenas o papel de entidade de acolhimento, mas também de entidade promotora do problema a resolver. O contacto com a empresa permitiu compreender o contexto funcional pretendido, transformar necessidades iniciais em requisitos concretos e desenvolver uma aplicação ajustada ao ambiente para o qual foi pensada.

1.2 Objetivos e Resultados Esperados

Do ponto de vista académico, o estágio procurou complementar a formação teórico-prática adquirida ao longo da licenciatura, promover a integração no mundo profissional e consolidar hábitos de trabalho e autonomia. Do ponto de vista do projeto, o objetivo principal consistiu no desenvolvimento de uma plataforma web para apoio a uma cooperativa agrícola, capaz de centralizar informação operacional dos produtores e apoiar processos de gestão, compra, venda e comunicação dentro da rede cooperativa.

De forma mais específica, foram definidos os seguintes objetivos:

- identificar os requisitos funcionais a partir do contexto e das necessidades da entidade de acolhimento;
- fundamentar as decisões de desenho da solução com base na análise de plataformas existentes no setor;
- seleccionar tecnologias adequadas ao desenvolvimento da aplicação;
- criar um protótipo de alta fidelidade para orientar a interface da plataforma;
- desenhar um modelo relacional capaz de suportar os principais fluxos do sistema;
- implementar uma aplicação web modular com autenticação, perfis de produtor, catálogo, stocks, necessidades, marketplace, recomendações, encomendas, alertas, mensagens e suporte;
- garantir a persistência, organização e integridade dos dados através de uma base de dados relacional;
- colocar a aplicação em ambiente de produção;
- consolidar competências técnicas e profissionais na área do desenvolvimento de software.

Como resultado esperado, pretendia-se obter uma plataforma funcional, organizada e preparada para evolução futura. A solução deveria permitir aos produtores gerir produtos e stocks, identificar necessidades e excedentes, publicar anúncios, receber recomendações, criar encomendas e comunicar com outros utilizadores da plataforma. A organização modular da aplicação permite que a solução possa evoluir de forma gradual, acompanhando futuras alterações nos requisitos da cooperativa.

1.3 Plano de Trabalho

O plano de trabalho foi organizado de forma progressiva, acompanhando a evolução natural do projeto. Numa primeira fase, foi realizado o estudo inicial do problema, da entidade de acolhimento e do setor agrícola cooperativo. Esta etapa permitiu compreender o contexto em que a solução seria aplicada e identificar as principais necessidades funcionais.

Seguidamente, foi realizado um estudo de mercado com o objetivo de analisar soluções semelhantes, perceber as suas funcionalidades e identificar lacunas. Esta análise ajudou a clarificar o posicionamento da plataforma e a perceber que o projeto não deveria limitar-se a um *marketplace*, mas sim integrar gestão de stocks, necessidades, recomendações e transações internas.

Após esta fase, procedeu-se ao estudo e seleção das tecnologias. Foram escolhidas ferramentas adequadas ao desenvolvimento de uma aplicação *web* modular, com base de dados relacional, interface simples e capacidade de funcionamento em ambiente de produção.

Numa fase posterior, foi criado o protótipo de alta fidelidade e desenhado o modelo relacional da base de dados. A implementação iniciou-se pelos módulos estruturais, como autenticação, perfis, catálogo e stocks, avançando depois para necessidades, *marketplace*, recomendações, encomendas, alertas, mensagens, suporte e administração. A Figura 1.1 apresenta o cronograma resumido das principais fases do plano de trabalho.

Esta organização permitiu desenvolver o projeto de forma incremental, respeitando as dependências entre módulos. Por exemplo, o *marketplace* dependia da existência de produtos e stocks; as recomendações dependiam de necessidades e anúncios; e as encomendas dependiam dos fluxos de compra previamente implementados. A validação funcional foi sendo realizada ao longo do desenvolvimento, o que permitiu ajustar comportamentos, corrigir problemas e garantir que cada módulo respondia aos objetivos definidos antes de avançar para o seguinte.

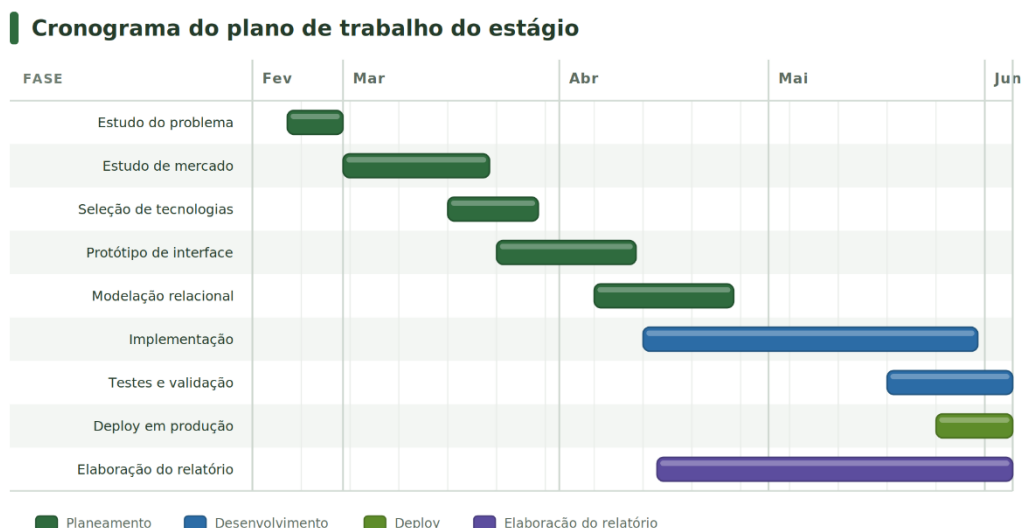


Figura 1.1 — Cronograma resumido do plano de trabalho do estágio.

1.4 Estrutura do Relatório

O presente relatório encontra-se organizado em oito capítulos, seguidos das referências bibliográficas e dos anexos.

O primeiro capítulo corresponde à introdução, onde é apresentado o enquadramento do estágio, a entidade de acolhimento, os objetivos definidos, o plano de trabalho e a estrutura do relatório.

O segundo capítulo apresenta o estado da arte, abordando a transformação digital no setor agrícola, as plataformas de gestão e comercialização agrícola, os sistemas de apoio à decisão e os modelos de previsão aplicados à agricultura. Este capítulo permite enquadrar o projeto no contexto de soluções e estudos existentes.

O terceiro capítulo descreve a metodologia de trabalho adotada ao longo do estágio, desde o estudo inicial do problema até à organização do desenvolvimento e validação progressiva da solução.

O quarto capítulo apresenta a arquitetura geral da solução, descrevendo a organização da aplicação, o padrão arquitetural utilizado, o papel da base de dados, o modelo de dados, a comunicação em tempo real e as decisões relacionadas com segurança e configuração.

O quinto capítulo apresenta as tecnologias e ferramentas utilizadas, justificando as escolhas tecnológicas adotadas nas áreas de desenvolvimento, base de dados, interface, comunicação em tempo real, serviços externos e colocação em produção.

O sexto capítulo descreve as atividades desenvolvidas durante o estágio. Neste capítulo são apresentados os principais módulos implementados, incluindo a modelação do sistema, autenticação, catálogo, inventário, marketplace, recomendações, encomendas, alertas, mensagens, suporte, administração e colocação em produção.

O sétimo capítulo apresenta os resultados obtidos, incluindo as funcionalidades implementadas, os resultados técnicos alcançados, a validação funcional e a análise crítica face aos objetivos propostos.

O oitavo capítulo apresenta a conclusão, refletindo sobre os principais resultados e contribuições do estágio, as limitações da solução e as perspetivas de trabalho futuro.

As referências bibliográficas reúnem as fontes consultadas ao longo do relatório. Os anexos incluem documentação complementar, nomeadamente os requisitos funcionais atualizados, o esquema SQL da base de dados, o diagrama entidade-relação e exemplos de interface da plataforma.

2 Estado da Arte

A transformação digital do setor agrícola tem vindo a alterar de forma significativa a forma como os produtores planeiam, registam, monitorizam e comercializam a sua produção. No contexto da Agricultura 4.0, os sistemas digitais passaram a assumir um papel central na recolha, organização e exploração de dados operacionais, permitindo apoiar decisões relacionadas com planeamento, rastreabilidade, eficiência de recursos e coordenação entre diferentes intervenientes da cadeia agroalimentar. Contudo, a literatura sublinha que o impacto destas tecnologias não depende apenas da sua sofisticação técnica, mas também da acessibilidade, da interoperabilidade entre sistemas e da adequação das soluções aos contextos reais de utilização agrícola (Trendov et al., 2019; Dawson Jr. et al., 2024).

Neste enquadramento, o presente capítulo analisa o estado da arte relativo à transformação digital em cooperativas agrícolas, às plataformas de gestão e comercialização agrícola, aos sistemas de apoio à decisão e aos modelos de previsão aplicados à agricultura. O objetivo desta análise consiste em identificar as principais abordagens atualmente existentes, compreender as suas vantagens e limitações e evidenciar as lacunas que justificam o desenvolvimento de soluções mais integradas, especialmente em contextos cooperativos.

2.1 Cooperativas Agrícolas e a Transformação Digital

As cooperativas agrícolas desempenham um papel relevante na organização do setor, nomeadamente através da agregação de produtores, da partilha de serviços e da coordenação comercial. Neste contexto, a digitalização pode reforçar a sua capacidade de gestão, melhorar a ligação com os membros, aumentar a transparência operacional e criar novos mecanismos de criação de valor baseados em plataformas digitais. Estudos recentes sugerem que as cooperativas não devem ser vistas apenas como utilizadoras de tecnologias, mas também como potenciais orquestradoras de ecossistemas digitais capazes de articular dados, relações entre membros e mecanismos de coordenação económica (Dawson Jr. et al., 2024).

Apesar desse potencial, a transformação digital em contexto cooperativo continua a enfrentar obstáculos significativos. A literatura destaca fatores como a heterogeneidade dos produtores, a limitada literacia digital, a escassez de investimento, a fraca interoperabilidade entre sistemas e a dificuldade em converter dados operacionais em ações práticas para os utilizadores (Trendov et al., 2019). Neste sentido, a digitalização de cooperativas agrícolas não deve ser entendida apenas como informatização de processos administrativos, mas antes como uma reconfiguração da relação entre produção, informação e transação — o que torna particularmente relevante analisar soluções que apoiem não só o registo interno da exploração, mas também a redistribuição de recursos, a coordenação entre membros e a tomada de decisão baseada em dados partilhados (Dawson Jr. et al., 2024; Trendov et al., 2019).

2.2 Plataformas de Gestão e Comercialização Agrícola

A literatura distingue, de forma geral, duas grandes famílias de soluções digitais no setor agrícola. A primeira corresponde aos Sistemas de Informação de Gestão Agrícola (Farm Management Information Systems — FMIS), orientados para a gestão operacional da exploração, incluindo planeamento, registo de atividades, rastreabilidade, gestão de recursos e análise de desempenho (Fountas et al., 2015). A segunda corresponde às plataformas digitais de comercialização, como marketplaces agrícolas, soluções de comércio eletrónico agroalimentar ou plataformas de apoio a cadeias curtas de abastecimento, que privilegiam a visibilidade da oferta, a gestão de encomendas e a relação entre produtores e compradores (Zeng et al., 2017; Burgess & Sunmola, 2022).

As revisões de literatura mostram que os FMIS evoluíram de ferramentas relativamente simples de registo para sistemas mais sofisticados de suporte à gestão agrícola. No entanto, também demonstram que existe uma grande heterogeneidade funcional entre plataformas, bem como limitações persistentes em termos de usabilidade, custo, padronização e interoperabilidade. Em particular, muitos FMIS revelam-se fortes na vertente operacional, mas menos eficazes na ligação entre informação da exploração e ação comercial imediata (Fountas et al., 2015).

Por outro lado, a literatura sobre comércio eletrónico agroalimentar evidencia que estas plataformas têm potencial para reduzir intermediação, melhorar o acesso ao mercado e facilitar a relação entre produtores e compradores (Zeng et al., 2017; Ma et al., 2024). No contexto específico das cadeias curtas de abastecimento, Burgess e Sunmola (2022) analisam requisitos de qualidade associados a plataformas digitais, destacando a importância de visibilidade, fluxo de informação em tempo real e adequação às necessidades dos intervenientes. Ainda assim, estas soluções nem sempre se encontram ligadas a inventário real, a dados operacionais atualizados ou a mecanismos automáticos de apoio à decisão, o que limita a capacidade de resposta a necessidades concretas e a utilização de informação operacional em tempo útil.

2.2.1 AGRIVI

O AGRIVI constitui uma referência importante no domínio das plataformas de gestão agrícola. A solução apresenta-se como uma plataforma digital orientada para a gestão da exploração, com funcionalidades de planeamento, registo de operações, rastreabilidade, análise económica e integração com sensores e sistemas empresariais. A Figura 2.1 apresenta um exemplo da interface da plataforma AGRIVI 360 Farm Enterprise.

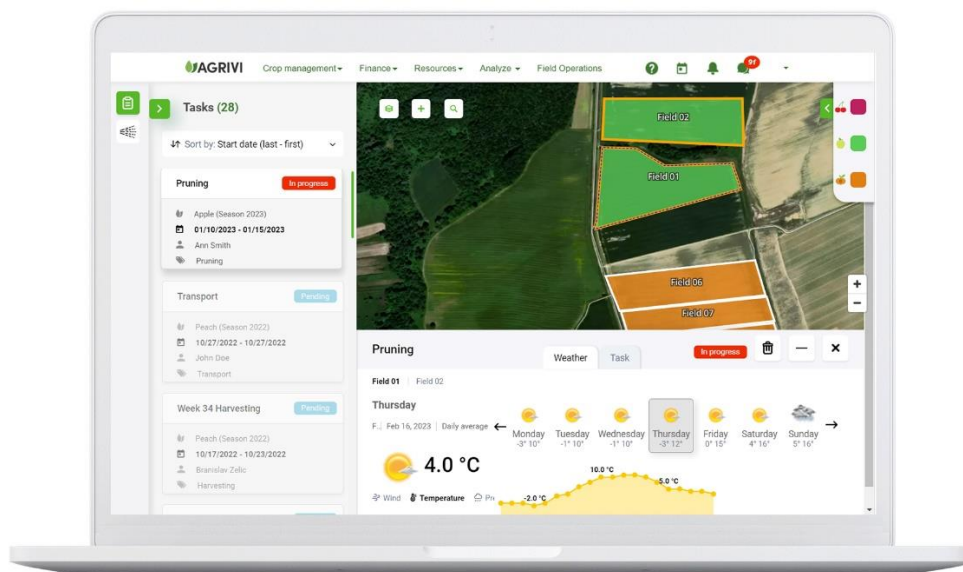


Figura 2.1 — Exemplo de interface da plataforma AGRIVI 360 Farm Enterprise

Relativamente à acessibilidade, a informação pública disponível no website da AGRIVI não apresenta preços diretos, remetendo o utilizador para marcação de demonstração. Este posicionamento sugere uma orientação mais empresarial, direcionada para explorações profissionais e organizações de maior dimensão (AGRIVI, n.d.).

Do ponto de vista do problema em estudo, o AGRIVI evidencia uma clara vocação para monitorização e gestão agronómica, mas não tem como foco principal a redistribuição interna de recursos entre produtores pertencentes à mesma rede. Esta orientação diferencia-se do problema em estudo, que se centra na articulação operacional entre produtores de uma cooperativa (Fountas et al., 2015; AGRIVI, n.d.).

2.2.2 Farmbrite

A Farmbrite apresenta uma abordagem mais híbrida, combinando gestão da exploração, controlo de inventário, encomendas e loja online. Ao contrário da AGRIVI, disponibiliza preços públicos no seu website, com planos a partir de 29 dólares por mês, e refere a existência de teste gratuito e descontos para novos agricultores e organizações sem fins lucrativos (Farmbrite, n.d.). Esta transparência torna a solução potencialmente mais acessível para pequenos e médios produtores. A Figura 2.2 apresenta um exemplo da página de planos da Farmbrite.

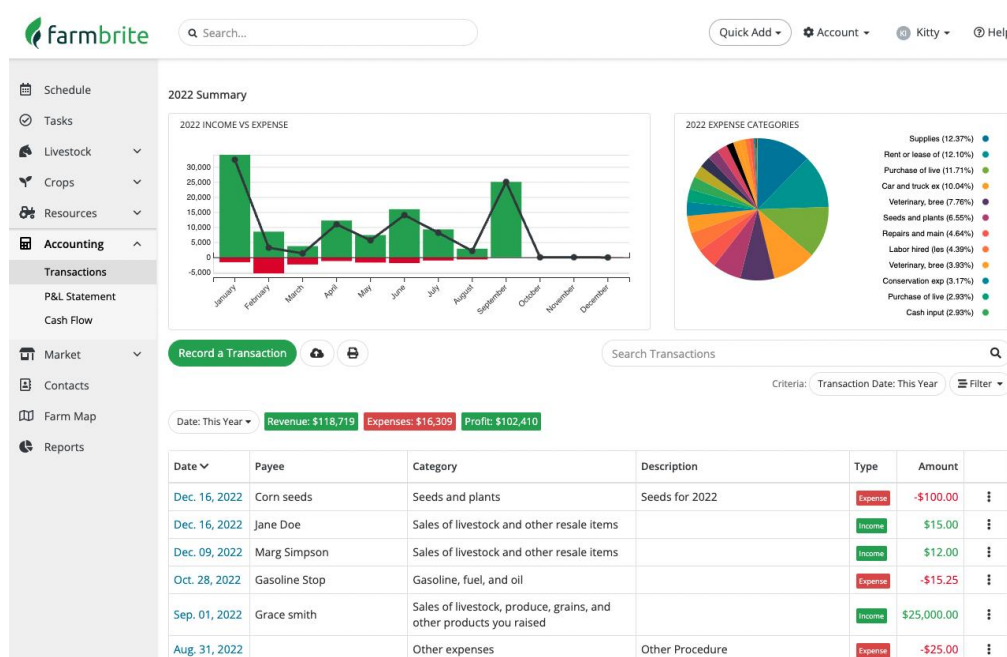


Figura 2.2 — Exemplo de página de planos e funcionalidades da Farmbrite

Ainda assim, a orientação comercial da Farmbrite surge sobretudo associada à venda ao exterior e à relação com clientes, e não à articulação de trocas internas entre produtores de uma mesma cooperativa. Embora constitua um exemplo relevante de aproximação entre inventário e canal de venda, a sua lógica diferencia-se do problema em estudo, que assenta na coordenação interna e no apoio à decisão entre membros da rede cooperativa.

2.2.3 Outras Soluções Identificadas

Para além das plataformas anteriormente referidas, foram identificadas soluções com posicionamentos distintos no ecossistema digital agrícola. Estas soluções não apresentam

exatamente o mesmo foco funcional, mas ajudam a compreender a diversidade de abordagens existentes no mercado.

A Levridge constitui um exemplo de solução empresarial orientada para o agronegócio. Apresenta-se como uma plataforma *cloud* baseada em Microsoft Dynamics 365, com funcionalidades associadas à contabilidade agrícola, gestão de operações, cadeia de abastecimento, vendas, contratos e processos administrativos. O seu posicionamento é claramente *enterprise*, direcionado para cooperativas de maior dimensão e organizações agroindustriais, com preços não disponibilizados publicamente (Levridge, s.d.; Microsoft AppSource, s.d.). A Figura 2.3 apresenta um exemplo da solução Levridge.

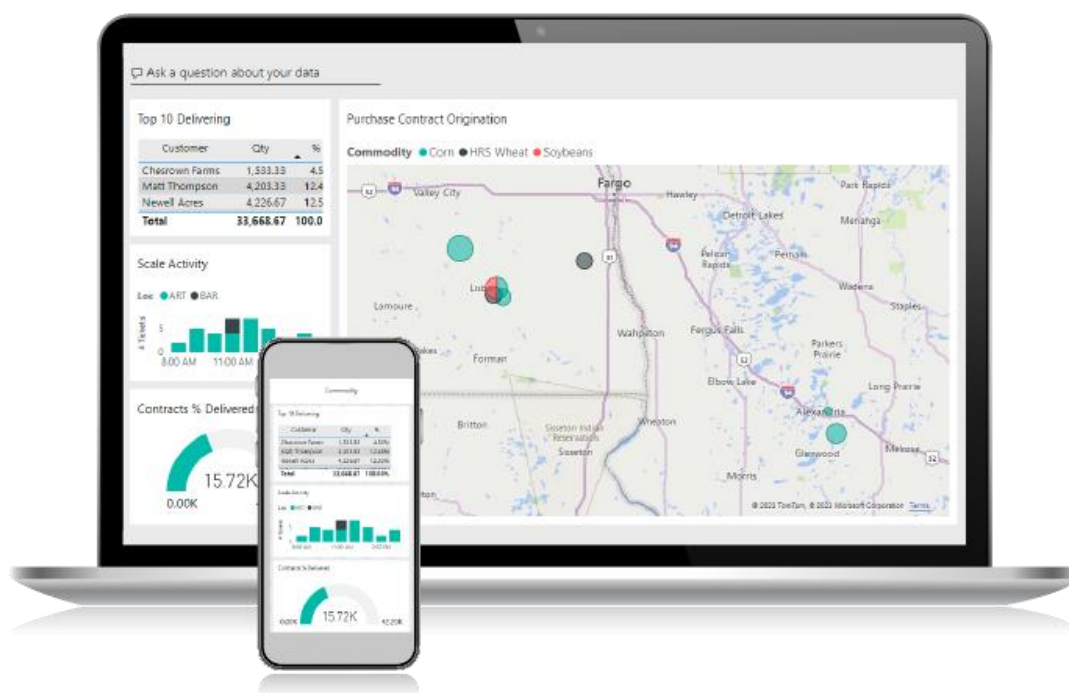


Figura 2.3 — Exemplo de solução ERP agrícola Levridge baseada em Microsoft Dynamics 365 Fonte: Levridge, n.d. / Microsoft AppSource, n.d.

A GrownBy aproxima-se de um modelo cooperativo orientado para venda direta ao consumidor final, privilegiando a relação *farm-to-consumer* e a valorização da proximidade entre produtor e

cliente (GrownBy, s.d.). Plataformas como Bushel e Agri Marketplace, por sua vez, privilegiam a ligação a compradores externos e transações B2B fora da lógica interna da cooperativa (Bushel, s.d.; Agri Marketplace, s.d.). Já fornecedores como a Trimble disponibilizam tecnologia e soluções de gestão agrícola orientadas sobretudo para a operação no terreno e a otimização da produção (Trimble, s.d.).

Esta diversidade demonstra que o ecossistema digital agrícola tem vindo a especializar-se em diferentes dimensões do problema. Algumas soluções são mais fortes na gestão empresarial, outras na venda direta ao consumidor e outras na ligação a compradores externos. No entanto, permanece relativamente rara a articulação coerente entre inventário atualizado, identificação de necessidades, apoio à decisão e transação interna numa rede de produtores.

2.2.4 Análise Comparativa

A síntese comparativa das soluções analisadas é apresentada no Anexo C (Tabela C.1). De seguida, discutem-se os principais contrastes identificados.

O AGRIVI destaca-se pela gestão agronómica e pela integração de dados operacionais. A Farmbrite diferencia-se pela acessibilidade dos preços e pela aproximação entre inventário e venda online. A Levridge apresenta maior robustez empresarial, sobretudo para organizações agroindustriais e cooperativas de maior dimensão. A GrownBy centra-se na ligação direta ao consumidor, enquanto plataformas como Bushel privilegiam transações B2B externas. No entanto, nenhuma destas soluções tem como foco principal a articulação operacional interna entre produtores de uma cooperativa regional.

A solução desenvolvida posiciona-se num espaço intermédio: não pretende substituir sistemas avançados de gestão agronómica nem ERPs agrícolas, mas procura articular inventário, necessidades, anúncios, recomendações e encomendas num fluxo operacional simples e orientado à cooperação entre produtores. É nesta integração funcional, assente em dados operacionais atualizados e em regras de apoio à decisão, que reside a sua principal diferenciação face às soluções analisadas.

2.3 Sistemas de Apoio à Decisão na Agricultura

Os sistemas de apoio à decisão têm vindo a ganhar importância crescente na agricultura digital, sobretudo devido ao aumento da disponibilidade de dados provenientes de sensores, registos operacionais, séries históricas e informação ambiental. A literatura descreve estes sistemas como instrumentos destinados a reduzir incerteza, apoiar escolhas informadas e melhorar a eficiência do uso de recursos, do planeamento e da resposta a problemas operacionais (Trendov et al., 2019).

No domínio da informática, os sistemas de recomendação têm sido amplamente estudados em áreas como comércio eletrónico e plataformas digitais. As abordagens mais conhecidas incluem recomendação baseada em conteúdo, filtragem colaborativa e recomendação sensível ao contexto. Mais recentemente, a investigação tem destacado a relevância de sistemas context-aware, capazes de incorporar múltiplas variáveis situacionais na geração de recomendações (Mateos & Bellogín, 2025). Esta perspetiva é especialmente útil em cenários agrícolas, onde uma recomendação não depende apenas de preferências abstratas, mas também de fatores como quantidade disponível, proximidade, prazo, preço, estado de stock e viabilidade logística.

Em contexto agrícola cooperativo, esta necessidade de contextualização torna-se particularmente importante. Não se trata apenas de recomendar um produto, mas de apoiar decisões entre vários agentes com interesses, restrições e disponibilidades distintas. Por isso, a utilidade de um sistema de recomendação agrícola não depende exclusivamente do modelo algorítmico utilizado, mas também da qualidade e atualidade dos dados, da interpretabilidade dos resultados e da integração com os fluxos reais de operação.

Outro aspeto central apontado pela literatura prende-se com a interoperabilidade entre sistemas. Em agricultura digital, o valor dos dados depende da sua capacidade de serem acedidos, partilhados e reutilizados entre diferentes soluções (Jouanjean et al., 2020; Trendov et al., 2019). Neste domínio, o padrão ADAPT, desenvolvido pela AgGateway, procura responder ao problema da fragmentação de dados agrícolas, oferecendo um esquema de dados sem dependências de software, orientado para a representação e reutilização de dados agrícolas em formatos abertos como JSON, GeoParquet e GeoTIFF (AgGateway, n.d.).

Embora a plataforma desenvolvida neste estágio não implemente diretamente o padrão ADAPT, a sua existência reforça a importância de estruturar dados agrícolas de forma consistente e preparada para futura integração com outros sistemas. De forma simplificada, o ADAPT evidencia que a interoperabilidade é um requisito cada vez mais relevante em plataformas agrícolas, especialmente quando se pretende relacionar dados operacionais, inventário, necessidades e mecanismos de apoio à decisão.

2.4 Modelos de Previsão Aplicados à Agricultura

Os modelos de previsão aplicados à agricultura têm vindo a assumir relevância crescente, sobretudo nas áreas de previsão de rendimento, planeamento e gestão de risco. As revisões sistemáticas recentes mostram que a aprendizagem automática e a aprendizagem profunda são hoje amplamente utilizadas para prever produtividade agrícola, recorrendo a variáveis como temperatura, precipitação, tipo de solo, séries temporais históricas e imagens (van Klompenburg et al., 2020).

Apesar do seu potencial, a literatura identifica várias limitações. Muitos modelos são fortemente dependentes do contexto geográfico, da qualidade dos dados e da disponibilidade de informação histórica consistente (van Klompenburg et al., 2020). Além disso, a passagem da previsão abstrata para o apoio operacional concreto continua a constituir um desafio. Prever produção futura não equivale, por si só, a gerar ações úteis para um utilizador final, como alertar para défices, sinalizar excedentes ou recomendar compras e vendas entre membros de uma cooperativa.

No contexto do projeto desenvolvido, não foi implementado um modelo preditivo baseado em aprendizagem automática. Esta opção resultou das limitações naturais do âmbito do estágio, nomeadamente o tempo disponível, a ausência de dados históricos validados e a necessidade de entregar uma primeira versão funcional da plataforma. Neste relatório, o termo "previsão" é utilizado para designar informação sobre disponibilidade futura de produto, registada na plataforma, e não o resultado de um modelo estatístico ou de aprendizagem automática. A solução adotou uma abordagem operacional baseada em regras e critérios explícitos, mais simples de validar e adequada à fase atual da plataforma. Os detalhes desta lógica são apresentados no Capítulo 6.

Neste sentido, os modelos de previsão devem ser entendidos como componentes de um ecossistema de decisão mais amplo. O seu verdadeiro valor emerge quando a previsão é ligada a inventário, necessidades declaradas, histórico de movimentos, regras de negócio e mecanismos de transação.

A literatura sobre plataformas digitais agroalimentares reforça precisamente esta ideia ao

evidenciar que visibilidade em tempo real, qualidade de dados, rastreabilidade e coordenação entre agentes são fatores críticos para transformar informação em valor operacional (Burgess & Sunmola, 2022).

2.5 Síntese Crítica do Estado da Arte

A análise realizada evidencia avanços relevantes no ecossistema digital agrícola, mas também uma fragmentação funcional persistente. Os sistemas de gestão agrícola revelam maturidade na organização da exploração, na rastreabilidade e na monitorização. As plataformas de comercialização oferecem canais digitais de encomenda, venda e relação com o mercado. Os sistemas de apoio à decisão e os modelos de previsão acrescentam capacidade analítica. No entanto, estas dimensões continuam frequentemente separadas (Fountas et al., 2015; van Klompenburg et al., 2020).

A principal lacuna identificada reside na dificuldade em ligar, de forma coerente, gestão operacional da exploração, deteção de necessidades ou excedentes e ação comercial imediata dentro de uma rede cooperativa. Muitos sistemas registam dados, mas não os convertem em oportunidades concretas de redistribuição interna. Muitos *marketplaces* suportam transações, mas não assentam em stock operacional atualizado nem em mecanismos de recomendação contextualizada (Dawson Jr. et al., 2024; Burgess & Sunmola, 2022).

É precisamente nesta interseção — entre dados operacionais, alertas, recomendações e ação concreta — que se identifica o maior espaço de diferenciação para plataformas digitais orientadas a contextos cooperativos agrícolas. A solução desenvolvida procura ocupar esse espaço, ligando inventário, necessidades, anúncios, pré-venda, recomendações e encomendas num fluxo operacional integrado, simples e orientado à cooperação entre produtores (Trendov et al., 2019; Mateos & Bellogín, 2025; AgGateway, n.d.).

3 Metodologias

O presente capítulo apresenta a metodologia de trabalho adotada ao longo do estágio, desde a compreensão inicial do problema até à validação progressiva da solução desenvolvida. A abordagem seguida pode caracterizar-se como um processo de desenvolvimento iterativo e incremental, no qual a solução foi construída por etapas sucessivas, cada uma validada antes de avançar para a seguinte. Esta opção, comum em projetos de *software* cujos requisitos estão sujeitos a clarificação progressiva, permitiu adaptar o desenvolvimento à medida que o conhecimento do domínio aumentava, reduzindo o risco de iniciar a implementação sem uma compreensão suficiente do problema (Sommerville, 2016).

O trabalho foi organizado de forma faseada. Começou-se pelo estudo do contexto da empresa e do problema apresentado, seguindo-se a análise de soluções existentes, o levantamento de requisitos, a prototipagem da interface, a implementação por módulos e a validação funcional. Em projetos de *software*, a clarificação gradual dos requisitos e a validação frequente das decisões tomadas são fatores importantes para evitar retrabalho e melhorar a adequação da solução final. No contexto deste estágio, esta preocupação foi particularmente relevante, uma vez que a plataforma envolvia vários fluxos interligados, como gestão de stocks, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e suporte ao utilizador.

3.1 Estudo inicial do problema e contexto da empresa

A fase inicial centrou-se na compreensão do problema proposto pela entidade de acolhimento. O objetivo era desenvolver uma plataforma web para apoiar uma cooperativa agrícola, permitindo aos produtores gerir produtos, stocks, necessidades, excedentes e transações internas.

Desde cedo se verificou que a solução pretendida não correspondia apenas a um espaço de compra e venda. A plataforma deveria apoiar a operação diária dos produtores, permitindo registar os produtos produzidos, controlar quantidades disponíveis, identificar excedentes, declarar necessidades e facilitar a articulação entre membros da mesma rede cooperativa.

Nesta fase foram também identificados os principais perfis de utilizador. O produtor representa o utilizador principal da plataforma, sendo responsável pela gestão dos seus produtos, stocks, anúncios, necessidades, encomendas e mensagens. Importa esclarecer que, ao nível técnico da aplicação, este perfil é designado por *cliente*; ao longo do relatório, contudo, adota-se preferencialmente o termo *produtor*, por ser mais claro e representativo do contexto cooperativo. O administrador, por sua vez, assume funções de supervisão, apoio, gestão de utilizadores, gestão de catálogo e acompanhamento global da plataforma.

3.2 Estudo de mercado e análise de soluções existentes

Após a compreensão inicial do problema, procedeu-se à análise de soluções existentes no mercado, abrangendo plataformas de gestão agrícola, marketplaces agrícolas, soluções orientadas para cooperativas e sistemas de apoio à decisão aplicados ao setor agrícola. A análise detalhada destas soluções, bem como o respetivo enquadramento na literatura, é apresentada no Capítulo 2.

Do ponto de vista metodológico, importa salientar que este estudo não constituiu um exercício teórico isolado, mas uma etapa que informou diretamente as decisões de desenho e o levantamento de requisitos. A observação das plataformas existentes permitiu compreender que funcionalidades eram comuns no setor, quais estavam ausentes em contextos cooperativos e de que forma a interface e o modelo de utilização influenciavam a adequação de cada solução ao utilizador final.

Esta análise reforçou o posicionamento do projeto. Ao constatar-se que as soluções existentes tendiam a concentrar-se numa área específica — gestão da exploração, venda externa ou

complexidade empresarial — e que nenhuma integrava de forma simples a gestão de stocks, a detecção de excedentes e necessidades, o marketplace interno e o apoio à decisão entre produtores da mesma rede, justificou-se a criação de uma solução mais ajustada ao contexto cooperativo regional.

3.3 Levantamento e análise de requisitos

O levantamento de requisitos constituiu uma etapa central do projeto, uma vez que permitiu transformar a ideia inicial da empresa num conjunto estruturado de funcionalidades. Esta fase não ocorreu apenas no início do estágio, tendo acompanhado a evolução do trabalho e as decisões tomadas ao longo do desenvolvimento, em coerência com a natureza iterativa da metodologia adotada.

Numa primeira fase, os requisitos permitiram compreender os objetivos principais da plataforma: apoiar uma cooperativa agrícola na gestão de produtos, stocks, necessidades, excedentes e transações internas entre produtores. A partir desta base, foram definidos os principais módulos do sistema, nomeadamente autenticação, perfil do produtor, catálogo, inventário, necessidades, marketplace, recomendações, encomendas, alertas, mensagens, suporte, administração e integração.

Os requisitos foram organizados por módulos funcionais, permitindo distinguir melhor as áreas da aplicação e as respetivas responsabilidades. Esta organização foi importante porque o projeto envolve vários fluxos interligados. Por exemplo, uma necessidade registada por um produtor pode originar uma recomendação; uma recomendação pode gerar uma ou mais encomendas; e uma encomenda pode afetar reservas e stocks. Para além dos requisitos funcionais, foram igualmente considerados requisitos não funcionais, relacionados com segurança, desempenho e usabilidade, cujo tratamento é detalhado nos capítulos de arquitetura e atividades desenvolvidas. O conjunto completo dos requisitos funcionais, na sua versão final, encontra-se no Anexo A.

Ao longo do estágio, algumas funcionalidades inicialmente previstas foram revistas ou reposicionadas. O *chatbot* com inteligência artificial, por exemplo, permaneceu fora do Produto Mínimo Viável (*Minimum Viable Product* — MVP), por não ser essencial para a primeira versão funcional da plataforma. Por outro lado, funcionalidades como pré-venda, suporte, mensagens em tempo real e ofertas dirigidas a necessidades ganharam maior importância, por responderem melhor aos fluxos reais identificados durante o desenvolvimento.

Esta evolução demonstrou que os requisitos funcionaram como um artefacto vivo, atualizado à medida que o projeto amadurecia. Esta abordagem permitiu evitar a implementação de funcionalidades pouco relevantes e concentrar o desenvolvimento nos módulos com maior impacto para o utilizador final.

3.4 Prototipagem e validação inicial da interface

Antes da implementação das principais páginas da aplicação, foi realizada uma fase de prototipagem de alta fidelidade. Esta etapa teve como objetivo explorar a organização visual da plataforma, a disposição dos elementos e os fluxos principais de navegação.

A prototipagem permitiu pensar a aplicação a partir da perspetiva do utilizador. Como a plataforma se destina a produtores agrícolas, cuja literacia digital pode ser variável, a interface tinha de ser simples, direta e fácil de compreender. Assim, foram privilegiadas páginas com navegação clara, cartões informativos, botões visíveis e organização funcional por áreas.

Foram exploradas páginas como o painel do produtor, o marketplace, o detalhe de anúncios, a gestão de stocks, as recomendações e algumas áreas administrativas. O objetivo desta fase não foi gerar o código final da aplicação, mas obter uma base visual que orientasse o desenvolvimento posterior.

A existência de um protótipo facilitou a discussão das ideias iniciais e permitiu antecipar problemas de organização da interface, testando diferentes formas de apresentar a informação antes de avançar para a implementação definitiva.

3.5 Organização do desenvolvimento e validação progressiva

O desenvolvimento foi organizado de forma incremental, começando pelas bases técnicas da aplicação e avançando depois para os módulos de negócio. Esta organização permitiu construir o sistema por etapas, validando cada componente antes de avançar para funcionalidades mais dependentes.

Inicialmente foram criadas a estrutura do projeto, a ligação à base de dados e as primeiras funcionalidades de autenticação. De seguida, foram trabalhados os módulos relacionados com

perfis de produtores, catálogo de produtos e gestão de stocks, que serviram de base para as funcionalidades comerciais da plataforma. Posteriormente, avançou-se para o desenvolvimento das necessidades, do marketplace, das recomendações, das encomendas e dos alertas. Esta sequência respeitou as dependências naturais do sistema: uma recomendação, por exemplo, só poderia ser útil depois de existirem produtos, stocks, necessidades e anúncios disponíveis.

Numa fase final, foram acrescentadas funcionalidades complementares, como mensagens em tempo real, suporte ao utilizador, notificações, mapa no detalhe dos anúncios, áreas administrativas e colocação da aplicação em produção. Estas funcionalidades contribuíram para tornar a plataforma mais completa e mais próxima de um cenário real de utilização.

A validação foi sendo realizada ao longo do desenvolvimento, em articulação com o orientador da empresa. Tratou-se sobretudo de uma validação funcional e contínua, baseada na verificação do comportamento de cada módulo e no *feedback* recolhido em reuniões de acompanhamento, e não de um processo formal de testes com utilizadores finais. Esta abordagem permitiu ajustar prioridades, corrigir problemas e garantir que cada módulo respondia aos objetivos definidos. A natureza e os resultados desta validação são aprofundados no Capítulo 7.

4 Arquitetura Geral da Solução

O presente capítulo apresenta a arquitetura geral da plataforma desenvolvida no âmbito do estágio. A arquitetura foi definida tendo em conta a dimensão do projeto, o tempo disponível, os requisitos funcionais identificados e a necessidade de construir uma aplicação funcional, modular e preparada para evolução futura.

A solução foi concebida como uma aplicação web para apoio a uma cooperativa agrícola, permitindo centralizar funcionalidades relacionadas com produtores, produtos, stocks, necessidades, anúncios, recomendações, encomendas, mensagens, alertas, suporte e administração. A arquitetura adotada procurou equilibrar simplicidade de desenvolvimento, organização interna do código, consistência dos dados e capacidade de funcionamento em ambiente de produção.

De forma geral, a plataforma assenta numa *framework* web, apoiada por uma base de dados relacional e organizada em módulos funcionais. A solução inclui ainda mecanismos de comunicação em tempo real, armazenamento externo de ficheiros, gestão de recursos estáticos, separação de configurações sensíveis e uma infraestrutura de produção preparada para disponibilizar a aplicação fora do ambiente local. A arquitetura geral é apresentada na Secção 4.1.

4.1 Visão geral da solução

A plataforma desenvolvida pode ser entendida como um sistema central de apoio à operação de uma cooperativa agrícola. O utilizador principal é o produtor, que utiliza a aplicação para gerir os seus produtos, consultar o estado dos seus stocks, identificar necessidades, publicar anúncios, responder a necessidades de outros produtores, acompanhar recomendações, criar encomendas e comunicar com outros membros da plataforma.

Para além do produtor, existe também o perfil de administrador. Este perfil tem acesso a áreas de supervisão e gestão, nomeadamente gestão de utilizadores, catálogo de produtos, auditoria, suporte e indicadores globais da plataforma. Esta separação entre produtor e administrador permitiu organizar a aplicação em dois níveis principais: a operação diária dos produtores e a gestão global da plataforma.

A solução foi desenhada para funcionar de forma autónoma. Ainda assim, a arquitetura prevê uma camada de integração, destinada a permitir comunicação futura com outros sistemas. Esta possibilidade foi considerada desde a modelação da base de dados, através da existência de campos de origem externa, identificadores externos e registos de sincronização.

A Figura 4.1 apresenta uma visão geral da arquitetura da solução, evidenciando a relação entre o utilizador, a aplicação web, a base de dados, os mecanismos de comunicação em tempo real, o armazenamento de ficheiros e os serviços externos utilizados em produção.

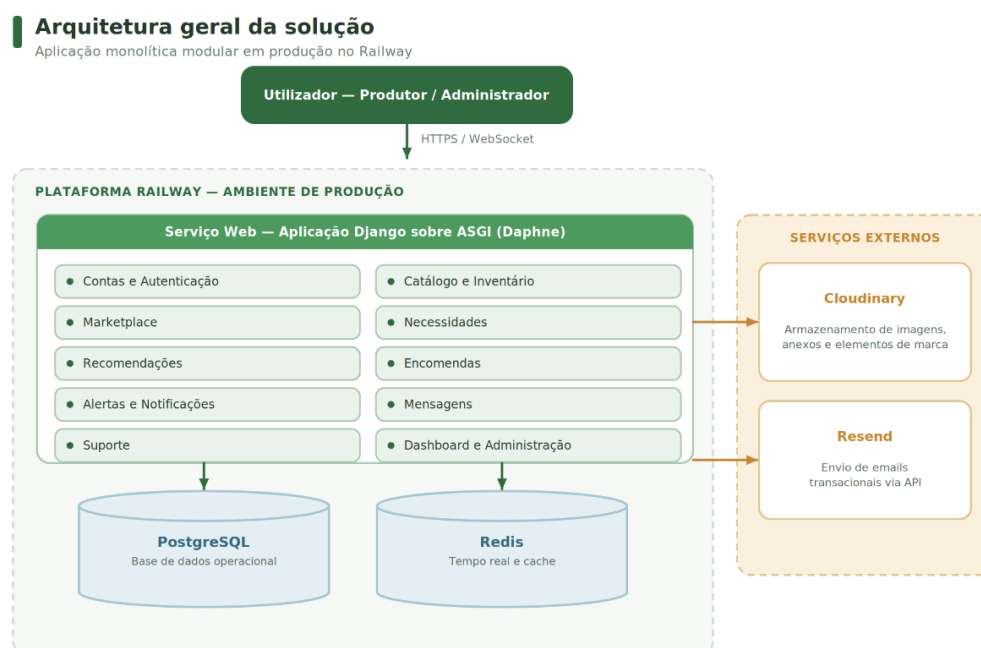


Figura 4.1 — Arquitetura geral da plataforma VISION4FARMS: componentes, módulos funcionais e serviços externos

4.2 Aplicação monolítica modular e organização em módulos

A aplicação foi desenvolvida seguindo uma abordagem de monólito modular. Isto significa que todos os módulos fazem parte do mesmo projeto, partilham a mesma base de dados e são disponibilizados como uma única aplicação web. No entanto, internamente, o código encontra-se separado por áreas funcionais, permitindo manter uma organização clara.

Esta opção revelou-se adequada ao contexto do estágio. Uma arquitetura distribuída, baseada em vários serviços independentes, poderia trazer vantagens em sistemas de grande escala, mas também aumentaria de forma significativa a complexidade de desenvolvimento, configuração, testes e colocação em produção. Para uma primeira versão funcional da plataforma, essa complexidade não teria um benefício proporcional. Esta distinção entre uma aplicação monolítica, desenvolvida como uma unidade, e uma arquitetura baseada em serviços independentes é frequentemente usada para contextualizar a adoção de microserviços (Fowler & Lewis, 2014).

A organização interna foi feita através de *apps*. Cada *app* representa uma área funcional da plataforma e concentra os ficheiros relacionados com essa área, como modelos, *views*, formulários, rotas, serviços auxiliares e *templates*. Esta separação permitiu dividir a aplicação em domínios como contas, catálogo, inventário, necessidades, marketplace, recomendações, encomendas, alertas, notificações, mensagens, suporte, definições, administração e integrações.

Esta estrutura facilitou a manutenção e a evolução do projeto. Quando uma funcionalidade precisava de ser alterada, era possível localizar mais facilmente a área responsável, sem comprometer diretamente todos os restantes módulos. Por exemplo, alterações no módulo de suporte não exigiam alterações diretas no módulo de inventário; e ajustes no módulo de mensagens podiam ser tratados sem modificar o funcionamento do catálogo de produtos.

A separação por módulos foi particularmente importante porque os fluxos da plataforma se relacionam entre si. Uma necessidade pode originar uma recomendação; uma recomendação pode originar encomendas; uma encomenda pode afetar stocks; e estes eventos podem gerar alertas ou notificações. Esta interligação entre módulos é retomada no Capítulo 6, onde são descritas as principais atividades desenvolvidas e os fluxos operacionais implementados.

4.3 Padrão MVT do Django

A aplicação segue o padrão *Model-View-Template* (MVT), utilizado pelo Django para organizar a relação entre dados, lógica e apresentação. Este padrão divide a aplicação em três responsabilidades principais.

O *Model* representa as entidades da aplicação e a ligação à base de dados. É nesta camada que se encontram as estruturas que representam utilizadores, produtores, produtos, stocks, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e restantes entidades do sistema.

A *View* recebe o pedido do utilizador, executa a lógica necessária e devolve uma resposta. Por exemplo, quando um produtor publica um anúncio no marketplace, a *view* responsável valida os dados recebidos, confirma se o utilizador tem permissões para realizar a ação, atualiza a base de dados e encaminha o utilizador para a página adequada.

O *Template* define a estrutura visual apresentada no navegador. É nesta camada que são construídas as páginas HTML, os formulários, as tabelas, os cartões informativos e os restantes elementos visuais da interface.

Um aspeto relevante na interface da plataforma é a utilização do HTMX, integrado neste padrão. Numa aplicação tradicional, cada interação devolve uma página HTML completa. Com o HTMX, uma mesma *view* pode devolver uma página completa ou apenas um fragmento de HTML, consoante o pedido seja normal ou um pedido HTMX. Isto permite atualizar apenas partes da página — por exemplo, uma lista de anúncios ou um contador de notificações — sem recarregar a página inteira e sem recorrer a uma arquitetura de *single-page application*. Desta forma, mantém-se a simplicidade do padrão MVT, acrescentando-lhe interatividade onde esta traz valor.

Esta separação contribuiu para uma organização mais clara do código. A lógica de negócio não ficou misturada diretamente com a apresentação visual, e a estrutura dos dados manteve-se separada da forma como a informação é apresentada ao utilizador. Este princípio está alinhado com a própria filosofia do Django, que valoriza baixo acoplamento entre componentes e organização clara da aplicação (Django Software Foundation, n.d.).

A Figura 4.2 ilustra o percurso completo de um pedido ao longo das três camadas do padrão MVT, desde a interação do utilizador no navegador até à resposta devolvida pelo sistema — página completa ou fragmento HTMX numa atualização parcial.

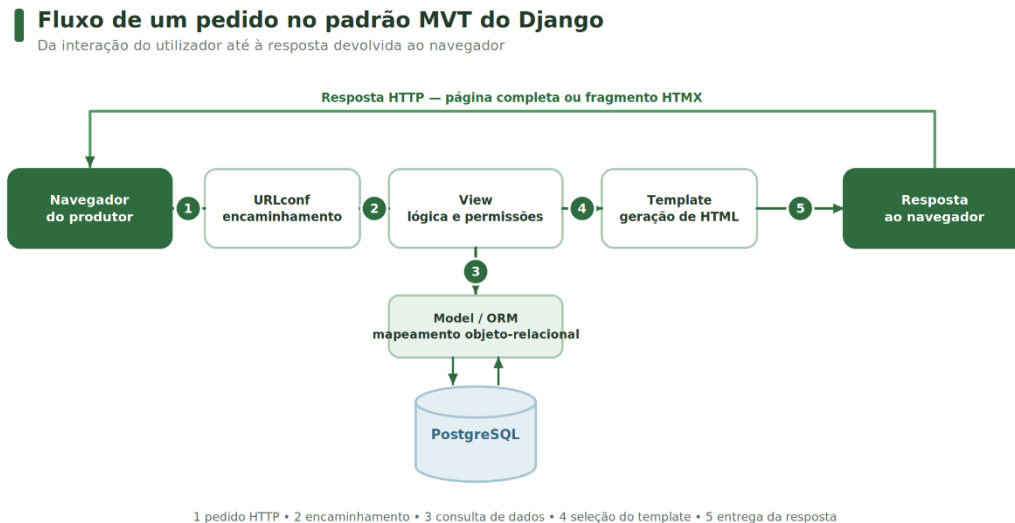


Figura 4.2 — Fluxo de um pedido HTTP no padrão MVT do Django, com percurso pelas camadas de encaminhamento, lógica, dados e apresentação

4.4 Base de dados como fonte de verdade

Uma decisão importante da arquitetura foi tratar a base de dados como a principal fonte de verdade da aplicação. Em vez de deixar a *framework* gerar automaticamente toda a estrutura através do sistema de migrações, o modelo relacional foi desenhado diretamente em PostgreSQL, com recurso ao PgAdmin 4 e ao seu editor de Diagramas Entidade-Relação.

Esta abordagem, conhecida como *schema-first*, significa que a estrutura das tabelas, as relações, as chaves estrangeiras e algumas regras de integridade foram pensadas primeiro ao nível da base de dados, seguindo os princípios de modelação relacional descritos na literatura de sistemas de bases de dados (Elmasri & Navathe, 2016). Só depois essa estrutura foi refletida nos modelos da aplicação.

Para permitir esta integração, a maioria dos modelos de negócio foi configurada com `managed = False`. Esta opção indica que as tabelas já existem e que não devem ser criadas ou alteradas automaticamente pelo sistema de migrações. Desta forma, a aplicação continua a conseguir consultar e manipular os dados, mas a estrutura principal da base de dados permanece controlada manualmente.

Esta decisão envolveu um compromisso consciente. Por um lado, garante controlo total sobre o *schema* e maior consistência num modelo com muitas entidades fortemente relacionadas — utilizadores, perfis de produtor, produtos, stocks, necessidades, anúncios, recomendações, encomendas, mensagens, alertas, notificações e pedidos de suporte —, em que qualquer falha nas relações poderia comprometer os fluxos principais da aplicação. Por outro lado, implica que as alterações de estrutura deixam de ser geridas automaticamente pelas migrações do Django e passam a ser aplicadas manualmente na base de dados e depois refletidas nos modelos. Considerou-se que, neste projeto, o controlo e a consistência compensavam esse custo adicional.

A utilização de identificadores UUID na maioria das entidades, gerados ao nível da própria base de dados através das extensões `uuid-oss` e `pgcrypto` do PostgreSQL, contribuiu ainda para uma maior independência entre registos e para uma melhor preparação da solução para integração futura com outros sistemas.

4.5 Modelo de dados e principais entidades

O modelo de dados foi organizado de acordo com os principais domínios funcionais da plataforma. Esta organização permitiu alinhar a estrutura da base de dados com os módulos da aplicação, evitando que os dados fossem tratados como elementos isolados.

No domínio da identidade e conta, a base de dados contempla informação relacionada com utilizadores, perfis de produtor, preferências e confirmação de conta. Esta separação permite distinguir a autenticação e o controlo de acesso da informação operacional utilizada nos fluxos da cooperativa.

No domínio do catálogo e inventário, são representados os produtos, categorias, produtos associados a produtores, stocks, movimentos de stock, necessidades e produção futura. Este conjunto de entidades suporta a gestão operacional dos produtores e serve de base para os fluxos comerciais da plataforma.

No domínio comercial, encontram-se as entidades associadas a anúncios, recomendações e encomendas. Estas entidades permitem representar anúncios de stock atual, anúncios de pré-venda, respostas privadas a necessidades, sugestões de compra ou venda, grupos de encomendas, itens de encomenda e histórico de estados.

No domínio da comunicação e apoio, o modelo contempla alertas, notificações, conversas, mensagens e pedidos de suporte. Estas entidades permitem acompanhar eventos relevantes, comunicar entre produtores e assegurar um canal de apoio ao utilizador.

Por fim, no domínio da administração e integração, existem entidades destinadas à auditoria e ao registo de sincronizações externas. Estas estruturas foram pensadas para reforçar a rastreabilidade da plataforma e preparar a solução para futuras ligações com outros sistemas.

No corpo do relatório interessa sobretudo evidenciar que o modelo de dados foi construído para suportar os fluxos principais da plataforma e garantir consistência entre módulos. A Figura 4.3 apresenta a organização do modelo de dados em dois grandes grupos: os domínios base, que sustentam identidade, catálogo, inventário e necessidades, e os domínios operacionais, que suportam o marketplace, as recomendações, as encomendas, a comunicação e a auditoria. A vista detalhada, com todas as tabelas, é apresentada na Figura 6.1, e o diagrama relacional completo é apresentado no Anexo B.

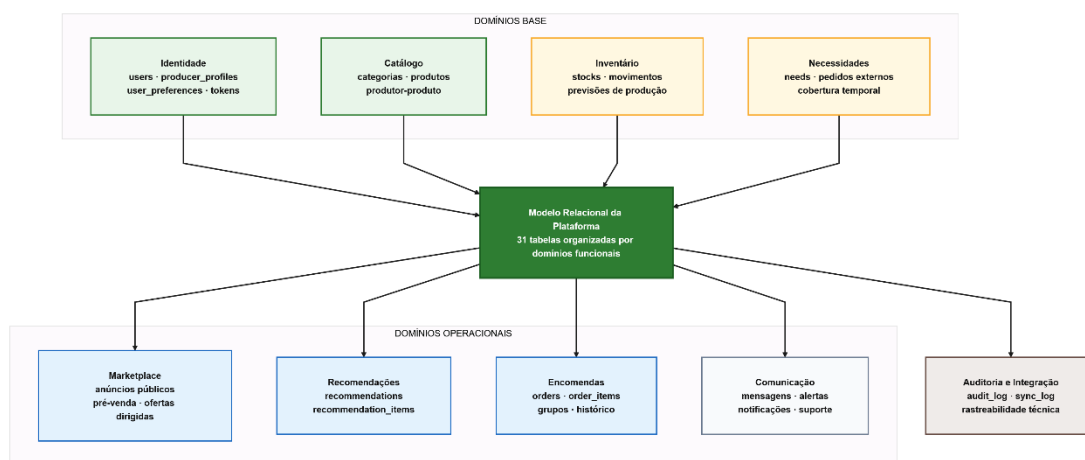


Figura 4.3 — Organização do modelo de dados em domínios base e domínios operacionais

4.6 Comunicação em tempo real e infraestrutura

A arquitetura da solução inclui mecanismos de comunicação em tempo real para funcionalidades em que a atualização imediata da interface acrescenta valor ao utilizador. Esta decisão é

particularmente relevante no módulo de mensagens entre produtores e nos indicadores visuais associados a alertas, mensagens não lidas e suporte.

A utilização de comunicação em tempo real permitiu aproximar alguns comportamentos da aplicação da experiência esperada em plataformas modernas de comunicação. Por exemplo, quando uma nova mensagem é enviada numa conversa, o outro utilizador pode receber a atualização sem ter de recarregar manualmente a página.

Ao mesmo tempo, estas funcionalidades foram introduzidas de forma controlada. A comunicação em tempo real foi aplicada apenas nos pontos em que fazia sentido, mantendo o restante funcionamento da plataforma baseado em pedidos tradicionais. Esta decisão evitou tornar toda a aplicação mais complexa apenas para suportar alguns fluxos específicos.

Do ponto de vista técnico, esta camada assenta em comunicação por *WebSocket*, gerida pelo Django Channels, tendo o Daphne como servidor *ASGI* e o Redis como camada de distribuição de eventos entre ligações. A infraestrutura de produção foi organizada de forma a suportar a aplicação, a base de dados e estes mecanismos de comunicação, permitindo validar a plataforma fora do ambiente local de desenvolvimento. O detalhe das ferramentas utilizadas nesta infraestrutura é apresentado no capítulo dedicado às tecnologias e ferramentas (Capítulo 5).

A Figura 4.4 representa o percurso de uma mensagem desde o momento em que é enviada pelo Produtor A até à sua receção pelo Produtor B, evidenciando o papel do Daphne como servidor *ASGI*, do Redis como camada de distribuição de eventos e da base de dados como ponto de persistência.

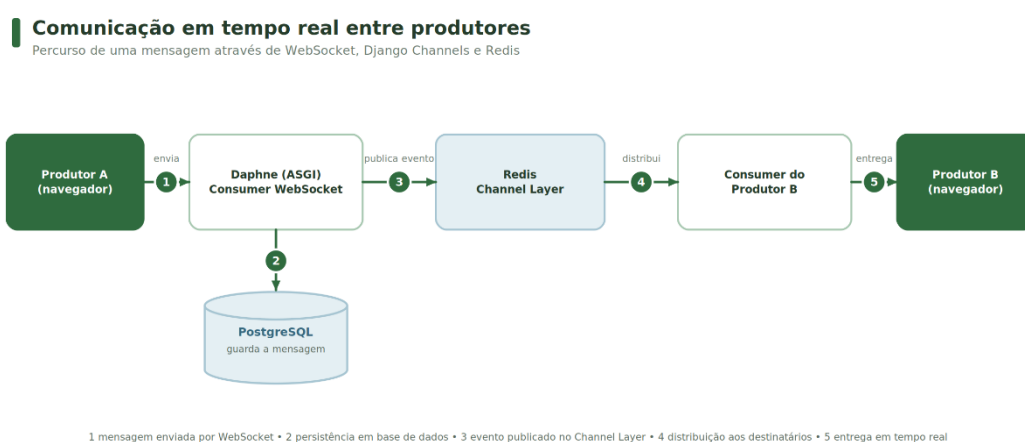


Figura 4.4 — Percurso de uma mensagem em tempo real: WebSocket, Django Channels, Redis e entrega ao destinatário

4.7 Segurança, ficheiros e configuração

A arquitetura da plataforma incluiu preocupações de segurança desde os módulos iniciais. A autenticação foi baseada em contas com confirmação por email, recuperação de palavra-passe, validação de sessão e separação entre perfis de produtor e administrador. Para reforçar a proteção de operações sensíveis, foram ainda adotados mecanismos como a limitação de tentativas (*rate limiting*) no início de sessão e no registo, a proteção contra falsificação de pedidos (*CSRF*) e a invalidação de sessões após a alteração da palavra-passe.

A gestão de acessos foi organizada através de permissões e validações específicas para cada tipo de utilizador. Algumas áreas da aplicação são exclusivas de produtores autenticados, enquanto outras estão reservadas a administradores. Esta separação é essencial para proteger funcionalidades sensíveis, como gestão de utilizadores, catálogo, suporte administrativo e auditoria.

A plataforma também prevê o registo de ações relevantes para efeitos de auditoria. Esta componente permite acompanhar alterações importantes realizadas por utilizadores ou pelo sistema, contribuindo para maior rastreabilidade e controlo operacional.

Relativamente aos ficheiros, a arquitetura distingue ficheiros enviados pelos utilizadores e ficheiros estáticos da própria aplicação. Os ficheiros enviados pelos utilizadores incluem, por exemplo, fotografias de anúncios, imagens de perfil, anexos de mensagens e imagens associadas a pedidos de suporte. Já os ficheiros estáticos incluem elementos necessários à interface, como folhas de estilo, *scripts* e imagens de navegação. O detalhe dos serviços de armazenamento utilizados é apresentado no Capítulo 5.

A configuração da aplicação foi pensada para separar dados sensíveis do código-fonte, recorrendo a variáveis de ambiente. Esta decisão permite manter diferentes configurações entre desenvolvimento e produção, evitando expor credenciais, chaves secretas ou ligações a serviços externos diretamente no código.

Em conjunto, estas decisões contribuíram para uma arquitetura mais segura, organizada e adequada ao funcionamento da plataforma em ambiente real.

5 Tecnologias e ferramentas Utilizadas

O presente capítulo apresenta as principais tecnologias e ferramentas utilizadas no desenvolvimento da plataforma. Depois de, no capítulo anterior, ter sido descrita a arquitetura geral da solução, importa agora explicar o papel concreto de cada tecnologia no projeto e a razão da sua utilização.

As tecnologias selecionadas procuraram responder às necessidades do estágio sem introduzir complexidade desnecessária. Como o projeto foi desenvolvido individualmente e num período limitado, privilegiaram-se ferramentas integradas, bem documentadas e adequadas à construção de uma aplicação *web* modular.

As tecnologias adotadas cobrem o desenvolvimento do *backend*, a base de dados, a interface, a comunicação em tempo real, o armazenamento de ficheiros, o envio de *emails*, o controlo de versões e a colocação da aplicação em produção.

5.1 Ambiente de desenvolvimento e controlo de versões

O Visual Studio Code foi utilizado como ambiente principal de desenvolvimento. Através desta ferramenta foram editados os diferentes ficheiros da aplicação, incluindo código Python, páginas HTML, folhas de estilo CSS, ficheiros JavaScript, *templates* e ficheiros de configuração.

A utilização do Visual Studio Code revelou-se adequada porque permitiu concentrar várias tarefas no mesmo ambiente. O editor disponibiliza terminal integrado, pesquisa global no projeto, realce de sintaxe, integração com Git e extensões de apoio ao desenvolvimento em Python e Django. Durante o estágio, o terminal integrado foi utilizado para executar comandos de desenvolvimento, instalar dependências, iniciar o servidor local e validar alterações realizadas na aplicação.

O GitHub foi utilizado para alojar o código-fonte do projeto e acompanhar a evolução do desenvolvimento. Através do Git, as alterações foram sendo registradas ao longo do tempo, permitindo manter histórico do trabalho, recuperar versões anteriores e organizar melhor o progresso da aplicação.

Para além do controlo de versões, o GitHub teve também um papel importante na colocação da plataforma em produção. O repositório foi ligado ao Railway, permitindo que o código desenvolvido localmente pudesse ser publicado de forma mais controlada e organizada.

As Figuras 5.1 e 5.2 apresentam, respetivamente, o ambiente de desenvolvimento utilizado e o repositório GitHub do projeto.

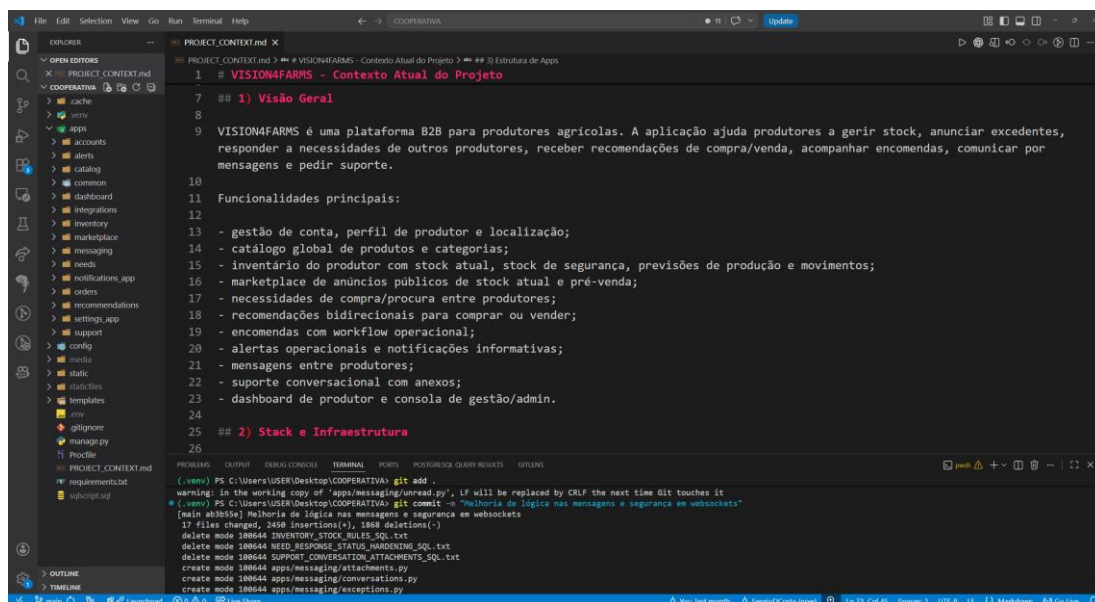


Figura 5.1 — Ambiente de desenvolvimento no Visual Studio Code

Já a Figura 5.2 mostra o repositório GitHub do projeto, utilizado para o controlo de versões e ligado ao Railway para a colocação em produção.

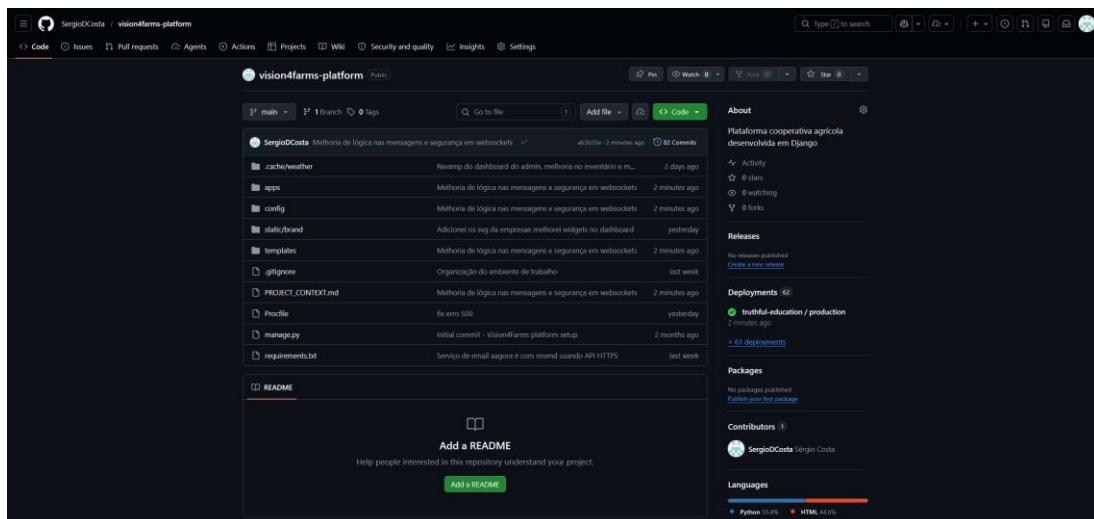


Figura 5.2 — Repositório GitHub do projeto

5.2 Prototipagem de alta fidelidade

O Google AI Studio foi utilizado, numa fase inicial, como ferramenta de apoio à criação do protótipo de alta fidelidade da plataforma. Esta ferramenta permitiu explorar propostas visuais para as páginas principais e gerar rapidamente variações de interface antes de avançar para a implementação definitiva.

No contexto do projeto, o protótipo obtido serviu de base visual para orientar o desenvolvimento das páginas finais, permitindo testar ideias de organização e disposição dos elementos sem produzir, desde logo, o código definitivo da aplicação. O enquadramento metodológico desta fase, bem como o seu papel no processo de desenvolvimento, são apresentados no Capítulo 3.

A Figura 5.3 apresenta um exemplo do protótipo de alta fidelidade criado com apoio do Google AI Studio.

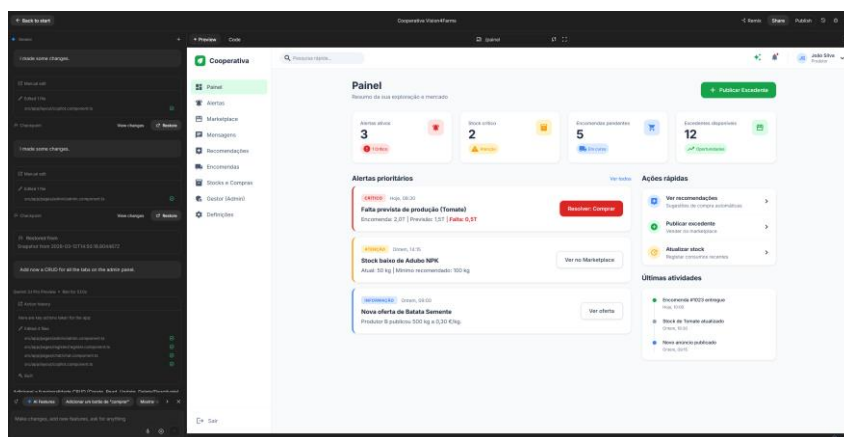


Figura 5.3 — Protótipo de alta fidelidade criado com apoio do Google AI Studio

5.3 Python e Django — *backend* da aplicação

Python foi a linguagem utilizada no *backend* da aplicação. Através dela foi implementada a lógica principal do sistema, incluindo autenticação, gestão de dados, regras de negócio, alertas, recomendações, encomendas, suporte e integração entre módulos.

Django foi a *framework* web adotada para estruturar a aplicação, tendo sido utilizado na versão 6.0.3. Esta escolha revelou-se adequada porque o Django disponibiliza, de forma integrada, várias funcionalidades necessárias ao projeto, como sistema de rotas, *views*, *templates*, formulários, sessões, validações de segurança e ligação à base de dados. A documentação oficial do Django destaca a rapidez de desenvolvimento, a segurança e a escalabilidade como características centrais da *framework* (Django Software Foundation, n.d.).

No projeto, o Django foi utilizado para receber os pedidos dos utilizadores, processar a lógica de negócio e devolver páginas HTML através do sistema de *templates*. Por exemplo, quando um produtor publica um anúncio no marketplace, a *view* correspondente valida os dados recebidos, verifica permissões, atualiza a base de dados e apresenta a página seguinte ao utilizador.

A organização por *apps* foi uma das vantagens mais relevantes da utilização do Django. Cada módulo da plataforma ficou separado numa área própria, como *accounts*, *inventory*, *marketplace*, *needs*, *recommendations*, *orders*, *messaging*, *support* ou *dashboard*. Esta divisão tornou o código mais fácil de localizar, compreender e manter.

A escolha do Django também evitou a necessidade de separar o *backend* e o *frontend* em projetos independentes. Para o contexto do estágio, esta decisão reduziu a complexidade e permitiu concentrar o desenvolvimento numa única aplicação web.

5.4 PostgreSQL e PgAdmin 4 — base de dados e modelação relacional

PostgreSQL foi utilizado como sistema de gestão de base de dados relacional. Uma base de dados relacional organiza a informação em tabelas, permitindo estabelecer relações entre diferentes entidades através de chaves primárias e chaves estrangeiras. Esta abordagem revelou-se adequada ao projeto, uma vez que a plataforma envolve várias entidades interligadas, como utilizadores,

produtores, produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e suporte.

No contexto do projeto, o PostgreSQL não serviu apenas para armazenar dados. A base de dados funcionou como uma parte central da solução, garantindo consistência entre os vários fluxos da plataforma. Por exemplo, uma encomenda pode estar ligada a um anúncio, a um produtor comprador, a um produtor vendedor, a itens de encomenda, a estados e a movimentos de *stock*.

O modelo foi desenhado seguindo uma abordagem *schema-first*. Isto significa que a estrutura da base de dados foi definida diretamente ao nível do PostgreSQL antes de ser refletida nos modelos Django. Esta opção permitiu maior controlo sobre tabelas, tipos de dados, relações e restrições de integridade.

O PgAdmin 4 foi utilizado como ferramenta gráfica de administração e modelação da base de dados. Através desta ferramenta foi possível visualizar tabelas, consultar dados, executar comandos SQL e acompanhar a estrutura relacional da plataforma.

Uma das funcionalidades mais importantes utilizadas foi o editor de Diagramas Entidade-Relação. Esta ferramenta permitiu representar graficamente as tabelas, chaves primárias, chaves estrangeiras e relações entre entidades. A visualização do modelo ajudou a validar a estrutura da base de dados antes e durante a implementação das funcionalidades no Django.

A Figura 5.4 apresenta o modelo relacional da plataforma no editor de Diagramas Entidade-Relação do PgAdmin 4.

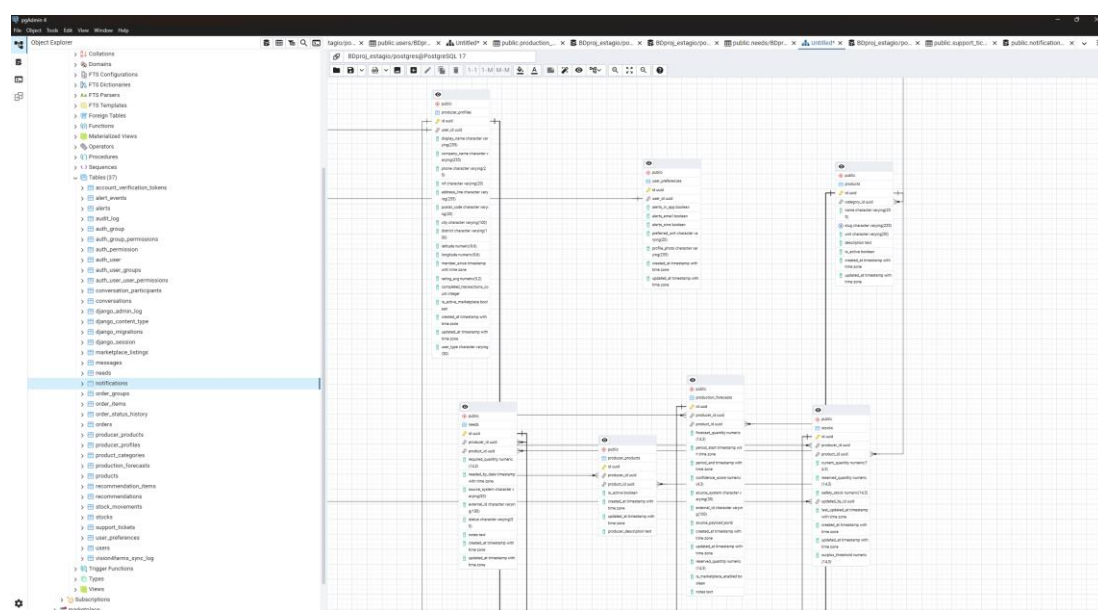


Figura 5.4 — Modelo relacional da plataforma no PgAdmin 4

5.5 Django Templates, HTML, CSS, Bootstrap, HTMX e Chart.js — interface e interatividade

O *frontend* corresponde à parte da aplicação com que o utilizador interage diretamente no navegador. Neste projeto, a interface foi desenvolvida com recurso ao sistema de *templates* do Django, juntamente com HTML, CSS, Bootstrap, HTMX e Chart.js.

O HTML foi utilizado para estruturar as páginas da aplicação, definindo elementos como títulos, formulários, tabelas, botões, menus e cartões informativos. O CSS permitiu personalizar o aspeto visual da plataforma, ajustando cores, espaçamentos, tamanhos, alinhamentos e comportamento visual dos elementos.

O Bootstrap foi utilizado para acelerar a criação de uma interface responsiva e consistente. Esta *framework* disponibiliza componentes e utilitários para construir interfaces web de forma mais rápida, incluindo grelhas, botões, formulários, cartões e barras de navegação (Bootstrap Team, n.d.). A sua utilização permitiu criar páginas funcionais sem desenhar todos os componentes de raiz.

O HTMX foi utilizado para adicionar interações dinâmicas em zonas específicas da interface. Esta biblioteca permite aceder a funcionalidades modernas do navegador diretamente através de atributos HTML, incluindo pedidos assíncronos e atualização parcial de páginas (HTMX, n.d.). No projeto, esta abordagem foi útil para tornar algumas operações mais fluidas, mantendo a simplicidade de uma aplicação *server-rendered*.

O Chart.js foi utilizado em áreas específicas da plataforma onde a visualização de dados ajudava a interpretar informação operacional. Por exemplo, no painel administrativo foi utilizado para representar a evolução de compras e vendas por semana, e nas áreas de inventário permitiu apoiar a leitura de dados relacionados com compras, vendas e produção. Desta forma, os gráficos funcionaram como complemento visual aos indicadores, sem transformar a aplicação numa plataforma analítica complexa.

Esta combinação permitiu criar uma interface suficientemente dinâmica sem recorrer a uma *framework* de *frontend* mais pesada, como React ou Vue.js. Para o contexto do estágio e para o perfil dos utilizadores previstos, esta opção foi mais simples, mais rápida de implementar e mais fácil de manter.

A Figura 5.5 apresenta um exemplo de interface desenvolvida com Django Templates, Bootstrap e HTMX, e a Figura 5.6 apresenta elementos gráficos produzidos com Chart.js.

```

154 <div class="pnl-card">
155 <div class="pnl-card-head"><h2 class="pnl-card-title">Ações rápidas</h2></div>
156 <div class="pnl-card-list">
157 <a href="/recomendacoes/" class="pnl-qa pnl-qa-blue">
158 <div class="pnl-qa-ico"><i class="bi bi-megat"></i></div>
159 <div class="pnl-qa-txt"><b>Recomendações</b><span>Sugestões de compra</span></div>
160 <i class="bi bi-chevron-right pnl-qa-arr"></i>
161 </a>
162 <a href="/marketplace/" class="pnl-qa pnl-qa-green">
163 <div class="pnl-qa-ico"><i class="bi bi-shop"></i></div>
164 <div class="pnl-qa-txt"><b>Publicar excedente</b><span>Gerir anúncios</span></div>
165 <i class="bi bi-chevron-right pnl-qa-arr"></i>
166 </a>
167 <a href="/inventory/meus_produtos/" class="pnl-qa pnl-qa-amber">
168 <div class="pnl-qa-ico"><i class="bi bi-box-seam-fill"></i></div>
169 <div class="pnl-qa-txt"><b>Atualizar stock</b><span>Registrar consumos</span></div>
170 <i class="bi bi-chevron-right pnl-qa-arr"></i>
171 </a>
172 <a href="/encomendas/" class="pnl-qa pnl-qa-slate">
173 <div class="pnl-qa-ico"><i class="bi bi-truck"></i></div>
174 <div class="pnl-qa-txt"><b>Ver encomendas</b><span>Acompanhar estados</span></div>
175 <i class="bi bi-chevron-right pnl-qa-arr"></i>
176 </a>
177 </div>
178 </div>
179 </div>
232 <div>
233 /* vars
234 .pnl {
235 --g: #16a34a; --gd: #15803d; --gs: #dcfce7;
236 --r: #dc2626; --rs: #f8e2e2;
237 --a: #d97706; --as: #fde0d5;
238 --b: #2553eb; --bs: #dbeafe;
239 --sl: #475569; --sls: #f1f3f9;
240 --tx: #8f172d; --mt: #64748b; --br: #e2e8f0;
241 --cbg: #ffffff; --pbg: #f1f3f9;
242 --rad: 85rem;
243 --shd: 0 1px 3px #rgba(15,23,42,.05),0 4px 14px #rgba(15,23,42,.05);
244 font-family: 'DM Sans', system-ui, sans-serif;
245 display: flex; flex-direction: column; gap: 1.25rem;
246 }
247
248 /* -- hero
249 .pnl-hero {
250 background: linear-gradient(135deg,#0d2117 0%,#18432a 55%,#16a34a 100%);
251 border-radius: var(--rad);
252 padding: 2rem 2.5rem;
253 display: flex; align-items: center; justify-content: space-between;
254 flex-wrap: wrap; gap: 1.25rem;
255 position: relative; overflow: hidden;
256 }
257 .pnl-hero::before {
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

Figura 5.5 — Exemplo de interface desenvolvida com Django Templates, Bootstrap e HTMX

Por sua vez, a Figura 5.6 ilustra a utilização do Chart.js na visualização de dados operacionais, como a evolução de compras e vendas apresentada no painel administrativo e nas áreas de inventário.

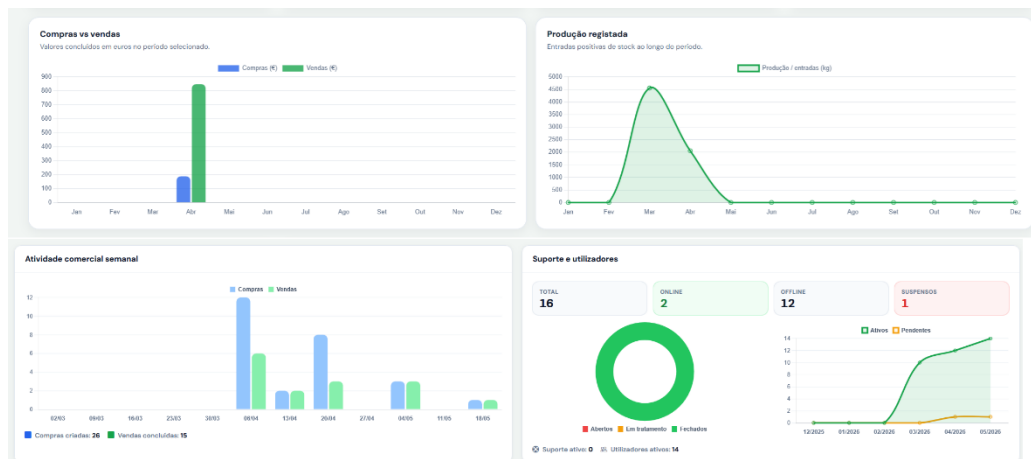


Figura 5.6 — Exemplo de interface desenvolvida com Chart.js

5.6 Django Channels, Redis e Daphne — comunicação em tempo real

Algumas funcionalidades da plataforma exigiam atualização imediata ou quase imediata da interface. Isto aconteceu sobretudo no módulo de mensagens entre produtores e nos indicadores visuais associados a mensagens não lidas, alertas e suporte.

Para responder a esta necessidade, foi utilizado Django Channels, que permite acrescentar comunicação assíncrona a uma aplicação Django, incluindo suporte a *WebSockets* (Django Channels, n.d.). Este tipo de comunicação permite manter uma ligação aberta entre o navegador e o servidor, possibilitando atualizações sem depender apenas de pedidos tradicionais iniciados pelo utilizador.

No projeto, esta tecnologia foi aplicada sobretudo ao módulo de mensagens e a alguns indicadores em tempo real. Por exemplo, quando uma nova mensagem é enviada numa conversa, a atualização pode aparecer ao outro utilizador sem que este tenha de atualizar manualmente a página.

O Redis foi utilizado como camada intermédia para encaminhar eventos para os utilizadores corretos. Esta função foi importante em cenários como mensagens em tempo real, em que uma ação realizada por um utilizador precisa de ser refletida quase imediatamente na interface de outro.

O Daphne foi utilizado como servidor ASGI em produção. A sua utilização permitiu servir tanto pedidos HTTP tradicionais como ligações *WebSocket*, mantendo a aplicação preparada para funcionalidades em tempo real.

Esta combinação foi introduzida apenas onde acrescentava valor. O restante funcionamento da aplicação manteve-se baseado em pedidos HTTP tradicionais, evitando tornar todo o sistema mais complexo do que o necessário.

5.7 Serviços externos, ficheiros, configuração e email

Para além das tecnologias principais, a plataforma depende de um conjunto de serviços e bibliotecas de apoio que tratam de responsabilidades específicas: armazenamento de ficheiros, recursos estáticos, configuração e envio de *emails*. Estas componentes não definem, por si só, a lógica

principal da aplicação, mas são essenciais para que a plataforma funcione corretamente fora do ambiente local de desenvolvimento.

O Cloudinary foi utilizado para armazenar ficheiros enviados pelos utilizadores. Estes ficheiros incluem, por exemplo, fotografias de anúncios, imagens de perfil, anexos de mensagens, imagens associadas a pedidos de suporte e até mesmo logótipos como visto na figura 5.8. A utilização de um serviço externo de armazenamento evitou depender do sistema de ficheiros local do servidor, o que é particularmente importante em ambientes *cloud*.

O WhiteNoise foi utilizado para servir ficheiros estáticos da aplicação, como CSS, JavaScript, imagens da interface e outros recursos necessários ao funcionamento visual da plataforma. Esta opção simplificou a disponibilização dos ficheiros estáticos em produção, mantendo a configuração alinhada com uma aplicação Django (WhiteNoise, n.d.).

A biblioteca python-decouple foi utilizada para separar configurações sensíveis do código-fonte. Dados como chave secreta da aplicação, credenciais da base de dados, ligação ao Redis, configurações de *email* e credenciais do Cloudinary foram definidos através de variáveis de ambiente. Esta separação contribuiu para uma configuração mais segura e flexível entre desenvolvimento local e produção (Python Decouple, n.d.).

O envio de *emails* foi necessário em fluxos como confirmação de conta, recuperação de palavra-passe, notificações de segurança e suporte ao utilizador. Em produção, este envio foi configurado com recurso ao Resend, como apresentado na figura 5.7, através da biblioteca django-anymail. A documentação do Resend apresenta uma integração específica com Django através de django-anymail, utilizando o *backend* `anymail.backends.resend.EmailBackend` (Resend, n.d.). A documentação do Anymail também identifica o Resend como um dos serviços suportados para envio transacional de *email* em aplicações Django (Anymail, n.d.).

Estas ferramentas contribuíram para tornar a aplicação mais preparada para ambiente real, separando responsabilidades importantes: ficheiros dos utilizadores, ficheiros estáticos da interface, configurações sensíveis e comunicações por *email*.

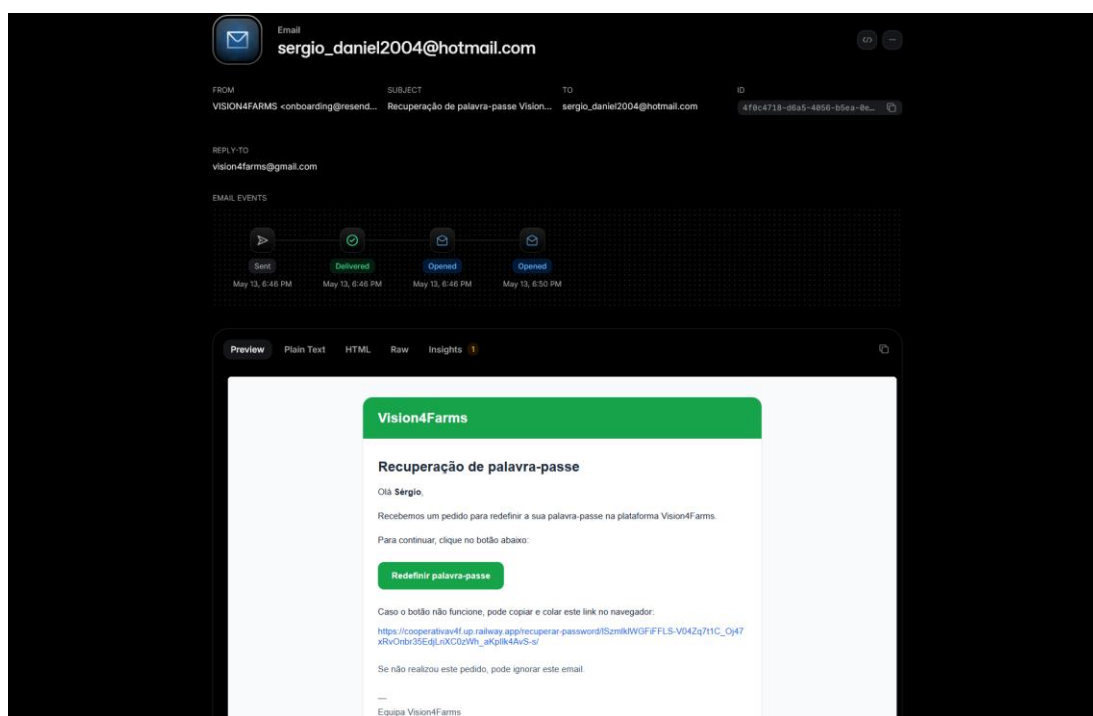


Figura 5.7 — Exemplo de email transacional de recuperação de palavra-passe enviado através do Resend.

Por sua vez a figura 5.8 demonstra o ambiente de armazenamento de toda a media da app.

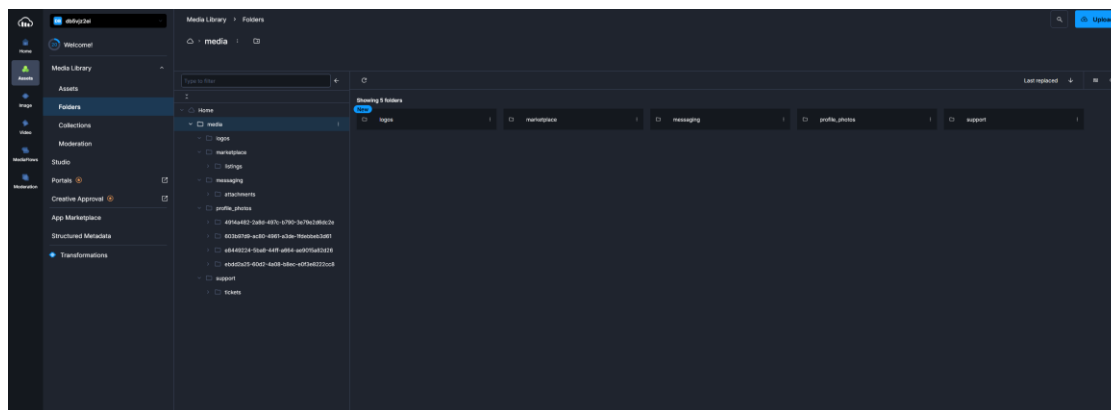


Figura 5.8 — Configuração dos serviços de media, estáticos e variáveis de ambiente

5.8 Railway — colocação em produção

O Railway foi utilizado como plataforma de alojamento da aplicação. Esta escolha permitiu colocar a plataforma em produção sem configurar manualmente um servidor tradicional.

No Railway foram configurados os serviços principais necessários ao funcionamento da aplicação: a aplicação Django executada com Daphne, a base de dados PostgreSQL e o Redis. Esta organização

permitiu validar a plataforma fora do ambiente local, testando-a em condições mais próximas de utilização real.

A documentação do Railway inclui um guia específico para implantação de aplicações Django e refere a possibilidade de configurar uma aplicação Django juntamente com uma base de dados PostgreSQL (Railway, n.d.). No projeto, esta capacidade foi importante porque permitiu concentrar no mesmo ambiente os serviços necessários à execução da plataforma.

O *deploy* foi associado ao repositório GitHub. Desta forma, o código desenvolvido localmente pôde ser publicado em produção de forma mais controlada e organizada. Esta etapa foi relevante para validar a aplicação como uma solução funcional e não apenas como um projeto executado em ambiente local de desenvolvimento.

A Figura 5.9 apresenta os serviços configurados no Railway para execução da plataforma.

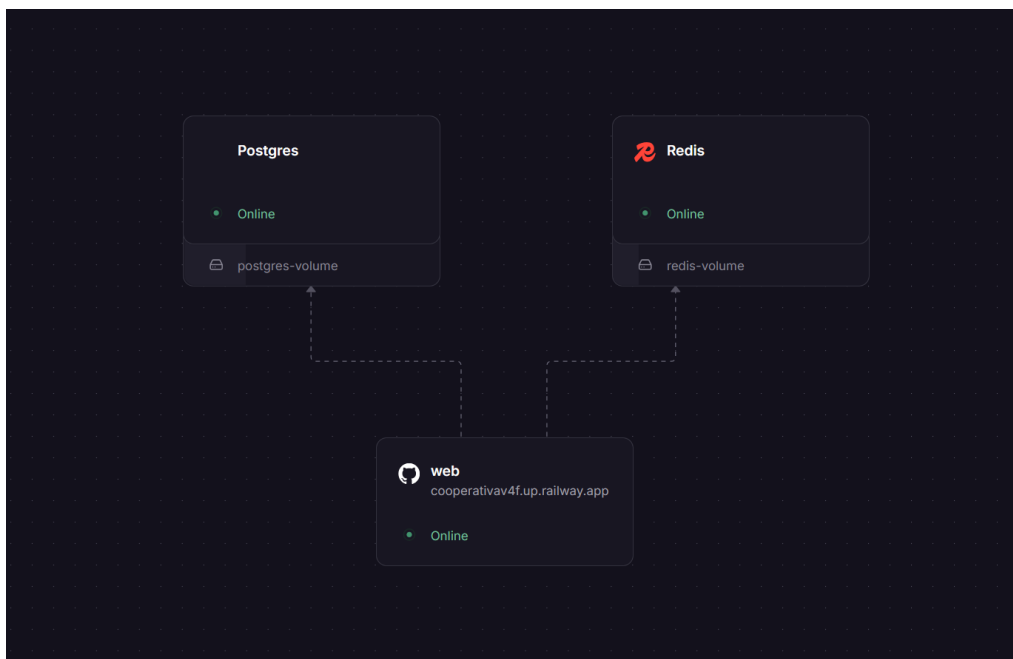


Figura 5.9 — Serviços configurados no Railway para execução da plataforma

5.9 Justificação geral das escolhas tecnológicas

As escolhas tecnológicas procuraram responder às necessidades do projeto sem introduzir complexidade desnecessária. A opção por Python e Django permitiu desenvolver uma aplicação *web* completa, com lógica de negócio, autenticação, formulários, rotas, *templates* e ligação à base de dados dentro de uma única estrutura.

Django foi preferido a alternativas como Flask ou FastAPI porque disponibiliza, de raiz, várias funcionalidades úteis para aplicações *web* completas, como sistema de autenticação, formulários, sessões, organização por *apps* e sistema de *templates*. Flask exigiria integrar manualmente mais bibliotecas para cobrir as mesmas necessidades, enquanto FastAPI seria mais adequado a uma API separada, o que obrigaria a desenvolver também uma aplicação *frontend* independente. Para o contexto do estágio, essa separação aumentaria a complexidade sem trazer benefícios proporcionais.

PostgreSQL foi escolhido por suportar adequadamente um modelo relacional com várias entidades e relações. A plataforma depende da consistência entre produtores, produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e suporte, pelo que a base de dados teve um papel central.

A utilização de Django Templates, HTML, CSS, Bootstrap e HTMX permitiu criar uma interface funcional e suficientemente dinâmica sem recorrer a uma *framework* de *frontend* mais pesada. Alternativas como React ou Vue.js poderiam oferecer maior separação entre interface e servidor, mas exigiriam uma API própria, mais configuração e maior esforço de integração. A opção tomada foi mais coerente com o tempo disponível, com o perfil dos utilizadores e com o objetivo de manter a navegação simples.

Django Channels, Redis e Daphne foram introduzidos apenas nas funcionalidades em que a comunicação em tempo real era necessária. Esta decisão permitiu acrescentar mensagens e indicadores dinâmicos sem transformar toda a aplicação numa arquitetura mais pesada.

Cloudinary, WhiteNoise, python-decouple, Resend, django-anymail e Railway foram utilizados para preparar a aplicação para produção, resolvendo necessidades práticas de armazenamento, ficheiros estáticos, configuração, *email* e alojamento.

Importa ainda referir uma decisão consciente quanto àquilo que não foi implementado. A aplicação integra serviços externos por API, mas não expõe uma API REST própria nesta versão. Embora a *framework* Django REST Framework estivesse disponível, optou-se por não a utilizar, uma vez que não existia requisito de aplicação móvel, integração com sistemas externos ou consumo por terceiros que justificasse o esforço adicional de desenvolver, documentar, autenticar e testar uma API. Esta opção reduziu a complexidade operacional e a superfície de ataque de uma aplicação já em produção, mantendo as permissões, validações e auditoria concentradas nos fluxos Django existentes. Ainda assim, a solução ficou preparada para essa evolução, através de campos como

source_system e external_id, que permitem importar dados de sistemas externos caso uma integração venha a justificar-se.

Em síntese, as tecnologias escolhidas permitiram construir uma plataforma funcional, modular, colocada em produção e alinhada com os objetivos definidos para o estágio. Para o tempo disponível e para a dimensão do projeto, esta combinação representou um equilíbrio adequado entre simplicidade, produtividade e robustez.

6 Atividades desenvolvidas

O presente capítulo descreve as principais atividades desenvolvidas durante o estágio, centrando-se na implementação da plataforma VISION4FARMS. Depois de definidos os requisitos, a arquitetura e as tecnologias utilizadas, o trabalho passou pela construção dos módulos funcionais que suportam a operação da cooperativa agrícola.

A plataforma foi desenvolvida como uma aplicação *web* para produtores agrícolas, permitindo gerir conta, perfil, produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, alertas, mensagens, suporte e administração. Estes módulos foram implementados de forma progressiva, respeitando as dependências entre funcionalidades. Por exemplo, o *marketplace* depende da existência de produtos e *stocks*; as recomendações dependem de necessidades e anúncios; e as encomendas dependem dos fluxos de compra e venda previamente definidos.

O desenvolvimento não consistiu apenas na criação de páginas isoladas. Cada módulo foi pensado como parte de um fluxo maior, onde uma ação do utilizador pode afetar várias áreas da aplicação. Uma necessidade pode originar uma recomendação; uma recomendação pode gerar encomendas; uma encomenda pode reservar ou movimentar *stock*; e esses eventos podem produzir alertas, notificações ou registos de auditoria. Esta lógica integrada foi uma das partes centrais do trabalho desenvolvido.

6.1 Modelação do Sistema e Estrutura de Dados

A primeira atividade relevante consistiu na definição da estrutura de dados da plataforma. Antes de avançar para a implementação dos módulos funcionais, foi necessário desenhar um modelo relacional capaz de suportar os principais fluxos do sistema.

O modelo de dados foi organizado em torno dos domínios principais da aplicação: identidade, catálogo, inventário, *marketplace*, necessidades, recomendações, encomendas, comunicação,

alertas, suporte e auditoria. Esta organização permitiu criar uma base sólida para o desenvolvimento, evitando que cada funcionalidade fosse tratada de forma isolada.

O modelo atual contempla entidades como *users*, *producer_profiles*, *products*, *stocks*, *needs*, *marketplace_listings*, *recommendations*, *orders*, *messages*, *alerts*, *support_tickets*, *audit_log* e *vision4farms_sync_log*. Estas entidades encontram-se relacionadas entre si para garantir que os diferentes fluxos da plataforma funcionam de forma consistente. Por exemplo, um produtor está associado a produtos, os produtos podem ter *stocks*, os anúncios podem ter origem em *stock* atual ou produção futura, e os itens de encomenda podem atualizar a cobertura de uma necessidade.

6.1.1 Implementação do esquema relacional

A implementação do esquema relacional foi realizada diretamente sobre a base de dados PostgreSQL, com base no ficheiro SQL consolidado do projeto. Nesta fase foram criadas as tabelas principais, definidas as chaves primárias e estrangeiras, aplicadas restrições de integridade e organizadas as relações entre os módulos.

Uma das decisões importantes foi separar a conta do utilizador do perfil funcional do produtor. Assim, a entidade *users* ficou responsável pelos dados de autenticação, enquanto *producer_profiles* passou a guardar informação operacional, como nome público, empresa, contacto e localização. Esta separação permitiu distinguir melhor a gestão de acesso da informação usada nos fluxos agrícolas.

Também foram criadas entidades específicas para representar o inventário e a atividade comercial. A tabela *stocks* guarda o estado atual do *stock* por produtor e produto, enquanto *stock_movements* mantém o histórico de alterações. A tabela *marketplace_listings* representa anúncios de *stock* atual, pré-venda ou resposta privada a uma necessidade. Já as entidades *recommendations*, *recommendation_items*, *order_groups*, *orders* e *order_items* suportam o fluxo entre recomendação, compra e encomenda.

Para reforçar a rastreabilidade, foram ainda criadas tabelas de histórico e apoio, como *order_status_history*, *alert_events*, *notifications*, *audit_log* e *vision4farms_sync_log*. Esta opção permitiu guardar não apenas o estado atual da plataforma, mas também parte da evolução dos processos realizados pelos utilizadores.

6.1.2 Modelo relacional, entidades principais e Diagrama Entidade-Relação

Concluída a criação do esquema, o modelo relacional foi organizado em nove domínios funcionais, num total de trinta e uma tabelas relacionais. Esta organização, representada na Figura 6.1, permitiu alinhar a estrutura da base de dados com os módulos da aplicação: cada domínio agrupa as tabelas que suportam uma área específica da plataforma.

A maioria das entidades utiliza identificadores do tipo UUID, gerados ao nível da própria base de dados. Esta opção, em vez de identificadores numéricos sequenciais, garante maior independência entre registos e prepara a solução para uma eventual integração com sistemas externos, onde a unicidade dos identificadores entre diferentes sistemas é importante.

As relações entre tabelas refletem os fluxos reais da plataforma. Cada utilizador tem um perfil de produtor associado numa relação de um para um, separando os dados de autenticação da informação operacional. Um produtor pode produzir vários produtos, e cada produto pode ser produzido por vários produtores, pelo que esta associação é resolvida através de uma tabela intermédia. Os *stocks* e as previsões de produção pertencem sempre a uma combinação de produtor e produto.

As relações mais relevantes encontram-se no domínio comercial. Um anúncio do *marketplace* pode ter origem em *stock* atual, numa previsão de produção ou numa resposta privada a uma necessidade. Esta origem é identificada através da relação do anúncio com a tabela correspondente. Da mesma forma, os itens de encomenda podem estar ligados a uma necessidade, o que permite recalcular automaticamente a sua cobertura, e uma recomendação pode estar associada à necessidade que lhe deu origem. É esta teia de relações que garante que uma ação do utilizador se propaga de forma consistente pelos vários módulos.

A Figura 6.1 apresenta uma visão geral do modelo, agrupada por domínios. O diagrama relacional completo, com todas as tabelas, colunas, chaves e cardinalidades, é apresentado no Anexo B, dada a sua dimensão.



Figura 6.1 — Visão geral do modelo de dados da plataforma, organizado por domínios funcionais

6.2 Gestão de Conta, Autenticação e Segurança

A implementação dos mecanismos de conta e autenticação constituiu uma das primeiras fases do desenvolvimento, uma vez que todas as restantes áreas da plataforma dependem da identificação do utilizador. O sistema distingue dois perfis principais: o produtor e o administrador. O produtor utiliza a plataforma para gerir a sua operação diária, enquanto o administrador tem acesso a áreas de supervisão, catálogo, gestão de utilizadores, suporte e auditoria.

Foram desenvolvidos fluxos de registo, confirmação de conta, início de sessão, recuperação de palavra-passe e validação de permissões. Em conjunto, estes mecanismos protegem o acesso às áreas privadas da plataforma e garantem que cada utilizador apenas executa ações compatíveis com o seu perfil.

6.2.1 Registo, confirmação de conta e recuperação de palavra-passe

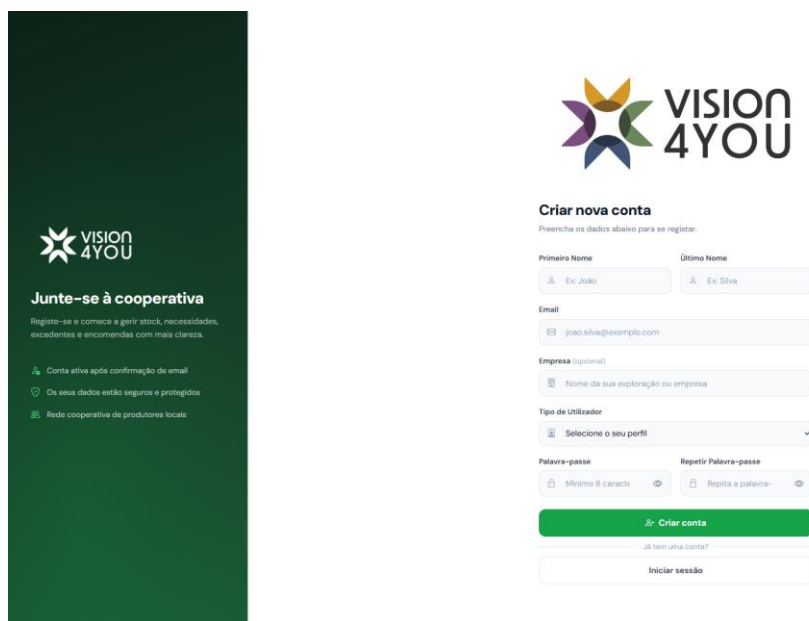
O fluxo de registo permite criar uma nova conta de produtor através de um formulário. As palavras-passe são verificadas pelos validadores do Django, que impedem a utilização de palavras-passe

demasiado simples ou previsíveis. Após a submissão, o sistema envia um email de confirmação, garantindo que a conta só fica plenamente ativa depois de validado o endereço de email.

A plataforma foi pensada para lidar com situações em que o envio do email falha. Nesses casos, a conta é na mesma criada e a interface orienta o utilizador a contactar o suporte. Os administradores podem ainda confirmar manualmente contas pendentes, evitando que um problema pontual de email impeça o acesso de um produtor à plataforma.

O sistema inclui também recuperação de palavra-passe. Quando um produtor solicita uma recuperação, os pedidos anteriores ainda pendentes são invalidados antes de ser emitido um novo, evitando a existência de várias ligações de recuperação válidas em simultâneo. Após a alteração da palavra-passe, é enviado um email de segurança que informa o utilizador da mudança e o aconselha a contactar o suporte caso não a tenha realizado.

Estes fluxos foram importantes para tornar a plataforma mais segura e adequada a um ambiente real de utilização. Como a aplicação gere dados de produtores, anúncios, encomendas e mensagens, não seria adequado permitir acesso sem uma validação mínima da identidade do utilizador.



The image displays two side-by-side screenshots of the Vision 4YOU registration interface. The left screenshot is a dark green banner with the Vision 4YOU logo at the top. Below the logo, the text reads 'Junte-se à cooperativa' followed by a paragraph: 'Registe-se e começa a gerir stock, necessidades, acidentes e encomendas com mais clareza.' At the bottom, there are three bullet points with icons: 'Conta ativa após confirmação de email', 'Os seus dados estão seguros e protegidos', and 'Rede cooperativa de produtoras locais'. The right screenshot shows the 'Criar nova conta' (Create new account) form. It features the Vision 4YOU logo at the top, followed by the title 'Criar nova conta' and a subtitle 'Preencha os dados abaixo para se registar'. The form includes input fields for 'Primeiro Nome' (First Name) and 'Último Nome' (Last Name), an 'Email' field, an 'Empresa (opcional)' (Optional Company) field, a 'Tipo de Utilizador' (User Type) dropdown menu, and 'Palavra-passe' (Password) and 'Repetir Palavra-passe' (Repeat Password) fields. A green 'Criar conta' (Create account) button is at the bottom, with a link 'Já tem uma conta?' (Already have an account?) and an 'Iniciar sessão' (Log in) button below it.

Figura 6.2 — Página de registo de conta de produtor

6.2.2 Gestão de sessões, permissões e proteção de acessos

Após a autenticação, a plataforma mantém uma sessão ativa do utilizador e valida o seu estado em cada área protegida. Esta validação verifica não apenas se o utilizador está autenticado, mas também se a conta se encontra ativa e se a sessão é legítima, comparando-a com a informação associada à sessão iniciada.

O controlo de acessos foi organizado através de decoradores aplicados às páginas da aplicação. Existem decoradores distintos para exigir um utilizador autenticado, para restringir o acesso exclusivamente a produtores e para restringir o acesso exclusivamente a administradores. Por exemplo, as páginas de gestão administrativa só podem ser acedidas por administradores, enquanto áreas como *stocks*, necessidades, mensagens e encomendas pertencem ao espaço operacional do produtor.

Foram também implementados redirecionamentos seguros. Quando uma página recebe um parâmetro a indicar para onde o utilizador deve ser encaminhado após uma ação, esse destino é validado antes de ser utilizado. Esta validação evita que o parâmetro seja explorado para encaminhar o utilizador para endereços externos ou indevidos.

Estes mecanismos foram concentrados em componentes comuns da aplicação, o que permitiu reutilizá-los de forma consistente em vários módulos. Por uma questão de segurança, a alteração da palavra-passe nas definições termina a sessão atual e invalida as restantes sessões do utilizador, obrigando a um novo início de sessão.

6.2.3 Auditoria, *rate limiting* e validação de operações

Para reduzir abusos e proteger fluxos sensíveis, foram aplicadas regras de *rate limiting* em operações como o início de sessão, o registo e a submissão de pedidos de suporte. Estas regras limitam o número de tentativas permitidas num curto intervalo de tempo — no caso do início de sessão, tanto por endereço IP como por email — ajudando a proteger a aplicação contra tentativas repetidas e utilização indevida.

Foi também implementado um registo de auditoria para as operações mais relevantes da plataforma. Sempre que ocorre uma ação significativa — como uma alteração de conta, uma operação administrativa ou uma interação de suporte — é criado um registo na tabela de auditoria.

Cada registo guarda a ação realizada, o utilizador associado, a entidade afetada, o endereço IP, a descrição do dispositivo e, quando aplicável, os valores anteriores e posteriores à alteração.

Esta abordagem foi pensada com cuidado quanto à informação guardada. O registo de auditoria nunca armazena dados sensíveis, como palavras-passe ou outros segredos — apenas metadados que permitem compreender o que foi alterado, por quem e em que momento. Além disso, uma falha no registo de auditoria não interrompe a operação principal, uma vez que a auditoria funciona como mecanismo de apoio e rastreabilidade.

O objetivo desta componente é reforçar a rastreabilidade da plataforma. Num sistema com gestão de contas, movimentos de *stock*, encomendas e ações administrativas, é importante poder reconstituir o historial das operações realizadas, tanto para apoio ao utilizador como para deteção de eventuais problemas.

Gestão da Cooperativa
Visão global, auditoria e administração do sistema

🔍 Pesquisar por ação, utilizador, IP, dispositivo: Mostrar 25 Filtrar J. Exportar

DATA / HORA	UTILIZADOR	AÇÃO	ENTIDADE	DETALHES	IP	DISPOSITIVO
> 23/05/2026 04:28	Ric Fazeres	Iniciou sessão	Utilizador	Início de sessão em Telemóvel - Firefox - Android.	92.250.102.26	Telemóvel - Firefox - Android

SESSÃO
Iniciou sessão
Início de sessão em Telemóvel - Firefox - Android.

RESUMO OPERACIONAL

AUTOR Ric Fazeres	ENTIDADE Utilizador 7cb63dbb-7bb3	DATA 23/05/2026 04:28
----------------------	---	--------------------------

ACESSO E DISPOSITIVO

IP 92.250.102.26	DISPOSITIVO Telemóvel - Firefox - Android
---------------------	--

Referência técnica

AÇÃO TÉCNICA USER_LOGIN	AUTOR Ric Fazeres
ENTIDADE users 7cb63dbb-7bb3-45fb-b6fd-02da3b4a7bb3	
IP 92.250.102.26	DISPOSITIVO Telemóvel - Firefox - Android
USER-AGENT Mozilla/5.0 (Android 16; Mobile; rv:150.0) Gecko/150.0 Firefox/150.0	

> 23/05/2026 04:27	Admin Cooperativa	Email confirmado por admin	Utilizador	Administrador confirmou manualmente a conta de sergio...	188.88.35.149	Computador - Firefox - Windows
> 23/05/2026 04:25	Admin Cooperativa	Iniciou sessão	Utilizador	Início de sessão em Computador - Firefox - Windows.	188.88.35.149	Computador - Firefox - Windows
> 23/05/2026 04:16	Joel Silva	Alterou perfil de produtor	Perfil de produtor	Utilizador atualizou a localização do produtor.	188.88.35.149	Computador - Firefox - Windows
> 23/05/2026 04:15	Joel Silva	Iniciou sessão	Utilizador	Início de sessão em Computador - Firefox - Windows.	188.88.35.149	Computador - Firefox - Windows

Figura 6.3 — Registos de auditoria no painel administrativo

6.3 Gestão de Produtos, Perfis e Inventário

Depois de implementada a camada de conta e segurança, o desenvolvimento avançou para os módulos que dão suporte à operação diária dos produtores: o catálogo de produtos, o perfil do produtor e a gestão de inventário. Estes módulos constituem a base sobre a qual assentam, mais tarde, o *marketplace*, as recomendações e as encomendas. Sem produtos associados aos produtores

e sem informação de *stock*, nenhuma dessas funcionalidades teria informação suficiente para funcionar corretamente.

A implementação desta área teve como objetivo transformar a informação agrícola de cada produtor em dados organizados e reutilizáveis pela plataforma. Assim, o catálogo permite normalizar os produtos, o perfil identifica quem produz, e o inventário permite acompanhar quantidades, movimentos, compromissos externos e produção futura.

6.3.1 Catálogo global de produtos e categorias

O catálogo da plataforma é partilhado por todos os produtores. Em vez de cada produtor trabalhar com designações próprias e isoladas, existe um catálogo global, organizado por categorias, que garante maior consistência na identificação dos produtos.

Esta decisão foi importante para o funcionamento da rede cooperativa. Se cada produtor registasse livremente o mesmo produto com nomes diferentes, seria mais difícil cruzar necessidades com excedentes, gerar recomendações fiáveis ou apresentar resultados coerentes no *marketplace*. Ao partilhar um catálogo comum, a plataforma garante que uma necessidade de determinado produto corresponde ao mesmo produto que outro produtor pode ter disponível.

A gestão formal do catálogo é uma responsabilidade do administrador. É o administrador que cria, organiza e atualiza categorias e produtos, mantendo o catálogo coerente. No fluxo principal, o produtor seleciona, a partir do catálogo global, os produtos que efetivamente produz. Desta forma, evita-se a criação descontrolada de designações diferentes para o mesmo produto e garante-se maior consistência entre inventário, necessidades, recomendações e anúncios.

6.3.2 Perfil do produtor e associação de produtos

Cada produtor possui um perfil que reúne a sua informação operacional, como nome público, empresa, contacto e localização. Esta informação é distinta dos dados de autenticação, que são guardados separadamente, conforme descrito na modelação do sistema.

A partir do catálogo global, o produtor associa ao seu perfil os produtos que produz. Esta associação é o que liga o catálogo partilhado à realidade de cada produtor: define quem produz o quê dentro da

cooperativa. Um mesmo produto pode estar associado a vários produtores, e cada produtor pode produzir vários produtos.

Esta associação é também o ponto de partida para o inventário. Só faz sentido registrar *stock* de um produto depois de o produtor indicar que efetivamente o produz. Desta forma, o perfil e a associação de produtos funcionam como a ligação entre o catálogo global e a gestão operacional descrita nas secções seguintes.

6.3.3 Gestão de *stocks*, movimentos e previsões de produção

O módulo de inventário permite ao produtor registrar e acompanhar o *stock* de cada produto que produz. Cada registo de *stock* corresponde a uma combinação de produtor e produto, guardando a quantidade atualmente existente e a quantidade reservada.

Sempre que uma quantidade é alterada, a plataforma regista um movimento de *stock*. Em vez de simplesmente substituir o valor anterior, o sistema mantém um histórico de movimentos, indicando o que mudou, quando e por quem. Este histórico é importante para a rastreabilidade do inventário, pois permite compreender a evolução das quantidades ao longo do tempo, em vez de mostrar apenas o estado atual.

Para além do *stock* atual, o produtor pode registrar previsões de produção. Uma previsão indica que um determinado produto estará disponível num período futuro, ainda que não exista quantidade disponível no momento. Esta informação é usada para apoiar a pré-venda e para avaliar se determinados compromissos futuros podem ser cumpridos com produção prevista.

A separação entre *stock* atual e previsão de produção foi uma decisão relevante do projeto. Permite que a plataforma represente, de forma distinta, aquilo que existe no presente e aquilo que poderá existir no futuro. Estes dois tipos de informação alimentam fluxos diferentes no *marketplace*, nas recomendações, nos alertas e na análise de cobertura de necessidades.

A Figura 6.4 apresenta o detalhe de um produto no inventário, onde são visíveis os indicadores de stock total, disponível e reservado, bem como o histórico completo de movimentos — incluindo importações, reservas por encomenda, débitos por conclusão e respetivas notas de contexto.

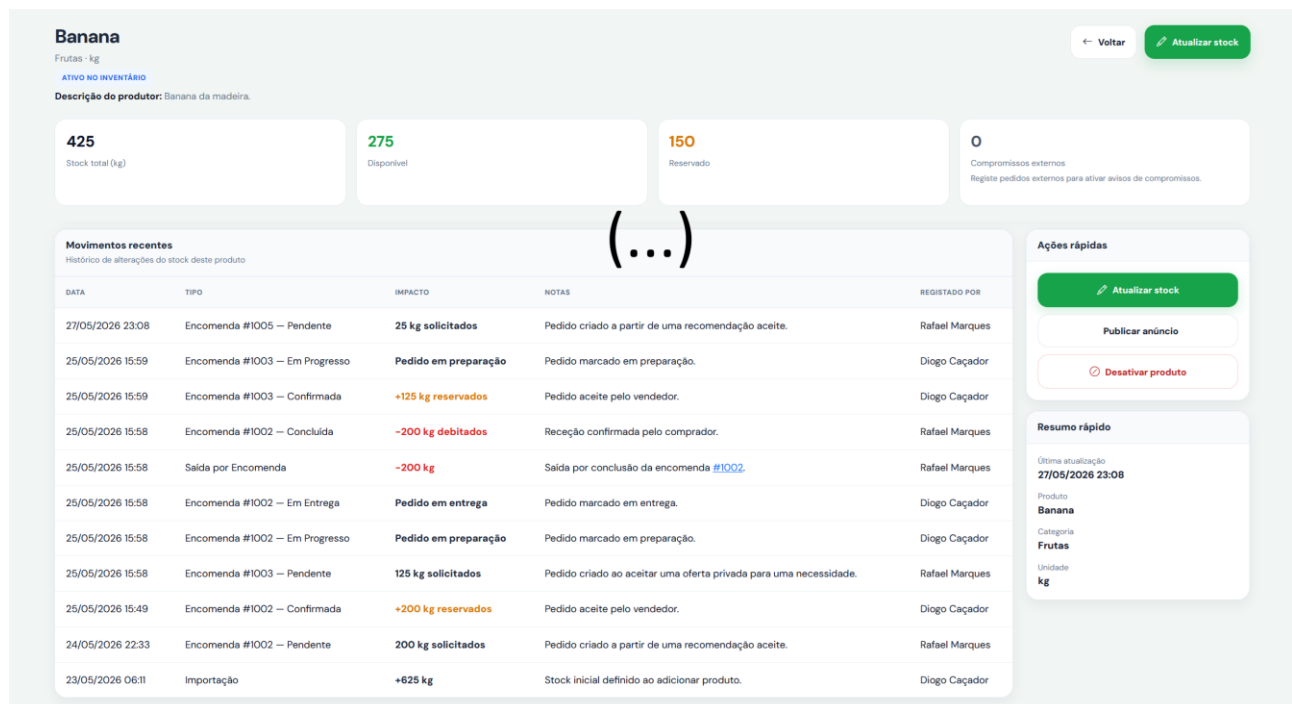


Figura 6.4 — Detalhe de um produto no inventário: indicadores de stock, histórico de movimentos e ações rápidas

6.3.4 Cálculo de disponibilidade, stock crítico e excedentes

O inventário não se limita a guardar quantidades. A plataforma interpreta os valores registados para apoiar a decisão do produtor, distinguindo *stock* real, quantidade reservada, compromissos externos, produção futura útil e margem temporal vendável.

Para cada produto, a disponibilidade é calculada a partir da quantidade atual e das reservas existentes. Quando existem pedidos externos de clientes, a plataforma considera também as quantidades comprometidas e a produção futura disponível a tempo. Desta forma, o estado do produto não depende apenas do *stock* existente no momento, mas também da capacidade de cumprir compromissos futuros.

Esta lógica permite classificar situações de risco e oportunidade. Um produto pode ser assinalado como crítico quando o *stock* atual e a produção futura útil não são suficientes para responder aos

compromissos assumidos. Pelo contrário, quando existe margem temporal acima desses compromissos, a plataforma pode sugerir a publicação de excedente no *marketplace*.

Desta forma, o inventário deixa de ser apenas uma lista de quantidades e passa a funcionar como instrumento de apoio à decisão. A plataforma ajuda o produtor a perceber se deve comprar, vender ou manter o produto, tendo em conta não apenas o momento atual, mas também compromissos e previsões futuras.

A Figura 6.5 ilustra o estado do inventário de um produtor com produtos em situações distintas.

Stocks, Compras e Vendas
Gira o inventário e acompanhe compras, vendas e produção registrada.

Adicionar Produto Publicar Anúncio

Inventário Atual
4 produtos · 2 críticos · 0 perto dos compromissos · 2 com potencial de venda

PRODUTO STOCK ATUAL RESERVADO COMPROMISSOS EXTERNOS ESTADO AÇÕES SUGESTÃO

Frutas (1 produto)

Banana Frutas - kg	425 kg Disponível: 300 kg	125 kg	0 kg Publicável: 300 kg	DISPONÍVEL	👁️ ✎️ 🗑️	
------------------------------	-------------------------------------	---------------	-----------------------------------	-------------------	----------	--

Hortícolas (3 produtos)

Batata Hortícolas - kg	200 kg Disponível: 200 kg	0 kg	800 kg	DÉFICE TEMPORAL	👁️ ✎️ 🗑️	
Cebola Hortícolas - kg	500 kg Disponível: 500 kg	0 kg	700 kg	DÉFICE TEMPORAL	👁️ ✎️ 🗑️	
Soja Hortícolas - kg	150 kg Disponível: 150 kg	0 kg	0 kg Publicável: 150 kg	DISPONÍVEL	👁️ ✎️ 🗑️	

Figura 6.5 — Estado do inventário do produtor com indicadores de disponibilidade, reservas e compromissos externos

6.4 Marketplace Cooperativo e Necessidades

O *marketplace* é o módulo central da componente comercial da plataforma. É aqui que os produtores publicam o que têm disponível para vender, consultam oportunidades de compra e respondem a necessidades existentes. Estas necessidades podem ser criadas manualmente pelos produtores ou resultar de pedidos externos de clientes, quando a plataforma identifica défice temporal de determinado produto.

Este módulo transforma a informação operacional do inventário em oportunidades concretas de troca dentro da rede cooperativa. Assim, produtos, *stocks*, previsões, necessidades e encomendas deixam de ser elementos isolados e passam a alimentar um fluxo comum de compra e venda interna.

6.4.1 Publicação de anúncios e pré-venda

O produtor pode publicar anúncios no *marketplace* a partir do seu inventário. Um anúncio dá visibilidade a um produto que o produtor tem disponível para vender aos restantes membros da cooperativa.

A plataforma distingue dois tipos principais de anúncio. O anúncio de *stock* atual diz respeito a um produto que já existe e está disponível no momento, estando associado a um registo de *stock*. O anúncio em pré-venda diz respeito a um produto que ainda não está disponível em *stock*, mas que se encontra associado a uma previsão de produção futura.

Esta distinção permite que a procura comece antes da colheita ou produção efetiva, dando aos produtores mais tempo para planear compras e vendas. Na prática, a pré-venda permite transformar informação de produção futura numa oportunidade comercial, sem confundir essa previsão com *stock* real.

Cada anúncio reúne a informação necessária para que outro produtor possa avaliar a oferta, como produto, quantidade, preço, modo de entrega, localização e fotografia opcional. A origem do anúncio — *stock* atual ou previsão de produção — é definida ao nível do modelo de dados, garantindo que cada anúncio está ligado à informação correta.

6.4.2 Gestão de necessidades e respostas privadas

Para além de publicar o que têm disponível, os produtores podem declarar necessidades. Uma necessidade representa a intenção de adquirir determinada quantidade de um produto, sendo o ponto de partida para vários fluxos da plataforma.

As necessidades funcionam de forma complementar aos anúncios. Enquanto o anúncio representa uma oferta, a necessidade representa uma procura. A partir de uma necessidade, outro produtor pode enviar uma resposta privada, ou seja, uma proposta de venda dirigida especificamente ao produtor que declarou essa necessidade.

Esta separação entre anúncios públicos e respostas privadas foi pensada para cobrir dois cenários distintos. O anúncio público é adequado quando o produtor pretende dar visibilidade alargada ao produto que tem disponível. A resposta privada é adequada quando se pretende responder diretamente a um pedido concreto, mantendo a proposta restrita aos produtores envolvidos.

No estado atual da plataforma, as necessidades também podem estar relacionadas com pedidos externos de clientes. Quando existe um conjunto de pedidos externos que não consegue ser coberto pelo *stock* atual e pela produção futura útil, a plataforma pode gerar ou atualizar uma necessidade agregada. Esta necessidade passa a representar, dentro da rede cooperativa, uma procura real que resulta de compromissos comerciais assumidos pelo produtor.

6.4.3 Listagem, pesquisa, filtros e detalhe de anúncios

À medida que o número de anúncios cresce, torna-se essencial que os produtores consigam encontrar rapidamente aquilo que procuram. Por isso, o *marketplace* disponibiliza uma página de listagem com mecanismos de pesquisa e filtragem.

Os produtores podem filtrar os anúncios por critérios como pesquisa, categoria, origem do anúncio, ordenação e disponibilidade, reduzindo a listagem àquilo que é efetivamente relevante. Cada anúncio pode depois ser aberto numa página de detalhe, que reúne a informação da oferta e disponibiliza as ações possíveis, como contactar o vendedor ou avançar para uma encomenda.

Para além das ofertas, o *marketplace* pode também apresentar procuras agregadas associadas a necessidades em aberto ou parcialmente cobertas. Estas procuras são visualmente distintas dos anúncios de venda e encaminham o produtor para o fluxo de resposta à necessidade. Desta forma, o *marketplace* deixa de ser apenas uma montra de produtos disponíveis e passa também a mostrar oportunidades de venda baseadas em necessidades reais.

No detalhe do anúncio, a plataforma pode apresentar um mapa com a localização da exploração ou uma aproximação por cidade/distrito, de acordo com as opções de privacidade definidas pelo vendedor. Esta solução permite dar contexto geográfico suficiente para apoiar a decisão, sem expor necessariamente a localização exata do produtor.

A Figura 6.6 apresenta o *feed* público do *marketplace* cooperativo, onde são visíveis anúncios de venda de outros produtores e procuras agregadas resultantes de défices identificados.

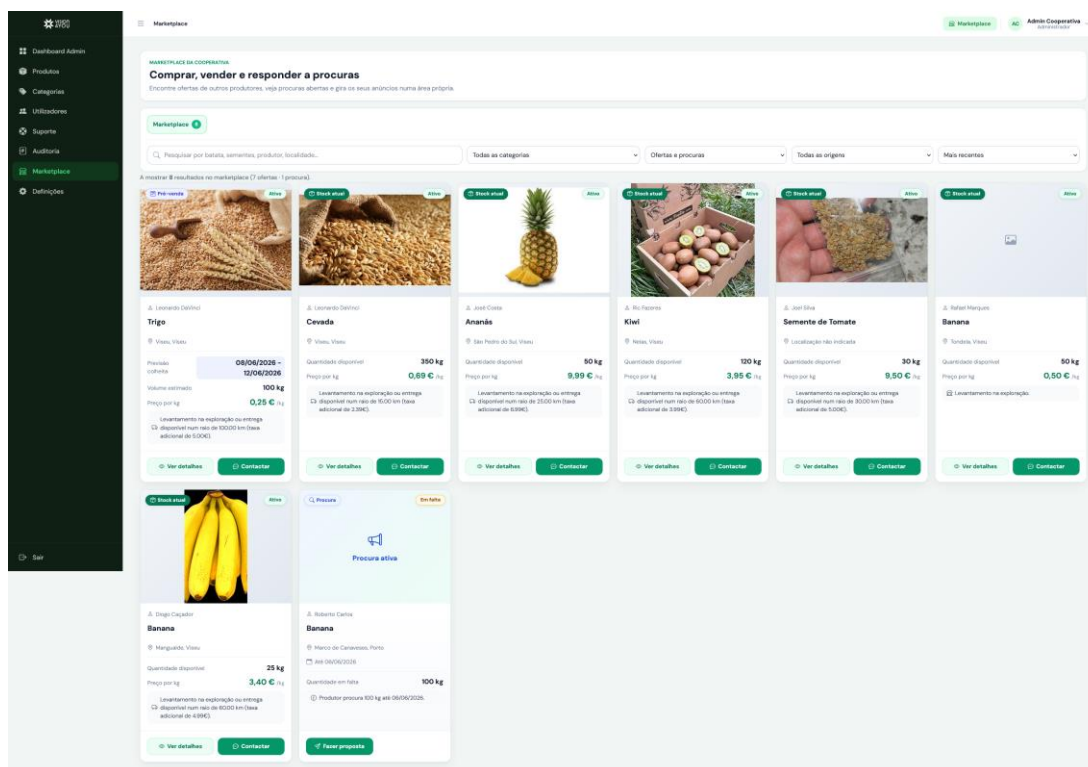


Figura 6.6 — Feed do marketplace cooperativo com anúncios de venda e procura agregadas

6.4.4 Cobertura de necessidades e regras operacionais

Uma necessidade não é um elemento estático. À medida que vão sendo criadas propostas, recomendações ou encomendas associadas a essa necessidade, a plataforma recalcula a sua cobertura, ou seja, a parte da quantidade pretendida que já se encontra assegurada.

Este cálculo permite representar uma necessidade como aberta, parcialmente coberta ou coberta. A cobertura é atualizada de forma automática, à medida que os fluxos de compra evoluem, garantindo que o estado da necessidade acompanha a realidade operacional.

A cobertura de necessidades tem consequências nos restantes módulos. Uma necessidade que se aproxima do prazo sem estar coberta pode dar origem a um alerta, chamando a atenção do produtor. Da mesma forma, é a partir das necessidades por satisfazer que o sistema de recomendações identifica oportunidades de compra ou venda dentro da rede cooperativa.

No caso das necessidades geradas a partir de pedidos externos de clientes, a plataforma considera o défice temporal existente (ver secção 6.3.4): não basta haver stock em abstrato, é preciso que chegue a tempo da data pretendida. Quando o défice desaparece, a necessidade automática pode ser marcada como coberta, refletindo que o compromisso deixou de representar uma falta operacional.

Desta forma, as necessidades funcionam como um elo de ligação entre vários módulos: nascem de uma intenção ou compromisso do produtor, podem ser respondidas através de propostas privadas, alimentam recomendações, originam encomendas e contribuem para a geração de alertas.

6.5 Sistema de Recomendações e Apoio à Decisão

O sistema de recomendações foi desenvolvido para transformar a informação existente na plataforma em sugestões práticas para o produtor. Em vez de obrigar o utilizador a interpretar manualmente *stocks*, necessidades, anúncios e produção futura, a plataforma procura identificar oportunidades de compra ou venda e apresentá-las de forma orientada.

Este módulo funciona como uma camada de apoio à decisão. A recomendação não substitui a decisão do produtor, mas organiza a informação disponível para facilitar a escolha. Assim, quando existe défice de produto, o sistema pode sugerir onde comprar; quando existe margem vendável, pode sugerir a publicação de excedente ou a resposta a uma necessidade de outro produtor.

A implementação foi baseada numa lógica operacional e explicável. O objetivo não foi criar um modelo preditivo automático, mas sim usar os dados reais já existentes na aplicação: *stock* atual, quantidade reservada, compromissos externos, produção futura útil, necessidades abertas e anúncios disponíveis no *marketplace*. Esta opção tornou as recomendações mais transparentes e adequadas à fase atual da plataforma.

Recomendações Inteligentes

Veja onde precisa de comprar, onde tem excedente para vender e avance com a ação certa.

1 Produto
 Escolha o produto para avaliar compra ou venda.

2 Sugestão
 O sistema indica a ação recomendada para o stock.

3 Confirmar
 Valide o resumo final antes de gerar a encomenda.

Passo 1 — Decidir o que comprar ou vender

Veja rapidamente os produtos com maior necessidade e os excedentes mais relevantes.

PEDIDOS DE CLIENTES
Comprar

PRODUTO	DISPONÍVEL	COMPROMISSOS	A COMPRAR
Cevada Em falta para pedidos externos até 30/06/2026	150 kg	600 kg	450 kg

EXCEDENTE
Vender

PRODUTO	DISPONÍVEL	COMPROMISSOS	A VENDER
Trigo Margem vendável após compromissos externos	215 kg	0 kg	215 kg

STOCK DISPONÍVEL
150 kg
 Stock atual 500 kg · reservado 0 kg · compromissos 600 kg.

SUGESTÃO DE COMPRA
450 kg
 Quantidade em falta para cumprir pedidos externos.

AÇÃO SELECIONADA
Cevada
 Comprar para cumprir pedidos. Valor sugerido para cumprir pedidos externos em aberto. Pode ajustar manualmente antes de gerar.

Quantidade a comprar

Gerar recomendação de compra

Figura 6.7 — Recomendação de compra apresentada ao produtor com base em défices e excedentes do inventário

6.5.1 Lógica geral do sistema de recomendações

A lógica geral do sistema de recomendações parte do estado operacional do produtor. A plataforma analisa os produtos associados ao seu inventário e identifica situações em que pode ser necessário comprar ou vender.

Do lado da compra, a recomendação surge quando existe défice temporal, ou seja, quando o produto se encontra em situação crítica nos termos descritos na secção 6.3.4. Nestes casos, o sistema procura ofertas compatíveis no *marketplace* e apresenta sugestões de aquisição.

Do lado da venda, a recomendação surge quando existe margem temporal vendável, isto é, excedente acima dos compromissos existentes (ver secção 6.3.4). Nesses casos, o sistema pode sugerir a publicação de excedente ou a resposta a necessidades abertas de outros produtores.

A página de recomendações foi organizada para apresentar estas duas possibilidades de forma clara: produtos a comprar e produtos a vender. Esta separação evita que o produtor tenha de interpretar sozinho todos os indicadores do inventário e aproxima o módulo de uma lógica de ação concreta.

A recomendação guarda informação sobre o produtor, o produto, a quantidade analisada, a origem funcional, o estado, a síntese da sugestão e o custo estimado quando aplicável. Cada recomendação pode ainda ter itens associados, representando as ofertas concretas sugeridas ao utilizador. O

modelo atualizado inclui as entidades `recommendations` e `recommendation_items`, ligadas a produtores, produtos, anúncios e, quando aplicável, necessidades ou alertas.

A Figura 6.8 sintetiza o fluxo completo de decisão do sistema, desde a análise do estado temporal do inventário até à criação de um fluxo operacional concreto.

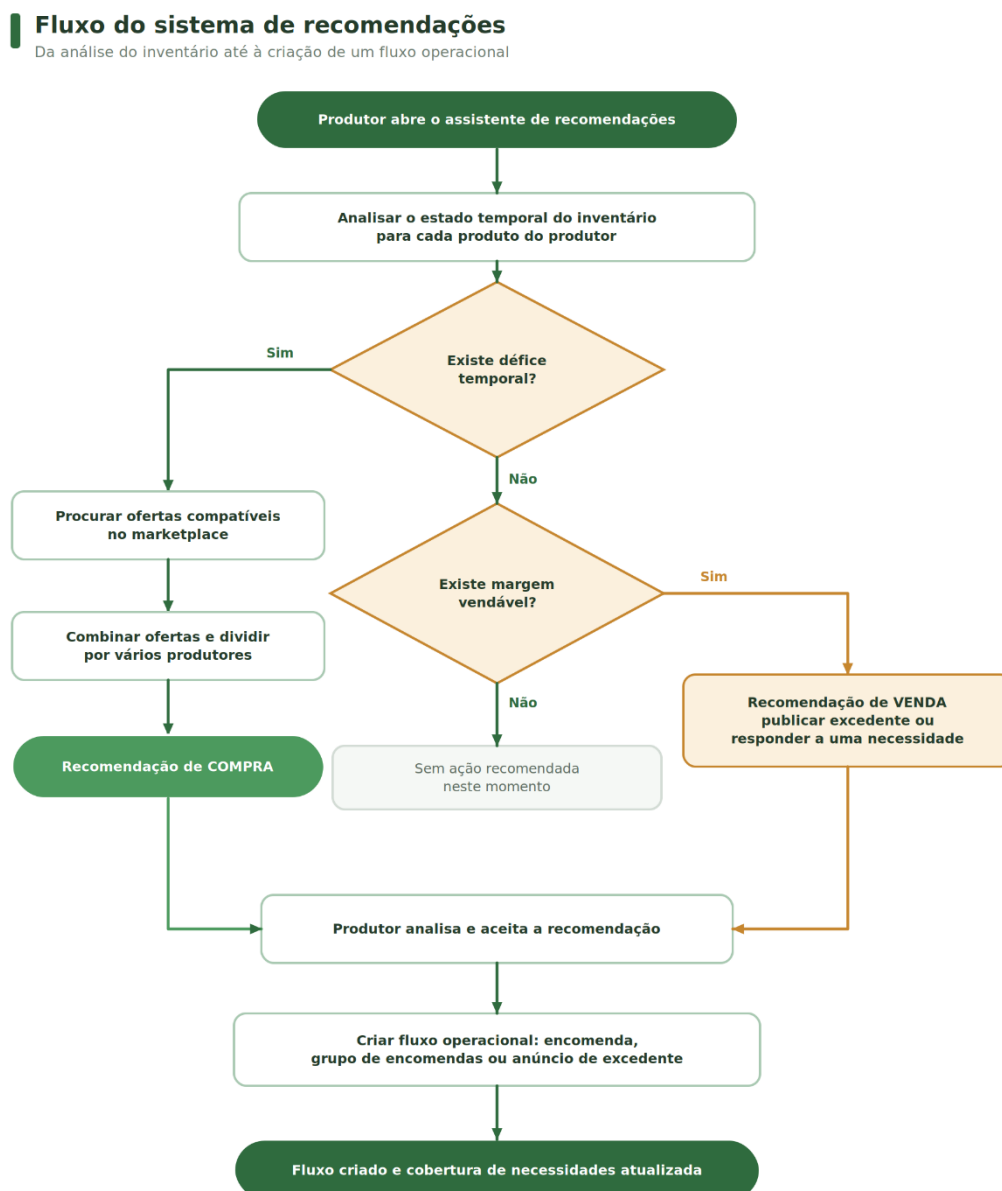


Figura 6.8 — Fluxo de decisão do sistema de recomendações: do estado temporal do inventário à criação do fluxo operacional

6.5.2 Recomendações de compra e venda

As recomendações de compra foram implementadas para apoiar produtores que precisam de determinado produto. Quando a plataforma identifica uma falta, procura anúncios públicos

compatíveis no *marketplace*. Estes anúncios podem estar associados a *stock* atual ou a pré-venda, desde que sejam compatíveis com o produto e com a quantidade necessária.

A recomendação apresenta ao produtor as opções encontradas, incluindo vendedor, produto, quantidade sugerida, preço unitário e subtotal. Sempre que possível, a plataforma procura cobrir a necessidade com ofertas disponíveis, dando prioridade a soluções que consigam responder ao pedido de forma mais direta.

Quando não existe cobertura suficiente no *marketplace*, o produtor pode ser encaminhado para a criação de uma necessidade. Desta forma, a falta deixa de ser apenas um problema interno do produtor e passa a ser visível como uma oportunidade para outros membros da cooperativa responderem.

As recomendações de venda seguem a lógica inversa. Quando o produtor tem margem vendável, a plataforma identifica oportunidades para transformar esse excedente em ação. Essa ação pode consistir na publicação de um anúncio no *marketplace* ou na resposta direta a uma necessidade aberta de outro produtor.

Esta bidirecionalidade foi uma parte importante do módulo. O sistema não se limita a perguntar “o que falta?”, mas também ajuda a responder “o que pode ser vendido?”. Assim, o produtor recebe apoio tanto para repor produtos em falta como para aproveitar excedentes de forma mais organizada.

A Figura 6.9 apresenta uma recomendação de compra gerada pelo sistema, com indicação do produto em déficit, da quantidade sugerida e das ofertas compatíveis identificadas no *marketplace*.

Recomendações Inteligentes

Veja onde precisa de comprar, onde tem excedente para vender e avance com a ação certa.

1
Produto

Escolha o produto para avaliar compra ou venda.

2
Sugestão

O sistema indica a ação recomendada para o stock.

3
Confirmar

Valide o resumo final antes de gerar a encomenda.

Passo 2 — Sugestão de compra

O sistema selecionou ofertas disponíveis a tempo para cobrir a quantidade necessária.

Compra para cumprir pedidos dos seus clientes. Esta recomendação está ligada à procura de Cevada com data crítica em 30/06/2026. Ao confirmar, a encomenda contará para a cobertura dessa procura e não será tratada apenas como reposição normal de stock.

PRODUTO	QUANTIDADE A COMPRAR	DÉFICE RESTANTE	TOTAL ESTIMADO
Cevada	450	0	324,00 €

A recomendação cobre a quantidade a comprar. Pode avançar para confirmar a encomenda.

A procura foi calculada a partir de pedidos externos dos seus clientes. Continua como sugestão interna até optar por a publicar.

Joel Silva
450 kg · 0,72 €/kg

324,00 €
subtotal

Disponível agora (prioridade)
Melhor opção inicial da recomendação
Entrega disponível

Outras opções no mercado

Podes abrir e fechar esta secção para comparar alternativas e substituir um produtor sugerido.

Ver outras opções no mercado

← Ajustar quantidades

Comprar para cumprir pedidos

Figura 6.9 — Recomendação de compra apresentada ao produtor com base no défice temporal do inventário

6.5.3 Combinação de ofertas e divisão por produtores

Uma das funcionalidades mais relevantes do sistema de recomendações é a possibilidade de combinar várias ofertas para satisfazer uma mesma necessidade. Esta situação ocorre quando nenhum produtor tem, individualmente, quantidade suficiente para cobrir o pedido, mas vários produtores conseguem, em conjunto, aproximar-se ou atingir a quantidade necessária.

Nestes casos, a plataforma divide a sugestão por vários vendedores. Cada parcela da recomendação fica representada como um item, com o respetivo vendedor, produto, quantidade sugerida, preço unitário e subtotal. Esta estrutura permite ao produtor perceber não apenas o total recomendado, mas também a contribuição de cada vendedor.

A divisão por produtores aproxima o sistema da realidade cooperativa. Em vez de depender apenas de uma oferta ideal, a plataforma consegue aproveitar várias disponibilidades parciais existentes na rede. Isto é especialmente relevante em contextos agrícolas, onde diferentes produtores podem ter pequenas quantidades disponíveis em momentos distintos.

O utilizador mantém controlo sobre a recomendação. Os itens sugeridos podem ser apresentados como selecionados ou não selecionados, permitindo que o produtor ajuste a decisão antes de avançar. Assim, o sistema propõe uma combinação, mas a confirmação final continua a depender do utilizador.

Esta lógica está suportada pela estrutura `recommendation_items`, que representa cada parcela da solução sugerida e mantém ligação ao anúncio recomendado, ao vendedor, ao produto, à quantidade e ao valor económico da sugestão.

6.5.4 Criação automática de fluxos operacionais

A recomendação não termina na apresentação da sugestão. Quando o produtor aceita uma recomendação, a plataforma transforma essa decisão num fluxo operacional concreto. Dependendo do caso, pode ser criada uma encomenda, um grupo de encomendas, uma resposta a uma necessidade ou um anúncio de excedente.

No caso de uma recomendação de compra, a aceitação pode originar uma ou mais encomendas. Se todos os itens selecionados pertencerem ao mesmo vendedor, o fluxo é simples e gera uma encomenda associada a esse vendedor. Se a recomendação estiver dividida por vários vendedores, a plataforma pode criar um grupo de encomendas, mantendo cada vendedor numa encomenda separada.

Esta separação é importante porque uma encomenda individual não deve misturar vendedores diferentes. O comprador pode acompanhar o grupo global, mas cada vendedor gere apenas a sua própria encomenda. Esta regra está prevista no modelo funcional do projeto, onde os grupos de encomenda agregam múltiplas subencomendas originadas por uma recomendação dividida por vendedor .

Quando a recomendação está associada a uma necessidade, a aceitação da sugestão também pode atualizar a cobertura dessa necessidade. Assim, o sistema mantém coerência entre a recomendação, a encomenda criada e o estado da necessidade que motivou o processo.

No caso de uma recomendação de venda, o sistema pode encaminhar o produtor para publicar um excedente no *marketplace* ou responder diretamente a uma necessidade existente. Desta forma, a recomendação funciona como ponto de partida para uma ação concreta, e não apenas como uma indicação informativa.

Esta ligação entre recomendação e ação foi uma das decisões mais importantes do módulo. A plataforma não se limita a apresentar uma análise; procura transformar essa análise num passo operacional que o produtor consegue executar dentro do próprio sistema.

6.6 Gestão de Encomendas e Fluxos Operacionais

Depois de o sistema de recomendações ter sido capaz de transformar informação operacional em sugestões concretas, o passo seguinte foi garantir que essas sugestões pudessem dar origem a transações reais entre produtores. A gestão de encomendas constitui, nesse sentido, o módulo que concretiza a passagem da intenção à ação: é aqui que uma recomendação aceite, uma compra direta no *marketplace* ou a aceitação de uma proposta privada se transformam num compromisso formal entre comprador e vendedor.

Este módulo foi implementado de forma a acompanhar todo o ciclo de vida de uma encomenda, desde a sua criação até à conclusão ou cancelamento. Cada etapa desse ciclo tem consequências noutras áreas da plataforma, nomeadamente no inventário, nas necessidades, nos alertas e nas notificações. Por esta razão, a gestão de encomendas não funciona como uma funcionalidade isolada, mas sim como um eixo central que liga vários módulos entre si.

6.6.1 Criação de encomendas

Uma encomenda pode ser criada a partir de três fluxos distintos. O primeiro corresponde à compra direta de um anúncio no *marketplace*: quando um produtor encontra uma oferta que lhe interessa, pode selecionar a quantidade pretendida e avançar para a criação da encomenda diretamente a partir da página de detalhe do anúncio. O segundo fluxo resulta da aceitação de uma recomendação de compra: quando o sistema sugere uma ou mais ofertas para cobrir uma necessidade, o produtor pode confirmar a sugestão e a plataforma cria automaticamente a encomenda correspondente. O terceiro fluxo está associado à aceitação de uma proposta privada recebida em resposta a uma necessidade: o dono da necessidade pode consultar as propostas recebidas e, ao aceitar uma delas, dá origem a uma nova encomenda.

Em todos os casos, a criação da encomenda regista a informação necessária para a transação: o comprador, o vendedor, o produto, a quantidade, o preço, as condições de entrega e, quando

aplicável, a ligação à necessidade ou ao anúncio que originou a compra. Esta informação permite que ambas as partes acompanhem o estado da encomenda e que a plataforma mantenha coerência entre os diferentes módulos.

Para proteger a integridade dos dados em cenários de utilização simultânea, as operações de compra foram implementadas com mecanismos de bloqueio transacional compatíveis com PostgreSQL. Esta precaução garante que dois produtores não consigam, ao mesmo tempo, comprar a mesma quantidade de um anúncio, evitando situações em que a quantidade vendida ultrapassa a quantidade efetivamente disponível.

6.6.2 Grupos de encomendas e relações entre entidades

Quando uma recomendação de compra envolve vários vendedores, a plataforma não cria uma única encomenda com múltiplos vendedores. Em vez disso, é criado um grupo de encomendas, que agrega várias sub-encomendas, cada uma associada a um vendedor diferente. Esta separação é necessária porque cada vendedor gere apenas a sua própria encomenda, com as suas quantidades, os seus estados e as suas ações.

O comprador, por sua vez, pode consultar o grupo como um todo, acompanhando o progresso global da compra. Desta forma, a plataforma consegue representar uma situação frequente em contexto cooperativo: a necessidade de adquirir um produto a vários produtores diferentes, cada um com uma parte da quantidade pretendida.

Ao nível do modelo de dados, cada encomenda contém um ou mais itens. Cada item representa uma linha da transação, com o produto, a quantidade, o preço unitário e o subtotal correspondente. Quando a encomenda está associada a uma necessidade, o item guarda também essa ligação, permitindo que a plataforma recalcule automaticamente a cobertura da necessidade à medida que as encomendas avançam.

Esta estrutura — grupos, encomendas e itens — permitiu manter a flexibilidade necessária para representar desde compras simples, com um único vendedor e um único produto, até cenários mais complexos em que uma recomendação dividida por vários produtores origina múltiplas sub-encomendas agrupadas sob o mesmo contexto de compra.

6.6.3 *Workflow de estados e timeline operacional*

Cada encomenda segue um ciclo de vida composto por seis estados possíveis: pendente, confirmada, em progresso, em entrega, concluída e cancelada. Este ciclo foi desenhado para refletir as etapas naturais de uma transação entre produtores, desde o momento em que o pedido é criado até à sua conclusão ou eventual cancelamento.

Quando uma encomenda é criada, entra no estado pendente. Neste momento, o vendedor é notificado e pode decidir aceitar ou recusar o pedido. Ao aceitar, a encomenda avança para o estado de confirmada. A partir daí, o vendedor pode indicar que está a tratar do produto, passando a encomenda para o estado em progresso, e posteriormente registar o envio, transitando para o estado em entrega. Quando o comprador confirma a receção, a encomenda é marcada como concluída.

A qualquer momento antes da conclusão, qualquer uma das partes pode cancelar a encomenda, indicando um motivo e, opcionalmente, notas adicionais. O cancelamento tem consequências operacionais, nomeadamente a libertação de quantidades reservadas, conforme descrito na secção seguinte.

Para manter o registo completo da evolução de cada encomenda, todas as transições de estado são guardadas numa tabela de histórico. Cada entrada regista o estado atribuído, o utilizador responsável, notas de contexto quando aplicável e o momento em que ocorreu. Esta informação permite reconstituir a *timeline* completa da encomenda e contribui para a rastreabilidade da plataforma.

Na interface, o detalhe da encomenda foi organizado de forma a apresentar a informação de forma clara e orientada à ação. O comprador e o vendedor veem a mesma encomenda, mas as ações disponíveis diferem consoante o perfil: o vendedor pode aceitar, preparar e enviar; o comprador pode confirmar a receção. A área de cancelamento foi posicionada de forma separada, com campos para motivo e observações, evitando que esta ação fosse confundida com as restantes.

A Figura 6.10 apresenta o detalhe de uma encomenda em progresso, com a *timeline* de estados percorridos e as ações disponíveis para o vendedor.

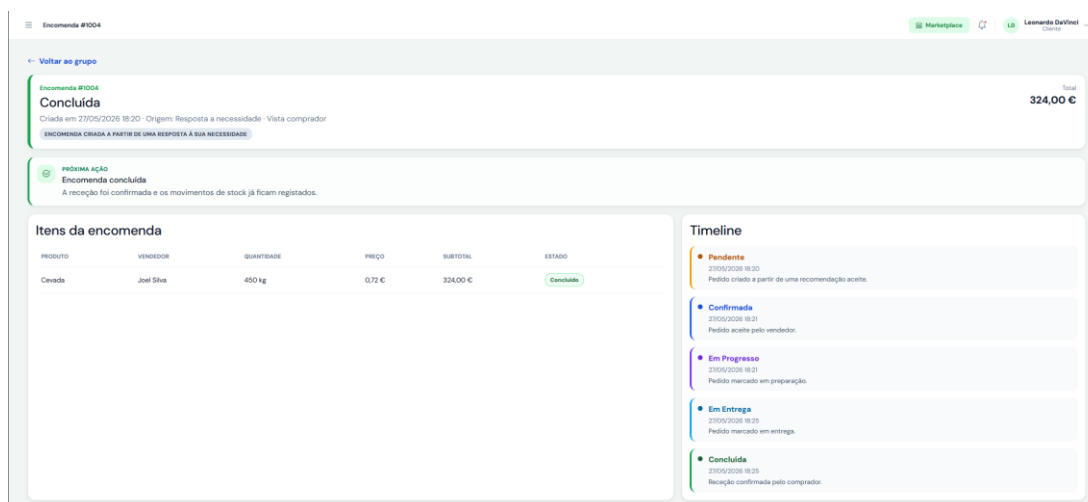


Figura 6.10 — Detalhe de uma encomenda e timeline de estados

6.6.4 Reservas, receção e atualização de stock

A criação de uma encomenda não é apenas um registo administrativo. Do ponto de vista operacional, uma encomenda pendente ou confirmada implica que uma determinada quantidade de produto está comprometida. Para refletir esta realidade, a plataforma reserva automaticamente a quantidade encomendada no *stock* do vendedor no momento da criação.

Esta reserva garante que o *stock* disponível do vendedor é atualizado imediatamente, evitando que a mesma quantidade seja vendida duas vezes. Enquanto a encomenda estiver ativa, a quantidade reservada permanece indisponível para novas vendas ou anúncios.

Quando a encomenda é cancelada, o processo inverte-se: a reserva é libertada e a quantidade volta a estar disponível no inventário do vendedor. Esta libertação é feita de forma automática, garantindo que o cancelamento não deixa quantidades bloqueadas sem justificação.

Na conclusão da encomenda, a reserva é consumida e são registados os movimentos de *stock* aplicáveis. Do lado do vendedor, a operação reflete a saída da quantidade vendida; do lado do comprador, a receção pode refletir a entrada no inventário, de acordo com o fluxo operacional definido.

Quando a encomenda está associada a uma necessidade, a sua conclusão contribui também para a atualização da cobertura dessa necessidade. Desta forma, uma necessidade que estava parcialmente

coberta pode passar a estar totalmente coberta após a receção de uma ou mais encomendas, fechando o ciclo entre procura, recomendação, compra e entrega.

Quando a encomenda tem origem numa proposta privada a uma necessidade, a aceitação ou rejeição da proposta é também refletida no estado da resposta. Assim, o sistema mantém coerência entre a proposta recebida, a encomenda criada e o estado da necessidade que motivou todo o processo.

6.7 Alertas, Notificações e Comunicação

Uma plataforma que gere stocks, necessidades, encomendas e prazos precisa de ser capaz de chamar a atenção do utilizador no momento certo. Sem mecanismos de alerta e comunicação, o produtor teria de verificar manualmente cada área da aplicação para perceber se algo mudou ou se existe alguma situação que exige a sua atenção. Esta secção descreve os três mecanismos implementados para resolver este problema: alertas operacionais, notificações informativas e mensagens entre produtores.

6.7.1 Alertas operacionais e deduplicação

Os alertas operacionais foram implementados para sinalizar situações que exigem atenção ou acompanhamento por parte do produtor. Ao contrário de uma notificação genérica, um alerta operacional está ligado a uma condição concreta do sistema: um produto com *stock* crítico, uma necessidade sem cobertura, um prazo a aproximar-se, uma oportunidade de compra ou venda, uma encomenda que aguarda confirmação ou uma entrega em atraso.

A geração de alertas está ligada à mesma fonte central de cálculo utilizada pelo inventário e pelas recomendações. Desta forma, um alerta de *stock* crítico só é criado quando existe um défice temporal real, ou seja, quando o *stock* atual e a produção futura útil não são suficientes para cobrir os compromissos externos a tempo. Esta precaução evita que o sistema gere falsos alarmes em situações onde a produção prevista é suficiente para responder aos pedidos.

Na página de alertas, a informação foi organizada em separadores — ativos, adiados e resolvidos — e em secções temáticas dentro dos alertas ativos: ações urgentes, risco agrícola, oportunidades e informação. Esta organização permite ao produtor distinguir rapidamente entre situações que exigem ação imediata e informações de acompanhamento.

O produtor pode resolver um alerta, adiá-lo para revisão posterior ou adiar todos os alertas visíveis de uma só vez. Estas ações foram implementadas com atualizações parciais da interface, permitindo que as listas de alertas fossem atualizadas sem recarregar toda a página.

Para evitar a repetição de alertas sobre a mesma situação, foi implementada lógica de deduplicação. Quando uma condição operacional já tem um alerta ativo associado, o sistema atualiza o alerta existente em vez de criar um novo. Desta forma, o produtor não recebe múltiplos alertas sobre o mesmo problema, mantendo a página de alertas limpa e orientada à ação.

Os indicadores visuais de alertas pendentes são apresentados na barra superior da aplicação. Estes indicadores são atualizados em tempo real através de Django Channels e Redis, quando disponíveis. Caso o serviço de comunicação em tempo real não esteja acessível, a página continua a funcionar normalmente através do carregamento tradicional, garantindo que uma falha técnica no Redis não impede o acesso aos alertas.

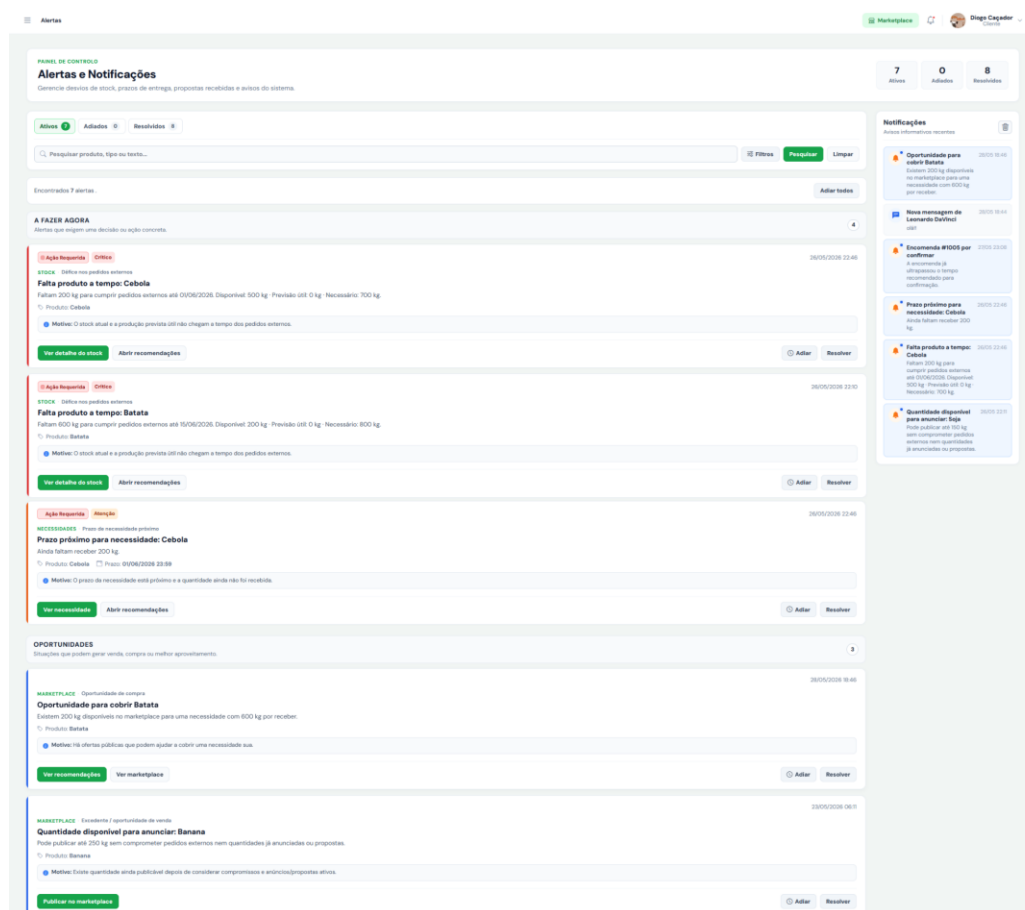


Figura 6.11 — Página de alertas operacionais com separadores e secções temáticas

6.7.2 Notificações informativas

Para além dos alertas operacionais, a plataforma inclui um sistema de notificações informativas, destinado a registar eventos recentes de menor urgência. Estas notificações complementam os alertas, informando o produtor sobre situações como mensagens não lidas, alterações no estado de encomendas ou oportunidades derivadas de alertas.

As notificações informativas são apresentadas na página de alertas, numa secção dedicada que mostra as últimas notificações recentes do utilizador. O produtor pode limpar as notificações quando já não forem relevantes, sem que essa ação afete os alertas operacionais. Esta separação foi importante para garantir que limpar uma notificação não resolve nem apaga um alerta real que possa exigir acompanhamento.

Para evitar repetições, as notificações são deduplicadas por utilizador e por alerta. Se uma nova notificação disser respeito a um alerta já notificado, a notificação existente é atualizada em vez de se criar uma nova entrada. Esta lógica evita, por exemplo, que o produtor receba várias notificações repetidas sobre a mesma oportunidade de compra.

6.7.3 Mensagens entre produtores e comunicação em tempo real

O módulo de mensagens permite a comunicação direta entre produtores, através de conversas individuais dentro da plataforma. Esta funcionalidade foi pensada para complementar os fluxos de compra e venda, permitindo que produtores esclareçam dúvidas, negoceiem condições ou combinem detalhes logísticos sem necessidade de recorrer a canais externos.

Uma conversa pode ser iniciada a partir de um anúncio no *marketplace* ou a partir de uma encomenda. Se já existir uma conversa anterior entre os mesmos dois produtores nesse contexto, a plataforma reutiliza a conversa existente em vez de criar uma nova. Esta decisão evita a dispersão de mensagens por conversas duplicadas e mantém todo o histórico de comunicação num único local.

A comunicação é suportada por *WebSocket*, o que permite que as mensagens cheguem ao destinatário de forma imediata, sem necessidade de recarregar a página. Na interface, um indicador mostra o estado da ligação em tempo real do utilizador à conversa. Este indicador reflete o estado técnico da ligação *WebSocket* do próprio utilizador e não a presença do outro participante, evitando criar expectativas incorretas sobre disponibilidade.

A interface foi desenhada para funcionar tanto em ecrãs de computador como em dispositivos móveis. Em ecrãs maiores, a página apresenta a lista de conversas e o conteúdo da conversa selecionada lado a lado. Em ecrãs mais pequenos, a lista e a conversa alternam conforme a navegação, com um botão para voltar à lista de conversas. A área de composição de mensagens ajusta-se automaticamente ao conteúdo, e os erros são apresentados através de avisos visuais não intrusivos.

As mensagens de texto têm um limite aplicado ao nível do servidor, e o envio está protegido por limites de frequência por utilizador e por conversa, reduzindo o risco de utilização abusiva. A abertura da página de mensagens sem uma conversa explicitamente selecionada não marca automaticamente a primeira conversa como lida, respeitando a intenção do utilizador.

Para além de mensagens de texto, a plataforma suporta eventos de sistema dentro das conversas. Estes eventos são mensagens geradas automaticamente para registar ações relevantes, como a criação de uma encomenda ou o envio de uma proposta, mantendo o contexto da relação comercial dentro da conversa.

Quando uma mensagem nova chega a uma conversa que o utilizador tenha previamente arquivado, a conversa regressa automaticamente à lista de conversas ativas, garantindo que mensagens importantes não ficam escondidas.

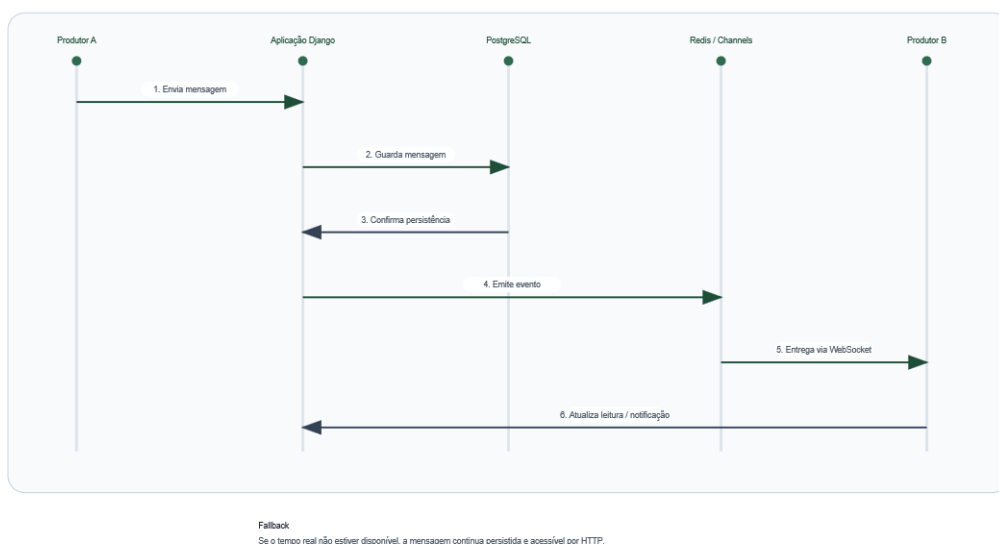


Figura 6.12 — Fluxo de envio de mensagem em tempo real

Mesmo que a comunicação em tempo real não esteja temporariamente disponível, a mensagem continua registada na base de dados e pode ser consultada através do carregamento normal da página, garantindo que a falha do canal em tempo real não implica perda de informação.

6.7.4 Gestão de anexos e conversas

Para além de texto, as mensagens podem incluir anexos. O envio de ficheiros é realizado através de um pedido HTTP separado, sendo o ficheiro armazenado através do serviço de *storage* configurado — em produção, o Cloudinary. Antes de aceitar o ficheiro, o sistema valida a extensão, o tipo de conteúdo e o tamanho, que não pode exceder dez megabytes.

Quando o anexo é uma imagem, a plataforma apresenta uma miniatura clicável diretamente na conversa, permitindo ao utilizador pré-visualizar o conteúdo sem sair da página de mensagens. Para ficheiros de outros tipos, é apresentado um cartão de ficheiro com a informação relevante. Na interface, cada anexo apresenta o seu estado individual — pronto, a enviar, enviado ou falhou — permitindo ao utilizador acompanhar o progresso do envio.

Foi também tida em consideração a possibilidade de o envio do anexo ser bem-sucedido ao nível do armazenamento, mas falhar na difusão em tempo real. Nesse cenário, a mensagem é guardada normalmente e o utilizador é informado através de um aviso, em vez de receber um erro que pudesse sugerir que o envio falhou por completo. Esta decisão contribuiu para uma experiência mais resiliente, em que falhas pontuais no serviço de comunicação em tempo real não comprometem a entrega efetiva das mensagens.

As notificações de mensagens não lidas são deduplicadas por conversa. Quando o utilizador abre uma conversa com mensagens por ler, as notificações associadas a essa conversa são automaticamente marcadas como lidas, mantendo os indicadores da plataforma atualizados.

A Figura 6.13 apresenta a interface de mensagens entre produtores, evidenciando a organização das conversas, o estado da ligação em tempo real e o envio de anexos durante a comunicação.

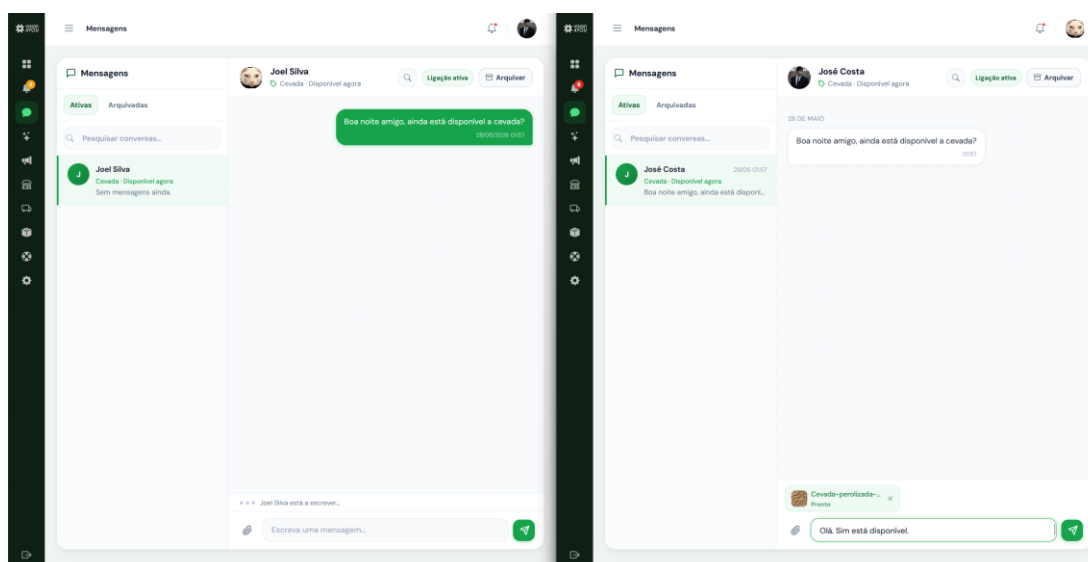


Figura 6.13 — Conversa entre produtores com comunicação em tempo real e envio de anexos

6.8 Suporte, Administração e Monitorização

Para que a plataforma pudesse funcionar de forma sustentável em ambiente real, era necessário garantir dois aspetos complementares: que os produtores tivessem um canal de apoio quando precisassem de ajuda, e que os administradores conseguissem acompanhar o estado geral da plataforma. Esta secção descreve os módulos de suporte, administração e monitorização implementados para responder a essas necessidades.

6.8.1 Sistema de suporte ao utilizador

O suporte ao utilizador foi implementado como um sistema de *tickets* conversacional. Quando um produtor precisa de ajuda, pode criar um pedido de suporte a partir da página dedicada, descrevendo o problema ou a dúvida e, opcionalmente, anexando imagens. Cada pedido de suporte é registado como um *ticket*, que pode ser acompanhado tanto pelo produtor como pela administração.

A comunicação dentro de cada *ticket* funciona como uma conversa. O produtor pode enviar novas mensagens enquanto o *ticket* estiver aberto, e o administrador pode responder sem que essa resposta feche automaticamente o pedido. O fecho do *ticket* é uma ação explícita, realizada pelo administrador quando considera que a questão foi resolvida. Esta decisão evita que uma resposta parcial ou uma pergunta de acompanhamento encerrem prematuramente o pedido.

Os anexos de suporte seguem as mesmas regras de validação aplicadas noutras áreas da plataforma: apenas imagens nos formatos mais comuns são aceites, com um limite de dez megabytes por ficheiro. Em produção, os anexos são armazenados no Cloudinary.

Do lado do administrador, a gestão de suporte é feita através de uma página dedicada no painel administrativo. O administrador pode consultar a fila de *tickets*, assumir um *ticket* para si, responder, e marcar como resolvido quando a questão estiver tratada. A listagem dos *tickets* é ordenada pela última interação, facilitando a identificação de pedidos que aguardam resposta.

Cada *ticket* passa por três estados: aberto, assumido e fechado. O estado assumido indica que um administrador está a tratar do pedido, dando visibilidade ao produtor de que o seu pedido foi recebido e está a ser acompanhado.

A página de suporte inclui também uma secção de perguntas frequentes, que permite ao utilizador consultar respostas a dúvidas comuns sobre a plataforma antes de abrir um *ticket*. Esta secção abrange temas como recuperação de conta, interpretação de indicadores de *stock*, funcionamento do *marketplace*, gestão de necessidades e apoio a encomendas.

A Figura 6.14 apresenta o detalhe de um *ticket* de suporte no painel administrativo, incluindo a informação do utilizador, o histórico da conversa, o estado do pedido e as ações disponíveis para resposta e resolução.

← Voltar

Ticket #1002

Bug no publicar anúncio.

Nome: Diogo Caçador

Email: diogo.basket.26@gmail.com

Role: CLIENTE

Empresa: Agrijoja

Telefone: —

Conversa

Mensagens trocadas entre o utilizador e a equipa de suporte.

Diogo Caçador 26/05/2026 10:42

Boa tarde, ao publicar anúncio deparei-me com um bug quando tento escolher o método de entrega não aparece a opção de escolher a distância que estou disposto a entregar o meu produto.

Admin Cooperativa 26/05/2026 10:43

Boa tarde! Já coloquei os nossos melhores vixes coders a tratar da situação. Aguarde uns momentos e já vai ficar tudo bem.

Responder

A resposta mantém o ticket aberto. Feche o ticket apenas quando o problema estiver resolvido.

Mensagem

Escreva uma resposta para continuar a conversa.

Imagens opcionais

Explorar... Nenhuns ficheiros selecionados.

JPG, PNG, WEBP ou GIF. Máximo 10MB por imagem.

Enviar resposta

Resumo

CREADO EM: 26/05/2026 10:42

CLAIMED EM: 26/05/2026 10:42

FECHADO EM: —

ADMINISTRADOR ATRIBUÍDO: Admin Cooperativa

Resolver

Marcar como resolvido

Figura 6.14 — Gestão administrativa/conversa de um ticket de suporte com histórico de conversa

Esta organização permite que o administrador acompanhe o pedido sem perder o contexto da conversa, responda diretamente ao utilizador e apenas encerre o *ticket* quando o problema estiver resolvido.

6.8.2 *Dashboard* do produtor e painel administrativo

A plataforma inclui dois painéis distintos, adequados a cada perfil de utilizador.

O painel do produtor constitui a página inicial da aplicação após a autenticação. Este painel foi desenhado para apresentar uma visão operacional do dia a dia do produtor, reunindo num único ecrã os indicadores mais relevantes: estado do inventário, encomendas em curso, necessidades por cobrir, oportunidades de compra ou venda e informação meteorológica da localização da exploração.

A informação sobre o inventário no painel utiliza a mesma fonte central de cálculo que alimenta os alertas e as recomendações. Desta forma, o painel mostra como produtos críticos apenas aqueles que apresentam défice temporal real, considerando o *stock* disponível e a produção futura útil antes de sinalizar qualquer risco. Esta coerência entre painéis, alertas e recomendações foi uma preocupação constante ao longo do desenvolvimento.

As ações sugeridas no painel adaptam-se ao estado do inventário do produtor. Se existem produtos em risco, são apresentadas ligações diretas para as recomendações. Se existe margem vendável, é sugerida a publicação de excedente. Se o produtor ainda não tem produtos ativos, a ação principal encaminha para o inventário. Esta abordagem permite que o painel não se limite a mostrar números, mas sugira ao produtor o próximo passo a tomar.

O *widget* de clima apresenta informação meteorológica resumida da localização do produtor, incluindo estado do tempo, temperatura e previsão de chuva. Quando o produtor ainda não definiu a sua localização, o *widget* apresenta uma ação que o encaminha para as definições, onde pode registar a sua morada ou utilizar a funcionalidade de geolocalização automática do navegador.

O painel administrativo, por sua vez, apresenta indicadores globais da plataforma: número de utilizadores, *tickets* de suporte ativos, encomendas em curso e atividade comercial. Um gráfico de atividade semanal mostra a evolução de encomendas criadas e itens concluídos nas últimas doze semanas, permitindo ao administrador acompanhar a dinâmica da cooperativa. O painel inclui

ainda acesso direto à gestão de utilizadores, ao catálogo de produtos e categorias, à auditoria e ao suporte.

As Figuras 6.15 e 6.16 apresentam, respetivamente, o painel do produtor e o painel administrativo, evidenciando a diferença entre a visão operacional individual e a visão global de gestão.

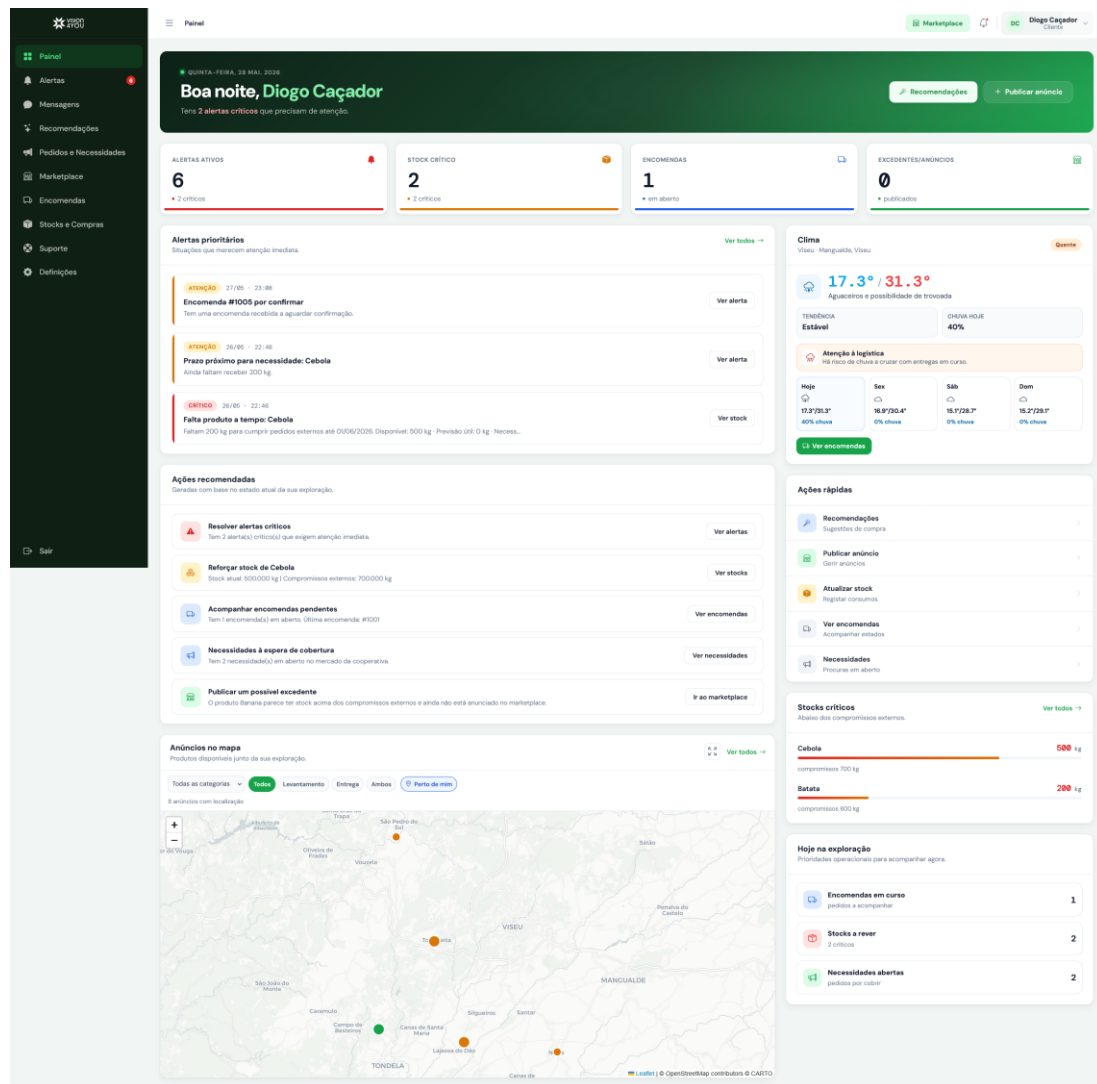


Figura 6.15 — Painel do produtor com alertas, indicadores de stock, clima e ações recomendadas

Seguidamente é demonstrado o painel do administrador e todas as suas funcionalidades na *sidebar* na figura 6.16.

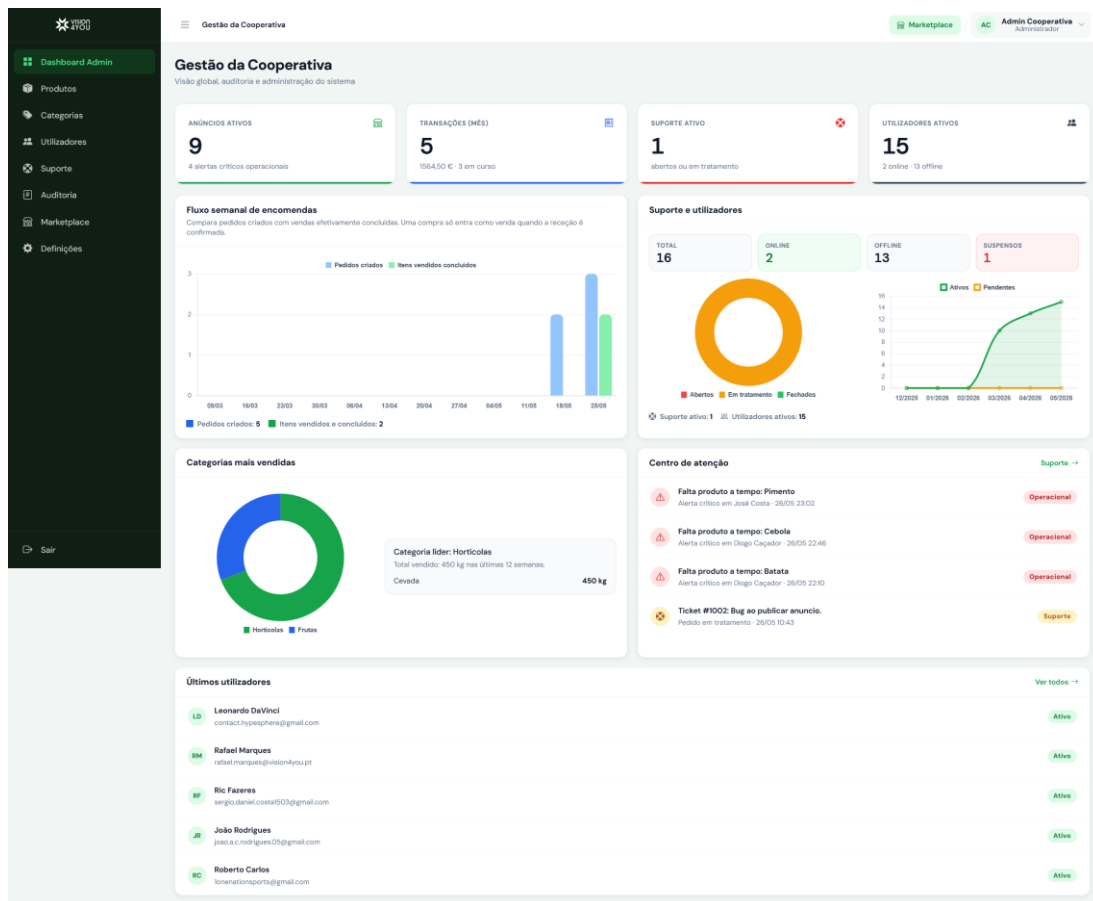


Figura 6.16 — Painel administrativo com métricas globais, suporte ativo e atividade comercial

6.8.3 Auditoria e registo técnico de operações

Conforme descrito na secção 6.2.3, a plataforma regista as operações mais relevantes numa tabela de auditoria. Embora a implementação técnica da auditoria tenha sido apresentada no contexto da segurança, o seu papel no acompanhamento operacional justifica uma referência nesta secção.

No painel administrativo, a auditoria é apresentada numa página dedicada, com pesquisa por tipo de ação, utilizador, endereço IP, descrição do dispositivo e conteúdo das alterações. Cada linha da tabela é expansível, permitindo ao administrador consultar o detalhe completo do registo sem sair da página.

O detalhe expandido foi organizado para apresentar primeiro um resumo operacional legível, seguido das alterações realizadas e, por fim, a referência técnica completa. Esta hierarquia visual foi pensada para que a informação mais útil para o administrador apareça primeiro, reservando os detalhes técnicos para uma consulta mais aprofundada quando necessário.

A tabela de auditoria suporta paginação e filtros, permitindo ao administrador navegar por grandes volumes de registos sem comprometer o desempenho da página. Em conjunto, estas funcionalidades contribuem para uma maior rastreabilidade e controlo sobre as ações realizadas na plataforma.

6.9 Colocação em Produção e Operação da Plataforma

A última fase das atividades desenvolvidas correspondeu à colocação da plataforma em ambiente de produção. Até este ponto, toda a aplicação tinha sido desenvolvida e testada em ambiente local. No entanto, para validar a solução como um sistema funcional e demonstrar que a plataforma era capaz de operar fora do contexto de desenvolvimento, foi necessário configurar uma infraestrutura de produção e preparar a aplicação para funcionar de forma autónoma.

As tecnologias e serviços utilizados em produção foram apresentados no Capítulo 5; nesta secção descreve-se o processo de configuração e validação da plataforma em ambiente real.

6.9.1 Organização da infraestrutura

A infraestrutura de produção foi organizada no Railway, uma plataforma de alojamento na *cloud* que permite configurar e executar serviços sem necessidade de gerir servidores físicos ou máquinas virtuais. No Railway foram criados três serviços principais: a aplicação *web*, a base de dados PostgreSQL e o Redis.

A aplicação *web* é executada como um serviço Django sobre ASGI, utilizando o Daphne como servidor. Esta configuração permite servir tanto pedidos HTTP tradicionais como ligações *WebSocket*, suportando as funcionalidades de comunicação em tempo real sem necessidade de um servidor adicional.

A base de dados PostgreSQL de produção armazena todos os dados operacionais da plataforma, incluindo utilizadores, produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, mensagens, alertas e suporte. O acesso à base de dados é feito exclusivamente através de variáveis de ambiente, garantindo que as credenciais não ficam expostas no código-fonte.

O Redis suporta duas responsabilidades: a distribuição de eventos em tempo real, através do Django Channels, e o funcionamento como camada de *cache* para integrações auxiliares, como os dados meteorológicos apresentados no painel do produtor.

Para além destes serviços, a infraestrutura depende de dois serviços externos. O Cloudinary é utilizado para armazenar ficheiros enviados pelos utilizadores — como fotografias de anúncios, imagens de perfil, anexos de mensagens e imagens de suporte — e também para alojar os elementos visuais de marca da plataforma, como logótipos e *favicon*. O Resend é utilizado para o envio de *emails* transacionais, incluindo confirmação de conta, recuperação de palavra-passe, notificações de segurança e comunicações de suporte.

Esta organização permitiu que a aplicação não dependesse do sistema de ficheiros local do contentor, que é efémero e não persiste entre reinícios. Ao separar o armazenamento de ficheiros, a persistência de dados e o envio de *emails* em serviços dedicados, a plataforma ficou preparada para funcionar de forma mais estável e independente do ciclo de vida do contentor de execução.

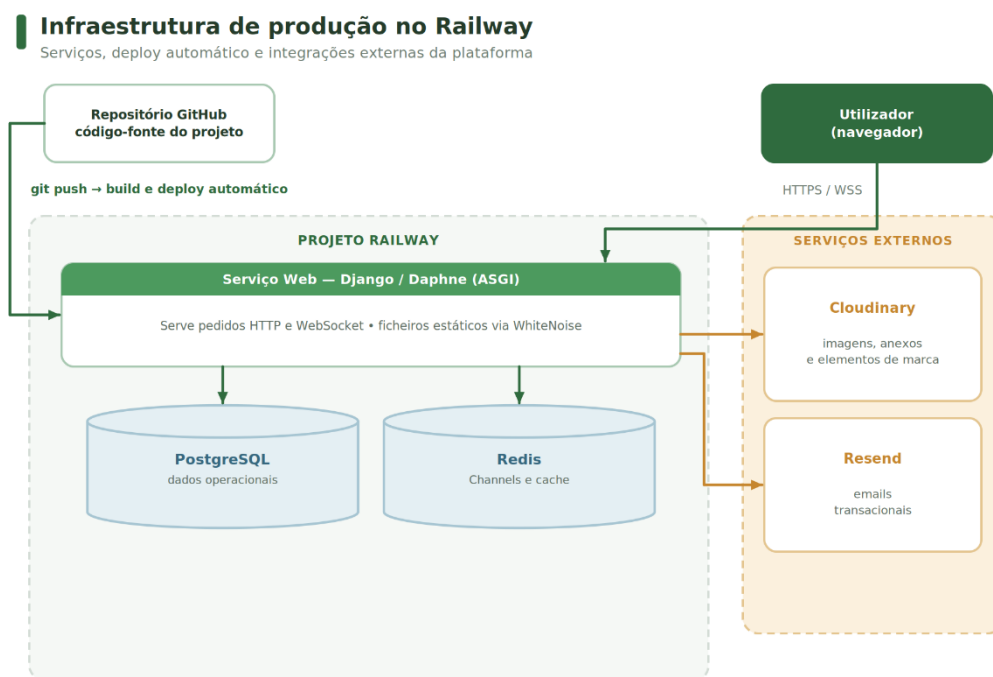


Figura 6.17 — Infraestrutura de produção no Railway: serviços internos, deploy automático a partir do GitHub e integrações externas

6.9.2 Deploy da aplicação e serviços associados

O processo de publicação da aplicação foi associado ao repositório GitHub do projeto. Esta ligação permitiu que o código desenvolvido localmente pudesse ser publicado em produção de forma controlada: quando uma nova versão é enviada para o repositório, o Railway reconstrói e reinicia o serviço automaticamente.

Antes de cada publicação, foi executado o comando de validação do Django para verificar a configuração da aplicação. Este passo permitiu detetar antecipadamente problemas de configuração, variáveis em falta ou incompatibilidades entre o código e o ambiente de produção.

A preparação dos ficheiros estáticos para produção foi feita com recurso ao WhiteNoise, que recolhe e comprime os ficheiros durante o processo de construção e os serve diretamente pela aplicação, sem necessidade de um servidor estático separado. Os elementos de marca dinâmicos, como logótipos e ícones, são servidos a partir de URLs do Cloudinary, configuráveis através de variáveis de ambiente.

O envio de *emails* em produção foi configurado com recurso ao Resend, integrado através da biblioteca *django-anymail*. Esta integração utiliza a API HTTP do Resend em vez do protocolo SMTP tradicional, simplificando a configuração e evitando dependências de servidores de *email* próprios. Em ambiente de desenvolvimento local, o envio de *emails* pode ser configurado com SMTP como alternativa.

6.9.3 Configuração operacional e execução em produção

A configuração da aplicação em produção foi organizada através de variáveis de ambiente, utilizando a biblioteca *python-decouple*. Cada definição sensível — como a chave secreta da aplicação, as credenciais da base de dados, as chaves de acesso ao Cloudinary e ao Resend, e os endereços de *email* — foi separada do código-fonte e definida exclusivamente no painel de configuração do Railway.

Esta separação contribuiu para uma configuração mais segura, uma vez que os segredos não ficam expostos no repositório. Ao mesmo tempo, permitiu manter configurações diferentes entre o ambiente de desenvolvimento local e o ambiente de produção, sem necessidade de alterar o código da aplicação.

Em produção, o modo de depuração foi explicitamente desativado, o que ativa os mecanismos de segurança do Django, como a proteção contra pedidos forjados (*Cross-Site Request Forgery*), a validação de domínios permitidos e a restrição de informação de erro apresentada ao utilizador.

Para manter o estado operacional da plataforma atualizado, foi prevista a execução periódica de um comando de sincronização. Este comando verifica e atualiza alertas operacionais, expira anúncios que ultrapassaram o seu prazo de validade e garante que o estado da plataforma reflete a realidade operacional atual. A sua execução pode ser configurada através do agendador do Railway ou de um mecanismo *cron* equivalente.

A colocação da aplicação em produção representou uma etapa importante do estágio, uma vez que permitiu validar a plataforma como uma solução funcional num ambiente real, ultrapassando a fase de projeto local. Embora o sistema não se destine, nesta fase, a utilização em larga escala, a sua execução em produção demonstrou que a arquitetura, as tecnologias e as decisões tomadas ao longo do desenvolvimento são viáveis e adequadas ao propósito da plataforma.

7 Resultados Obtidos

O presente capítulo apresenta os principais resultados obtidos com o desenvolvimento da plataforma VISION4FARMS. Depois de descritas as atividades realizadas, importa agora sintetizar as funcionalidades entregues, os resultados técnicos alcançados, os testes efetuados e o *feedback* considerado durante o desenvolvimento.

A análise dos resultados permite também avaliar em que medida os objetivos definidos inicialmente foram atingidos, comparar a solução desenvolvida com as plataformas estudadas no Estado da Arte e identificar limitações que poderão orientar trabalho futuro.

7.1 Síntese das funcionalidades Implementadas

A plataforma VISION4FARMS foi entregue como uma aplicação *web* funcional, organizada em módulos integrados e colocada em ambiente de produção. O sistema permite apoiar produtores agrícolas na gestão de produtos, *stock*, necessidades, anúncios, recomendações, encomendas, alertas, mensagens, suporte e administração.

Ao contrário de uma aplicação composta por páginas independentes, a solução desenvolvida articula vários fluxos entre si. Uma necessidade pode originar uma recomendação; uma recomendação pode criar encomendas; uma encomenda pode reservar ou movimentar *stock*; e estes eventos podem gerar alertas, notificações ou registos de auditoria.

A descrição detalhada das áreas funcionais implementadas é apresentada no Anexo C, Tabela C.1.

Em conjunto, estas funcionalidades permitiram cumprir o fluxo principal pretendido: registar informação operacional, identificar necessidades e excedentes, apoiar decisões de compra e venda, concretizar transações internas e acompanhar a operação através de alertas, comunicação e suporte.

7.2 Resultados Técnicos Obtidos

Do ponto de vista técnico, o principal resultado foi a construção de uma aplicação *web* modular, assente numa base de dados relacional e preparada para execução em produção.

A aplicação foi desenvolvida com Django e organizada em *apps* por domínio funcional. Esta divisão permitiu separar responsabilidades entre conta, catálogo, inventário, *marketplace*, necessidades, recomendações, encomendas, alertas, mensagens, suporte, *dashboard* e componentes comuns.

A base de dados foi implementada em PostgreSQL, seguindo uma abordagem *schema-first*. As tabelas de negócio foram definidas diretamente no esquema relacional e refletidas nos modelos Django com `managed = False`. Esta decisão deu maior controlo sobre a estrutura da base de dados, embora tenha exigido mais cuidado na gestão manual das alterações.

Um dos resultados técnicos mais relevantes foi a centralização do cálculo operacional do inventário através da função `calculate_inventory_commitment_state`. Esta lógica considera *stock* atual, reservas, compromissos externos, produção futura útil e margem vendável. Ao ser usada por diferentes módulos, como inventário, *dashboard*, alertas e recomendações, permitiu manter maior coerência entre as várias áreas da aplicação.

A comunicação em tempo real foi suportada por Django Channels, Redis e Daphne. Esta infraestrutura permitiu implementar funcionalidades como mensagens entre produtores, indicadores dinâmicos e suporte em tempo real, mantendo a aplicação compatível com pedidos HTTP tradicionais.

Em produção, a plataforma foi alojada no Railway, com PostgreSQL e Redis configurados como serviços associados. O Cloudinary foi utilizado para armazenar imagens, anexos e elementos de marca, enquanto o Resend assegurou o envio de *emails* transacionais. O WhiteNoise ficou responsável pelos ficheiros estáticos da aplicação.

Os principais resultados técnicos alcançados encontram-se sistematizados no Anexo C, Tabela C.2.

Estes resultados demonstram que a solução não ficou limitada a um protótipo local. A aplicação foi preparada para funcionar num ambiente real, com serviços externos, persistência de dados, comunicação em tempo real e mecanismos de segurança adequados ao âmbito do projeto.

7.3 Testes e Validação Funcional

A validação da plataforma foi realizada de forma progressiva ao longo do desenvolvimento. À medida que cada módulo era implementado, os seus fluxos principais eram testados antes de avançar para funcionalidades dependentes. Esta abordagem permitiu detetar problemas de forma contextualizada e reduzir inconsistências entre módulos.

Foram realizados testes manuais aos principais fluxos de utilização, incluindo registo de conta, autenticação, recuperação de palavra-passe, criação de produtos, atualização de stock, registo de produção futura, publicação de anúncios, criação de necessidades, recomendações, encomendas, mensagens e suporte.

Para ilustrar a profundidade da validação, descreve-se um cenário completo de utilização testado na plataforma. Para ilustrar a profundidade da validação, foi utilizado um cenário de teste representativo. Nesse cenário, um produtor (Produtor A) regista stock de 100 kg de trigo e compromissos externos de clientes que totalizam 180 kg com prazo a duas semanas. Possui ainda uma previsão de produção de 50 kg, mas com data de disponibilidade posterior ao prazo dos compromissos. A plataforma calcula que a produção futura não chega a tempo e classifica o produto como estando em défice temporal, gerando um alerta de stock crítico. O produtor consulta as recomendações e o sistema sugere a compra de 80 kg a dois outros produtores (Produtor B com 50 kg em stock e Produtor C com 30 kg em pré-venda), dividindo a sugestão por vendedores. O Produtor A aceita a recomendação, o que cria automaticamente um grupo de encomendas com duas sub-encomendas. Os vendedores confirmam e enviam. Quando o Produtor A confirma a receção, o stock é atualizado dos dois lados, a necessidade associada é marcada como coberta e o alerta é resolvido.

Este cenário permitiu validar, de forma integrada, o cálculo temporal de compromissos, a geração de alertas, o sistema de recomendações com divisão por vendedores, a criação automática de grupos de encomendas, o workflow de estados, a reserva e atualização de stock e a resolução de necessidades e alertas. A passagem bem-sucedida deste fluxo comprovou que os módulos comunicam corretamente entre si e que uma ação num ponto da plataforma se propaga de forma coerente pelas restantes áreas.

A validação dos pedidos externos de clientes foi particularmente exigente, porque esta funcionalidade afeta várias áreas da aplicação. Foram testados cenários em que os pedidos estavam cobertos pelo stock e pela produção futura útil, bem como situações em que existia défice temporal.

Nestes últimos casos, verificou-se que a aplicação gerava ou atualizava corretamente a necessidade agregada correspondente, e que a remoção do déficit (por entrada de stock ou conclusão de uma encomenda) marcava automaticamente a necessidade como coberta.

Também foram testados cenários de concorrência, nomeadamente a tentativa de dois produtores comprarem a mesma quantidade de um anúncio em simultâneo. Os mecanismos de bloqueio transacional implementados com `select_for_update()` do PostgreSQL evitaram que a quantidade vendida ultrapassasse a quantidade efetivamente disponível.

A comunicação em tempo real foi validada com dois navegadores abertos em simultâneo, simulando uma conversa entre dois produtores. Foram verificados o envio e a receção imediata de mensagens, o envio de anexos com pré-visualização, o comportamento do indicador de ligação e a resiliência do sistema quando o canal *WebSocket* era temporariamente interrompido — neste caso, as mensagens permaneciam na base de dados e ficavam disponíveis ao recarregar a página.

A síntese dos principais fluxos funcionais validados é apresentada no Anexo C, Tabela C.3.

Para além dos testes funcionais, foram realizadas verificações técnicas antes de cada publicação, nomeadamente através do comando `python manage.py check`, que permitiu detetar antecipadamente problemas de configuração ou incompatibilidades entre o código e o ambiente de produção.

Importa reconhecer que a validação foi realizada internamente, pelo próprio autor do projeto e em articulação com o orientador da empresa, sem envolver um grupo alargado de produtores reais. Esta limitação, embora natural no contexto de um estágio individual, implica que cenários de utilização menos previsíveis — como volumes elevados de dados, padrões de navegação inesperados ou necessidades de acessibilidade específicas — não foram ainda avaliados. Uma validação com utilizadores reais em contexto agrícola é identificada como uma das prioridades do trabalho futuro.

7.4 Feedback Recolhido e Ajustes Realizados

Para além da validação técnica, o desenvolvimento foi acompanhado por momentos de revisão e *feedback*, sobretudo através das reuniões com o orientador da empresa e da observação dos fluxos implementados. Estas revisões permitiram ajustar prioridades, clarificar comportamentos esperados e melhorar a experiência de utilização.

O *feedback* recebido incidiu principalmente sobre usabilidade, clareza da informação apresentada e alinhamento entre os fluxos da aplicação. Alguns dos ajustes mais significativos são descritos de seguida.

Um dos ajustes mais relevantes relacionou-se com a forma como o inventário apresentava o estado dos produtos. Numa versão inicial, a interface limitava-se a mostrar a quantidade em stock, sem distinguir reservas, compromissos externos ou produção futura. O *feedback* evidenciou que esta apresentação não era suficiente para apoiar decisões — o produtor precisava de perceber, de relance, se um produto estava em risco ou se tinha margem para vender. Este retorno motivou a implementação do cálculo temporal de compromissos, que se tornou uma das peças centrais da plataforma.

Outro ajuste incidiu sobre o marketplace. Inicialmente, o *feed* apresentava apenas anúncios de venda. A sugestão de tornar visíveis também as procuras agregadas (resultantes de necessidades por satisfazer) enriqueceu o marketplace, que passou a funcionar como um ponto de encontro entre oferta e procura, e não apenas como uma montra de produtos.

A página de alertas também sofreu ajustes. A primeira versão apresentava todos os alertas numa lista única, sem distinção de urgência. O *feedback* levou à organização em separadores (ativos, adiados, resolvidos) e em secções temáticas (ações urgentes, risco, oportunidades, informação), tornando a página mais orientada à ação.

Ao nível da comunicação, a decisão de mostrar o indicador de ligação *WebSocket* como estado técnico da ligação do próprio utilizador — e não como indicador de presença do outro participante — resultou de uma reflexão sobre expectativas incorretas que um indicador de "online/offline" poderia criar. Esta opção foi mais honesta face ao que o sistema realmente consegue garantir.

Estes ajustes contribuíram para transformar a plataforma numa ferramenta mais orientada à ação, reduzindo a necessidade de interpretação manual por parte do utilizador.

Uma síntese adicional de ajustes realizados encontra-se no Anexo C, Tabela C.4.

7.5 Análise crítica face aos Objetivos Propostos

O objetivo principal do estágio consistia no desenvolvimento de uma plataforma *web* para apoio a uma cooperativa agrícola. Considerando o resultado final, esse objetivo foi atingido, uma vez que a aplicação foi implementada, integrada e colocada em produção.

Ao nível funcional, a solução cobre os principais fluxos definidos para o estágio, sintetizados no Anexo D, Tabela D.1. O cumprimento mais relevante não está apenas na existência isolada de cada módulo, mas na forma como estes se articulam. A informação registada por um produtor — catálogo, inventário, produção futura, pedidos externos e necessidades — é convertida em alertas, recomendações e ações concretas, em vez de permanecer dispersa por páginas independentes.

O objetivo de apoiar a cooperação entre produtores foi igualmente concretizado. Através do *marketplace*, das necessidades e das recomendações, um produtor com falta de produto consegue encontrar disponibilidade noutros membros da rede, enquanto um produtor com excedente consegue identificar oportunidades de venda. Esta articulação responde diretamente à lacuna identificada no Capítulo 2, relacionada com a dificuldade em ligar gestão operacional, deteção de necessidades e ação comercial dentro de um contexto cooperativo.

Face às plataformas analisadas no Estado da Arte, a solução desenvolvida ocupa um posicionamento distinto. As soluções de gestão agrícola, como o AGRIVI, destacam-se pela rastreabilidade e monitorização da exploração, mas não têm como foco principal a transação interna entre produtores. As plataformas de comercialização, como a Farmbrite ou a GrownBy, suportam venda e exposição de produtos, mas estão orientadas sobretudo para venda externa ou para o consumidor final. Já soluções empresariais como a Levridge apresentam maior robustez, mas com um nível de complexidade mais adequado a organizações agroindustriais de maior dimensão.

A VISION4FARMS não pretende substituir nenhuma dessas categorias. O seu contributo está na articulação entre inventário atualizado, deteção de necessidades e excedentes, recomendações contextualizadas e transação interna entre produtores. Nenhuma das soluções analisadas foi identificada como cobrindo, de forma integrada e simples, este conjunto de fluxos no contexto específico de uma cooperativa agrícola regional.

Os objetivos de natureza acadêmica foram também cumpridos. O projeto exigiu aplicar, num contexto real, conhecimentos de análise de requisitos, modelação de dados, desenvolvimento *web*, autenticação, comunicação em tempo real, integração de serviços externos e colocação em produção. Para além da componente técnica, o estágio exigiu também capacidade de priorização, adaptação a alterações e validação progressiva das decisões tomadas.

Algumas opções implicaram concessões. A aplicação não expõe uma API REST própria nesta versão, porque os fluxos definidos podiam ser resolvidos com uma aplicação *server-rendered* baseada em Django Templates e HTMX. Esta decisão reduziu a complexidade, permitiu concentrar permissões, validações e auditoria nos fluxos existentes e evitou o desenvolvimento de uma camada adicional que não era essencial para o Produto Mínimo Viável.

Também não foi implementado um modelo preditivo com aprendizagem automática. O sistema de recomendações foi desenvolvido com base em regras operacionais explícitas e dados reais da aplicação. Esta opção revelou-se adequada para uma primeira versão, por tornar o funcionamento mais transparente, validável e ajustado ao âmbito do estágio.

De forma geral, o balanço face aos objetivos é positivo. A solução responde ao problema proposto, encontra-se funcional em produção e apresenta margem clara para evolução futura.

8 Conclusão

O presente capítulo encerra o relatório com uma reflexão sobre o trabalho realizado, os resultados alcançados, os desafios enfrentados, as limitações identificadas e as possibilidades de evolução futura da plataforma.

8.1 Principais Resultados e Contribuições

O estágio resultou no desenvolvimento de uma plataforma *web* funcional para apoio a uma cooperativa agrícola. A solução permite gerir conta, perfil, produtos, *stocks*, necessidades, anúncios, recomendações, encomendas, alertas, mensagens, suporte e administração, reunindo num único sistema vários fluxos que, de outra forma, tenderiam a estar dispersos.

O principal contributo funcional da plataforma é a ligação entre inventário, procura e transação. Um produtor pode registar *stock* e produção futura, declarar necessidades, consultar ofertas, receber recomendações, criar encomendas e comunicar com outros produtores. Esta sequência permite transformar dados operacionais em ações concretas dentro da rede cooperativa, respondendo à lacuna identificada no Estado da Arte: a dificuldade em ligar gestão operacional, deteção de necessidades e ação comercial imediata num único fluxo.

O contributo técnico mais relevante foi o mecanismo de cálculo temporal de compromissos do inventário. Ao considerar *stock* atual, reservas, compromissos externos, produção futura útil e prazos de entrega, esta lógica permite distinguir situações de risco real daquelas em que a cobertura já está assegurada. Como resultado, os alertas, as recomendações e os painéis apresentam informação mais coerente e reduzem a ocorrência de falsos alarmes.

A reutilização desta lógica por vários módulos — inventário, *dashboard*, alertas e recomendações — contribuiu para maior consistência interna da aplicação. Em vez de cada módulo calcular o estado do inventário de forma independente, a plataforma passou a depender de uma fonte comum de decisão, reduzindo incoerências e facilitando a manutenção.

Outro contributo relevante foi a capacidade de combinar ofertas de vários produtores para cobrir uma necessidade que nenhum produtor conseguiria satisfazer individualmente. Esta funcionalidade aproxima a plataforma da lógica cooperativa, em que as disponibilidades podem estar distribuídas por vários membros da rede.

A arquitetura adotada mostrou-se adequada ao contexto do projeto. A opção por uma aplicação monolítica modular em Django, apoiada por serviços como PostgreSQL, Redis, Daphne, Cloudinary, Resend, WhiteNoise e Railway, permitiu construir uma solução integrada, funcional e colocada em produção dentro do período disponível.

Em síntese, o trabalho desenvolvido demonstrou a viabilidade técnica e funcional de uma plataforma orientada para a cooperação entre produtores, assente em dados operacionais, recomendações e fluxos de transação interna.

8.2 Desafios Enfrentados e Limitações da Solução

Durante o desenvolvimento, um dos principais desafios foi a dimensão funcional do projeto. A plataforma não se limitava a uma área isolada, mas envolvia inventário, necessidades, *marketplace*, recomendações, encomendas, alertas, mensagens, suporte e administração. Esta interdependência exigiu atenção constante à coerência entre módulos, uma vez que alterações numa área podiam afetar várias outras. Por exemplo, uma alteração no cálculo do inventário tinha impacto direto nos alertas, nas recomendações e no *dashboard*.

Outro desafio relevante foi a modelação da base de dados. A abordagem *schema-first* permitiu maior controlo sobre a estrutura relacional, mas obrigou a maior cuidado na criação e alteração das tabelas. Como os modelos de negócio utilizam `managed = False`, cada alteração na base de dados teve de ser refletida manualmente nos modelos Django. Em retrospectiva, considera-se que esta decisão continuaria a ser adequada ao contexto do projeto, uma vez que o controlo sobre o *schema* compensou o custo adicional de manutenção manual. Ainda assim, em projetos com equipas maiores ou ciclos de alteração mais rápidos, esta abordagem poderia tornar-se mais exigente.

A lógica temporal do inventário representou a dificuldade técnica mais significativa. Não bastava saber quanto *stock* existia num dado momento; era necessário cruzar *stock* atual, reservas, compromissos externos e produção futura útil com os prazos de entrega previstos. A complexidade

residiu não apenas no cálculo em si, mas também na necessidade de garantir que todos os módulos consumiam este resultado de forma consistente. A solução passou pela centralização dessa lógica numa função reutilizável, depois de se verificar que abordagens mais dispersas poderiam gerar inconsistências entre módulos.

A comunicação em tempo real introduziu outro nível de complexidade. A integração do Django Channels com Redis e Daphne exigiu compreender o ciclo de vida das ligações *WebSocket*, a gestão de grupos de comunicação, a distribuição de eventos e o comportamento da aplicação quando a ligação se perde ou quando o Redis não está disponível. A decisão de tornar o tempo real uma funcionalidade com degradação controlada foi importante: quando o canal *WebSocket* falha, a aplicação continua a funcionar através dos mecanismos tradicionais, preservando os dados na base de dados.

A colocação em produção foi também uma etapa exigente. Foi necessário configurar base de dados, Redis, Cloudinary, Resend, WhiteNoise, variáveis de ambiente e execução com Daphne. A passagem do ambiente local, onde muitas dependências se encontram controladas, para um ambiente real, onde cada serviço precisa de ser configurado, autenticado e validado, revelou-se uma das aprendizagens mais relevantes do estágio. Esta fase evidenciou a diferença entre ter código funcional em desenvolvimento e ter uma aplicação preparada para operação.

A versão entregue apresenta, contudo, limitações que importa reconhecer. A validação foi realizada internamente, sem um grupo alargado de produtores reais, o que não permite ainda avaliar comportamentos de utilização menos previsíveis. O sistema de recomendações baseia-se em regras operacionais explícitas, não explorando padrões históricos, sazonalidade ou aprendizagem a partir de dados acumulados. A ausência de uma API REST própria limita cenários de integração futura com ERPs, aplicações móveis ou sistemas externos, embora essa decisão tenha sido adequada ao âmbito desta entrega.

Existem ainda limitações técnicas relacionadas com testes e monitorização. A plataforma não inclui uma cobertura significativa de testes automatizados de integração nem mecanismos avançados de observabilidade em produção. Estas ausências não impediram a validação funcional da solução, mas devem ser consideradas em fases futuras, sobretudo se a aplicação evoluir para utilização com mais utilizadores ou maior volume de dados.

Por fim, a plataforma não cobre funcionalidades avançadas de gestão agronómica, como rastreabilidade de campo, sensores, maquinaria agrícola ou monitorização técnica de culturas. Esta

limitação é coerente com o problema proposto, centrado na cooperação operacional entre produtores, mas impede uma comparação direta com soluções agrícolas empresariais de maior escala.

Estes aspetos não retiram valor ao trabalho desenvolvido. Pelo contrário, ajudam a enquadrar a plataforma como uma primeira versão funcional, que atingiu os objetivos do estágio e mantém margem clara para evolução.

8.3 Trabalho Futuro e Reflexão Final

A evolução futura da plataforma poderá seguir várias direções. A primeira passa pela validação com produtores reais em contexto de utilização prolongada. Esse contacto permitiria avaliar a clareza dos fluxos, a pertinência das recomendações, a utilidade dos alertas e a facilidade de utilização da aplicação no dia a dia agrícola. Sem esta validação, a plataforma encontra-se funcionalmente construída, mas ainda não plenamente comprovada em contexto real de operação.

Outra evolução importante seria a criação de uma API REST. Esta permitiria integrar a plataforma com sistemas externos, aplicações móveis, ERPs agrícolas ou outras ferramentas utilizadas por cooperativas e produtores. A modelação atual já prevê campos como `source_system` e `external_id`, que poderão facilitar essa evolução.

O sistema de recomendações poderá também ser enriquecido. Com maior volume de dados, seria possível estudar modelos preditivos ou mecanismos de análise histórica para antecipar necessidades, identificar padrões sazonais e tornar as sugestões mais personalizadas. Uma evolução natural seria a adoção de um sistema híbrido, que combinasse regras explícitas com aprendizagem baseada em dados, mantendo sempre a transparência necessária para que os utilizadores compreendam a origem das recomendações.

A criação de uma aplicação móvel ou de uma *Progressive Web App* seria igualmente relevante. Muitos produtores poderão precisar de consultar alertas, responder a encomendas ou atualizar *stock* diretamente no terreno, pelo que uma experiência mais adaptada a dispositivos móveis aumentaria a utilidade prática da solução.

Ao nível técnico, seria importante reforçar os testes automatizados, incluindo testes de integração entre módulos, testes de concorrência e testes de carga. Estes testes seriam especialmente relevantes em cenários com vários produtores a comprar, vender, enviar mensagens ou atualizar

stock em simultâneo. A monitorização em produção também poderá ser melhorada, através de mecanismos de observabilidade, registo de erros, métricas de desempenho e alertas técnicos.

Em retrospectiva, identificam-se dois aspetos que poderiam ter sido abordados de forma diferente. O primeiro seria a introdução mais cedo de dados de demonstração realistas, o que teria facilitado a validação contínua e a comunicação de resultados ao orientador. O segundo seria a implementação de testes automatizados desde fases mais iniciais do desenvolvimento, em vez de os deixar como trabalho futuro. Ainda que a dimensão funcional do projeto e o tempo disponível tenham condicionado estas decisões, ambas poderiam ter contribuído para uma validação mais sistemática da solução.

Uma das aprendizagens mais relevantes do estágio foi compreender que construir *software* em contexto real envolve muito mais do que escrever código. Foi necessário compreender um domínio que não era exclusivamente tecnológico, tomar decisões com informação incompleta, aceitar que nem todas as funcionalidades podiam ser implementadas no tempo disponível e justificar tecnicamente as prioridades assumidas.

A capacidade de distinguir o essencial do acessório foi uma das competências mais importantes desenvolvidas ao longo do estágio. Funcionalidades como o *chatbot* com inteligência artificial ou a API REST poderiam acrescentar valor em fases futuras, mas não eram prioritárias para a primeira versão funcional. Em contrapartida, o cálculo temporal do inventário, as recomendações, as encomendas e os alertas eram essenciais para responder ao problema central da plataforma.

Em síntese, o estágio permitiu desenvolver uma base funcional sólida, capaz de responder ao problema definido e preparada para evolução. O trabalho realizado demonstrou que é possível apoiar a cooperação entre produtores através de uma plataforma simples, integrada e orientada para decisões operacionais.

9 Referências Bibliográficas

AgGateway. (n.d.). *ADAPT Standard*. Retrieved May 22, 2026, from <https://adaptstandard.org/>

AGRIVI. (n.d.). *AGRIVI 360 Farm Enterprise*. Retrieved May 22, 2026, from <https://www.agrivi.com/products/360-farm-enterprise/>

Agri Marketplace. (n.d.). *Agri Marketplace*. Retrieved May 22, 2026, from <https://agrimp.com/>

Anymail. (n.d.). *Anymail: Django email integration for transactional ESPs*. Retrieved May 22, 2026, from <https://anymail.dev/>

Bootstrap Team. (n.d.). *Get started with Bootstrap*. Retrieved May 22, 2026, from <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

Burgess, P. R., & Sunmola, F. T. (2022). Exploring attractive quality requirements for short food supply chain digital platforms. *International Journal of Information Systems and Supply Chain Management*, 15(1), 1–24. <https://doi.org/10.4018/IJISSCM.304372>

Bushel. (n.d.). *Bushel: Software for agriculture*. Retrieved May 22, 2026, from <https://www.bushelpowered.com/>

Chart.js. (n.d.). *Chart.js documentation*. Retrieved May 22, 2026, from <https://www.chartjs.org/docs/>

Cloudinary. (n.d.). *Cloudinary documentation*. Retrieved May 22, 2026, from <https://cloudinary.com/documentation>

Dawson, G. E., Jr., Antunes, J. A. V., Jr., Wegner, D., & Adami, V. S. (2024). Creating a digital platform for the agricultural cooperative system through interorganizational collaboration. *Journal of Rural Studies*, 110, 103388. <https://doi.org/10.1016/j.jrurstud.2024.103388>

Django Channels. (n.d.). *Django Channels documentation*. Retrieved May 22, 2026, from <https://channels.readthedocs.io/>

Django Software Foundation. (n.d.). *Django documentation*. Retrieved May 22, 2026, from <https://docs.djangoproject.com/en/6.0/>

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Farmbrite. (n.d.). *Farm & livestock management software: Plans & pricing*. Retrieved May 22, 2026, from <https://www.farmbrite.com/pricing>

Fountas, S., Carli, G., Sørensen, C. G., Tsiropoulos, Z., Cavalaris, C., Vatsanidou, A., Liakos, B., Canavari, M., Wiebensohn, J., & Tisserye, B. (2015). Farm management information systems: Current situation and future perspectives. *Computers and Electronics in Agriculture*, 115, 40–50. <https://doi.org/10.1016/j.compag.2015.05.011>

Fowler, M., & Lewis, J. (2014). *Microservices*. Martin Fowler. <https://martinfowler.com/articles/microservices.html>

GrownBy. (n.d.). *Power your farm with GrownBy, the farm software co-op*. Retrieved May 22, 2026, from <https://coop.grownby.com/>

HTMX. (n.d.). *htmx documentation*. Retrieved May 22, 2026, from <https://htmx.org/docs/>

Jouanjean, M., Casalini, F., Wiseman, L., & Gray, E. (2020). *Issues around data governance in the digital transformation of agriculture: The farmers' perspective* (OECD Food, Agriculture and Fisheries Papers No. 146). OECD Publishing. <https://doi.org/10.1787/53ecf2ab-en>

Levridge. (n.d.). *Enterprise accounting solutions for agriculture*. Retrieved May 22, 2026, from <https://www.levridge.com/>

Mateos, P., & Bellogín, A. (2025). A systematic literature review of recent advances on context-aware recommender systems. *Artificial Intelligence Review*, 58, Article 20. <https://doi.org/10.1007/s10462-024-10939-4>

Ma, W., Rahut, D. B., Sonobe, T., & Gong, B. (2024). Linking farmers to markets: Barriers, solutions, and policy options. *Economic Analysis and Policy*, 82, 1102–1112. <https://doi.org/10.1016/j.eap.2024.05.005>

Microsoft AppSource. (n.d.). *Levridge for Agriculture*. Retrieved May 22, 2026, from <https://appsource.microsoft.com/en-us/product/web-apps/levridgeinc1689274437259.agriculture>

PostgreSQL Global Development Group. (n.d.). *PostgreSQL documentation*. Retrieved May 22, 2026, from <https://www.postgresql.org/docs/>

Python Decouple. (n.d.). *Python Decouple*. Retrieved May 22, 2026, from <https://pypi.org/project/python-decouple/>

Racius. (n.d.). *Visionary Devotion — Unipessoal Lda*. Retrieved May 29, 2026, from <https://www.racius.com/visionary-devotion-unipessoal-lda/>

Railway. (n.d.). *Deploy a Django app*. Retrieved May 22, 2026, from <https://docs.railway.com/guides/django>

Resend. (n.d.). *Send emails with Django*. Retrieved May 22, 2026, from <https://resend.com/docs/send-with-django>

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

- Trendov, N. M., Varas, S., & Zeng, M. (2019). *Digital technologies in agriculture and rural areas: Status report*. Food and Agriculture Organization of the United Nations.
<https://openknowledge.fao.org/handle/20.500.14283/ca4887en>
- Trimble. (n.d.). *Agriculture industry technology and farm management solutions*. Retrieved May 22, 2026, from <https://www.trimble.com/en/industries/agriculture>
- van Klompenburg, T., Kassahun, A., & Catal, C. (2020). Crop yield prediction using machine learning: A systematic literature review. *Computers and Electronics in Agriculture*, 177, 105709.
<https://doi.org/10.1016/j.compag.2020.105709>
- WhiteNoise. (n.d.). *WhiteNoise documentation*. Retrieved May 22, 2026, from <https://whitenoise.readthedocs.io/>
- Zeng, Y., Jia, F., Wan, L., & Guo, H. (2017). E-commerce in agri-food sector: A systematic literature review. *International Food and Agribusiness Management Review*, 20(4), 439–459.
<https://doi.org/10.22434/IFAMR2016.0156>

Anexo A — Requisitos Funcionais e Não Funcionais

Os requisitos funcionais descrevem comportamentos, ações e funcionalidades que a plataforma deve disponibilizar aos utilizadores, como registar contas, gerir *stocks*, publicar anúncios, gerar recomendações, criar encomendas ou enviar mensagens.

Os requisitos não funcionais descrevem qualidades e restrições transversais da solução, como segurança, usabilidade, privacidade, fiabilidade, rastreabilidade, manutenibilidade, integração e operação em ambiente de produção.

A.1 Requisitos funcionais

A.1.1 Autenticação, registo e gestão de conta

RF01 — O sistema deve permitir o registo público de novos utilizadores através de formulário.

RF02 — No registo, o sistema deve recolher, pelo menos, o primeiro nome, o último nome, o email, a palavra-passe e a confirmação da palavra-passe.

RF03 — O sistema deve permitir indicar, de forma opcional, a empresa ou exploração agrícola durante o processo inicial de registo.

RF04 — O sistema deve gerar e armazenar *tokens* de verificação de conta.

RF05 — O sistema deve enviar um email de confirmação após o registo do utilizador.

RF06 — A conta do utilizador só deve ficar ativa após a confirmação do email.

RF07 — O sistema deve permitir o *login* e o *logout* dos utilizadores.

RF08 — O sistema deve distinguir utilizadores do tipo produtor/cliente e administrador.

RF09 — O registo público deve criar apenas contas de produtor/cliente, sendo as contas administrativas criadas internamente.

RF10 — O sistema deve manter informação sobre o estado da conta, a origem do registo, o último acesso e as permissões associadas ao utilizador.

A.1.2 Perfil do produtor e produtos produzidos

RF11 — Cada utilizador produtor deve estar associado a um perfil funcional de produtor.

RF12 — O perfil do produtor deve permitir armazenar nome visível, empresa, telefone, NIF, morada, código postal, cidade e distrito.

RF13 — O sistema deve permitir registar coordenadas geográficas do produtor, quando disponíveis.

RF14 — O sistema deve permitir associar vários produtos ao mesmo produtor.

RF15 — A associação entre produtor e produto deve poder ser ativada ou desativada.

RF16 — A informação do perfil deve poder ser atualizada pelo próprio utilizador ou por um administrador autorizado.

A.1.3 Catálogo de produtos

RF17 — O sistema deve manter um catálogo central de categorias e produtos.

RF18 — Cada produto deve possuir categoria, nome, identificador técnico, unidade e descrição opcional.

RF19 — Os produtos e categorias devem poder ser marcados como ativos ou inativos.

RF20 — O administrador deve poder criar, editar e desativar produtos e categorias.

RF21 — O catálogo deve permitir manter uma designação consistente dos produtos utilizados nos módulos de inventário, necessidades, *marketplace*, recomendações e encomendas.

A.1.4 Gestão de stocks, necessidades e previsões

RF22 — O sistema deve permitir registar e atualizar o *stock* por produtor e produto.

RF23 — O sistema deve manter, para cada produto do produtor, a quantidade atual e a quantidade reservada.

RF24 — O sistema deve calcular o *stock* disponível com base na diferença entre a quantidade atual e a quantidade reservada.

RF25 — O sistema deve considerar os compromissos externos do produtor no cálculo do estado operacional do inventário.

RF26 — O sistema deve manter histórico dos movimentos de *stock*, incluindo tipo de movimento, quantidade, referência, observações e utilizador responsável.

RF27 — O sistema deve permitir registar previsões de produção por produtor, produto e período temporal.

RF28 — As previsões de produção devem permanecer separadas do *stock* real até serem assumidas como entrada efetiva.

RF29 — O sistema deve permitir registar necessidades de produto com quantidade, prazo, estado e observações.

RF30 — As necessidades devem poder ser criadas manualmente pelo produtor ou originadas por processos internos da plataforma.

RF31 — O sistema deve suportar os estados principais de uma necessidade: aberta, parcialmente coberta, coberta, ignorada ou cancelada.

RF32 — O sistema deve permitir recalcular a cobertura de uma necessidade com base em propostas, recomendações e encomendas associadas.

RF33 — O sistema deve permitir que necessidades e previsões sirvam de origem a alertas e recomendações.

RF34 — Ao aceitar uma recomendação associada a uma necessidade, o sistema deve atualizar automaticamente o estado de cobertura dessa necessidade.

A.1.5 Pedidos externos de clientes

RF35 — O produtor deve poder registar pedidos externos de clientes, indicando cliente, contacto, referência, produto, quantidade e data pretendida de entrega.

- RF36** — O sistema deve permitir listar, editar e cancelar pedidos externos de clientes.
- RF37** — O sistema deve calcular temporalmente a cobertura dos pedidos externos por produto e data.
- RF38** — O cálculo de cobertura deve considerar o *stock* atual disponível e a produção futura útil até à data pretendida de entrega.
- RF39** — Quando existir défice temporal, o sistema deve gerar ou atualizar automaticamente uma necessidade agregada.
- RF40** — A necessidade automática deve apresentar como quantidade o maior défice identificado e como prazo a primeira data crítica.
- RF41** — Quando o défice temporal deixar de existir, o sistema deve marcar a necessidade automática como coberta.
- RF42** — O sistema deve sincronizar os compromissos externos do produtor com os pedidos externos de clientes em aberto.

A.1.6 Marketplace interno

- RF43** — O sistema deve permitir publicar anúncios de venda interna entre produtores.
- RF44** — Um anúncio pode ter origem em *stock* atual ou numa previsão de produção futura.
- RF45** — Cada anúncio deve ter apenas uma origem principal, evitando que o mesmo anúncio esteja simultaneamente associado a *stock* atual e pré-venda.
- RF46** — Cada anúncio deve guardar produto, produtor, quantidade total, quantidade disponível, quantidade reservada, preço unitário, modo de entrega, observações e fotografia opcional.
- RF47** — O sistema deve suportar anúncios de *stock* atual, anúncios de pré-venda e ofertas privadas dirigidas a necessidades.
- RF48** — O sistema deve suportar diferentes modos de entrega, incluindo levantamento no local, entrega pelo produtor ou ambos.
- RF49** — O produtor deve poder definir raio de entrega e taxa de entrega, quando aplicável.
- RF50** — O sistema deve permitir editar, fechar, cancelar ou expirar anúncios.
- RF51** — O *marketplace* deve listar apenas anúncios públicos ativos e disponíveis.

RF52 — O *marketplace* deve permitir pesquisa, filtragem e ordenação de anúncios.

RF53 — O sistema deve permitir consultar o detalhe de um anúncio.

RF54 — O detalhe do anúncio deve permitir calcular o total estimado com base na quantidade pretendida.

RF55 — O detalhe do anúncio deve disponibilizar ações como comprar, contactar o produtor ou consultar informação adicional da oferta.

RF56 — O sistema deve permitir reservar quantidades de um anúncio para suportar recomendações e encomendas.

RF57 — O sistema deve impedir a eliminação física de anúncios com histórico transacional ou associados a recomendações, permitindo apenas o seu encerramento, cancelamento ou expiração.

A.1.7 Ofertas dirigidas a necessidades e privacidade no marketplace

RF58 — O sistema deve permitir criar ofertas privadas dirigidas a necessidades específicas.

RF59 — Uma oferta dirigida a uma necessidade deve ser visível apenas para o produtor que criou a oferta e para o produtor dono da necessidade.

RF60 — Uma oferta dirigida a uma necessidade só deve poder ser comprada pelo produtor dono dessa necessidade.

RF61 — A criação de uma oferta dirigida a uma necessidade não deve gerar um anúncio público normal no *marketplace*.

RF62 — O produtor deve poder escolher se pretende mostrar a sua localização no mapa do detalhe do anúncio.

RF63 — Quando a localização exata não estiver disponível ou estiver ocultada, o sistema deve poder apresentar localização aproximada ou omitir o mapa.

RF64 — O detalhe do anúncio pode apresentar mapa com localização da exploração, raio de entrega e localização aproximada do comprador, respeitando as regras de privacidade configuradas.

A.1.8 Recomendações e apoio à decisão

RF65 — O sistema deve disponibilizar um fluxo de recomendações orientado à compra e venda de produtos.

RF66 — O sistema deve identificar produtos que o produtor necessita de comprar com base em défices temporais.

RF67 — O sistema deve identificar produtos que o produtor pode vender com base na margem temporal vendável.

RF68 — No fluxo de compra, o utilizador deve indicar ou confirmar produto, quantidade necessária e prazo opcional.

RF69 — O sistema deve poder pré-preencher a necessidade com base em alertas, necessidades existentes ou informação operacional do inventário.

RF70 — O motor de recomendação deve gerar sugestões com base nos anúncios disponíveis no *marketplace*.

RF71 — Uma recomendação pode dividir a compra por vários produtores.

RF72 — Cada item recomendado deve apresentar vendedor, produto, quantidade sugerida, preço unitário e subtotal.

RF73 — O sistema deve guardar razões ou justificações associadas à recomendação.

RF74 — O utilizador deve poder ajustar quantidades e seleccionar ou desseleccionar itens sugeridos.

RF75 — O sistema deve manter o estado da recomendação, como gerada, aceite, ajustada, ignorada ou expirada.

RF76 — Ao aceitar uma recomendação de compra, o sistema deve permitir gerar uma ou mais encomendas.

RF77 — Ao aceitar uma recomendação de venda, o sistema deve encaminhar o produtor para publicação de anúncio ou resposta a uma necessidade.

RF78 — O sistema deve guardar custo total estimado, resumo da sugestão e data de aceitação.

RF79 — As recomendações devem poder ser originadas manualmente, por alerta ou por integração futura.

RF80 — A recomendação deve manter histórico suficiente para auditoria funcional do processo.

A.1.9 Encomendas

RF81 — O sistema deve permitir criar encomendas a partir do *marketplace*, de uma recomendação ou da aceitação de uma oferta privada.

RF82 — Cada encomenda deve ter número legível, comprador, origem, estado, total e informação de entrega.

RF83 — Uma encomenda individual pode conter vários itens do mesmo vendedor, mas não deve misturar vendedores distintos.

RF84 — Quando uma recomendação envolver vários vendedores, o sistema deve criar um grupo de encomendas.

RF85 — O grupo de encomendas deve permitir ao comprador acompanhar a compra global, mantendo cada vendedor associado apenas à sua própria sub-encomenda.

RF86 — Cada item de encomenda deve guardar produto, anúncio de origem, vendedor, quantidade, preço unitário e subtotal.

RF87 — O sistema deve suportar estados como pendente, confirmada, em progresso, em entrega, concluída e cancelada.

RF88 — O sistema deve manter histórico dos estados percorridos por cada encomenda.

RF89 — O utilizador deve poder consultar listas de encomendas e respetivos detalhes.

RF90 — O detalhe da encomenda deve apresentar itens, total, informação logística e *timeline* de estados.

RF91 — O vendedor deve poder aceitar, preparar e indicar o envio da encomenda.

RF92 — O comprador deve poder confirmar a receção da encomenda.

RF93 — O cancelamento de uma encomenda deve libertar as quantidades reservadas.

RF94 — A conclusão de uma encomenda deve refletir-se no *stock* e nos movimentos de inventário.

RF95 — Quando a encomenda estiver associada a uma necessidade, a sua conclusão deve atualizar a cobertura dessa necessidade.

A.1.10 Alertas e notificações

RF96 — O sistema deve gerar alertas automáticos com base em regras operacionais.

RF97 — Os alertas devem poder sinalizar situações como *stock* crítico, excedente vendável, necessidade sem cobertura, resposta recebida, anúncio a expirar, encomenda a exigir confirmação ou entrega em atraso.

RF98 — Cada alerta deve guardar produtor, produto, origem, severidade, título, descrição, estado e contexto associado.

RF99 — Um alerta pode estar associado a uma necessidade, previsão, anúncio, encomenda ou recomendação.

RF100 — O sistema deve permitir resolver alertas.

RF101 — O sistema deve permitir ignorar ou adiar alertas, registrando motivo quando aplicável.

RF102 — O sistema deve permitir distinguir alertas ativos, ignorados/adiados e resolvidos.

RF103 — O sistema deve evitar a duplicação de alertas ativos para o mesmo contexto lógico.

RF104 — O sistema deve manter histórico de eventos associados ao ciclo de vida dos alertas.

RF105 — O sistema deve apresentar notificações informativas associadas a alertas, mensagens, encomendas ou recomendações.

RF106 — As notificações devem poder ser marcadas como lidas.

RF107 — As notificações devem poder conter uma ligação para a ação ou página correspondente.

RF108 — O painel principal deve destacar alertas ativos e notificações recentes.

RF109 — Cada alerta deve disponibilizar ações contextuais diretas, como abrir o detalhe do *stock*, iniciar uma recomendação ou publicar excedente.

A.1.11 Mensagens entre produtores

RF110 — O sistema deve permitir iniciar conversas entre produtores.

RF111 — Uma conversa pode estar associada a um anúncio ou a uma encomenda.

RF112 — O botão “Contactar produtor” deve abrir ou criar uma conversa adequada ao contexto.

RF113 — O sistema deve suportar conversas individuais entre dois utilizadores.

- RF114** — O sistema deve manter o estado de leitura individual por participante.
- RF115** — O sistema deve permitir trocar mensagens de texto.
- RF116** — O sistema deve permitir registrar eventos de sistema dentro das conversas.
- RF117** — O sistema deve permitir enviar ficheiros anexos nas conversas.
- RF118** — O histórico das mensagens deve ficar persistido e disponível para consulta posterior.
- RF119** — O sistema deve permitir arquivar e desarquivar conversas por participante, sem apagar o histórico nem afetar a visibilidade para o outro participante.
- RF120** — Quando uma nova mensagem chega a uma conversa arquivada, a conversa deve regressar à lista de conversas ativas do destinatário.

A.1.12 Suporte ao utilizador

- RF121** — O utilizador autenticado deve poder contactar o suporte a partir de uma página dedicada.
- RF122** — O formulário de suporte deve permitir indicar assunto, mensagem inicial e anexar imagens.
- RF123** — O sistema deve criar um *ticket* de suporte persistente.
- RF124** — Cada *ticket* deve possuir número sequencial, estado inicial e dados do utilizador no momento da criação.
- RF125** — O sistema deve notificar os administradores ativos quando é criado um novo *ticket*.
- RF126** — O primeiro administrador a aceitar o *ticket* deve ficar como responsável pelo seu tratamento.
- RF127** — O administrador deve poder trocar mensagens com o utilizador sem fechar automaticamente o *ticket*.
- RF128** — O fecho do *ticket* deve ser explícito e realizado por ação administrativa.
- RF129** — O utilizador deve poder consultar o histórico dos seus *tickets*.
- RF130** — O utilizador deve poder responder a *tickets* que ainda não estejam fechados.
- RF131** — O sistema deve permitir anexar imagens às mensagens de suporte.
- RF132** — As ações relevantes do fluxo de suporte devem ser registadas em auditoria.

A.1.13 Administração, auditoria e integração

RF133 — O administrador deve ter acesso a um *dashboard* global da plataforma.

RF134 — O *dashboard* administrativo deve apresentar indicadores sobre utilizadores, encomendas, suporte, atividade comercial e alertas relevantes.

RF135 — O administrador deve poder gerir utilizadores.

RF136 — O administrador deve poder confirmar manualmente contas pendentes, quando necessário.

RF137 — O administrador deve poder suspender ou reativar utilizadores.

RF138 — O administrador deve poder gerir o catálogo de produtos e categorias.

RF139 — O administrador deve poder consultar os pedidos de suporte e responder aos utilizadores.

RF140 — O sistema deve manter um registo de auditoria das ações relevantes dos utilizadores e do sistema.

RF141 — O registo de auditoria deve guardar ação, utilizador, entidade afetada, data, contexto técnico e, quando aplicável, valores anteriores e posteriores.

RF142 — A plataforma deve funcionar de forma independente de integrações externas.

RF143 — O sistema deve estar preparado para integração futura com Vision4Farms ou sistemas equivalentes.

RF144 — A integração futura deve poder importar ou sincronizar necessidades, previsões, eventos ou outros dados operacionais.

RF145 — O sistema deve guardar histórico técnico das sincronizações realizadas.

A.2 Requisitos não funcionais

A.2.1 Segurança e controlo de acesso

RNF01 — O sistema deve garantir autenticação obrigatória para acesso às áreas privadas da plataforma.

- RNF02** — O sistema deve separar permissões entre produtores e administradores.
- RNF03** — As contas administrativas não devem poder ser criadas através do registo público.
- RNF04** — A conta de utilizador só deve ser considerada ativa após validação do email.
- RNF05** — As palavras-passe devem ser armazenadas através de mecanismos seguros de *hash*.
- RNF06** — O sistema deve utilizar *tokens* seguros para confirmação de conta, recuperação de palavra-passe e fluxos equivalentes.
- RNF07** — O sistema deve aplicar limitação de tentativas em operações sensíveis, como início de sessão, registo e submissão de pedidos de suporte.
- RNF08** — O sistema deve proteger formulários e sessões contra pedidos indevidos e acessos não autorizados.
- RNF09** — O sistema deve validar permissões antes da execução de ações sensíveis.
- RNF10** — A alteração da palavra-passe deve terminar a sessão atual e invalidar sessões anteriores do utilizador, quando aplicável.
- RNF11** — O sistema não deve guardar palavras-passe, *tokens*, chaves secretas ou outros dados sensíveis em registos de auditoria.

A.2.2 Privacidade

- RNF12** — O sistema deve permitir ao produtor controlar a apresentação da sua localização nos anúncios.
- RNF13** — O sistema deve permitir ocultar a localização exata do produtor, apresentando apenas localização aproximada ou omitindo o mapa quando necessário.
- RNF14** — Ofertas privadas dirigidas a necessidades não devem aparecer no *marketplace* público.
- RNF15** — Apenas os produtores diretamente envolvidos numa oferta privada devem conseguir aceder ao seu conteúdo.
- RNF16** — O arquivo de conversas deve afetar apenas o participante que arquiva, sem remover o histórico para o outro participante.
- RNF17** — Os dados pessoais dos utilizadores devem ser utilizados apenas nos contextos funcionais necessários à operação da plataforma.

A.2.3 Usabilidade e adequação ao utilizador

RNF18 — A interface deve ser simples, clara e adequada a produtores com diferentes níveis de familiaridade tecnológica.

RNF19 — A navegação deve estar organizada por áreas funcionais facilmente identificáveis.

RNF20 — O sistema deve apresentar informação operacional de forma direta, destacando situações que exigem ação.

RNF21 — Os alertas devem incluir ações contextuais que encaminhem o utilizador para a resolução do problema.

RNF22 — O sistema deve apresentar quantidades, estados, encomendas e notificações de forma compreensível para o utilizador final.

RNF23 — A plataforma deve reduzir a necessidade de interpretação manual de dados, apresentando recomendações, alertas e indicadores operacionais claros.

RNF24 — A interface deve permitir utilização em diferentes tamanhos de ecrã, incluindo dispositivos móveis.

A.2.4 Fiabilidade e disponibilidade

RNF25 — A plataforma deve funcionar de forma autónoma, sem depender obrigatoriamente de integração externa.

RNF26 — Uma falha temporária nos mecanismos de comunicação em tempo real não deve impedir o acesso às páginas principais.

RNF27 — As mensagens, encomendas, movimentos de *stock*, alertas e pedidos de suporte devem ser persistidos na base de dados.

RNF28 — O sistema deve evitar duplicação lógica de alertas para o mesmo contexto operacional.

RNF29 — O sistema deve evitar duplicação de necessidades automáticas ativas para o mesmo produtor, produto e contexto de origem.

RNF30 — Operações críticas, como criação de encomendas e reserva de quantidades, devem preservar a integridade dos dados.

RNF31 — O sistema deve manter comportamento previsível mesmo quando serviços complementares, como envio de email ou comunicação em tempo real, falham temporariamente.

A.2.5 Rastreabilidade e auditoria

RNF32 — O sistema deve manter registos de auditoria das ações relevantes realizadas por utilizadores, administradores e processos internos.

RNF33 — Os movimentos de *stock* devem ser registados para permitir compreender a evolução do inventário.

RNF34 — As alterações de estado das encomendas devem ficar registadas em histórico próprio.

RNF35 — As ações realizadas sobre alertas devem ser registadas como eventos associados ao respetivo alerta.

RNF36 — O sistema deve manter histórico das sincronizações ou processos de integração externa.

RNF37 — Anúncios associados a recomendações, encomendas ou histórico transacional devem ser preservados, evitando eliminação física no fluxo normal.

RNF38 — O sistema deve guardar contexto suficiente para permitir análise posterior de operações relevantes.

A.2.6 Manutenibilidade e modularidade

RNF39 — A aplicação deve estar organizada em módulos funcionais independentes.

RNF40 — Cada módulo deve concentrar responsabilidades relacionadas com uma área da plataforma, como contas, catálogo, inventário, *marketplace*, necessidades, recomendações, encomendas, alertas, mensagens, suporte, administração ou integrações.

RNF41 — A separação entre conta de utilizador e perfil funcional do produtor deve ser mantida.

RNF42 — O modelo de dados deve ser relacional, coerente e alinhado com os principais fluxos operacionais da plataforma.

RNF43 — A estrutura da aplicação deve permitir evolução futura sem exigir reescrita completa da solução.

RNF44 — O sistema deve estar preparado para adicionar novas funcionalidades, como API REST, aplicação móvel, integrações externas ou mecanismos de recomendação mais avançados.

RNF45 — A organização interna do código deve facilitar manutenção, validação e evolução progressiva da plataforma.

A.2.7 Integração e interoperabilidade

RNF46 — A plataforma deve estar preparada para integração futura com Vision4Farms ou sistemas equivalentes.

RNF47 — O sistema deve permitir distinguir dados criados manualmente de dados provenientes de sistemas externos.

RNF48 — As entidades relevantes devem poder guardar informação de origem externa, quando aplicável.

RNF49 — Os processos de sincronização externa devem manter registos técnicos sobre estado, erros e número de registos processados.

RNF50 — A ausência de integração externa ativa não deve impedir a utilização normal da plataforma.

A.2.8 Comunicação em tempo real e desempenho funcional

RNF51 — O sistema deve suportar comunicação em tempo real nas funcionalidades em que a atualização imediata acrescenta valor ao utilizador.

RNF52 — As mensagens entre produtores devem poder ser entregues de forma imediata quando o canal em tempo real estiver disponível.

RNF53 — Os indicadores de alertas, mensagens e suporte devem poder ser atualizados dinamicamente.

RNF54 — O sistema deve continuar acessível por renderização normal quando a comunicação em tempo real não estiver disponível.

RNF55 — A plataforma deve evitar operações redundantes, como criação repetida de notificações iguais para o mesmo evento.

RNF56 — As páginas com listagens ou históricos devem suportar mecanismos de paginação, filtragem ou pesquisa quando necessário.

A.2.9 Armazenamento e gestão de ficheiros

RNF57 — O sistema deve permitir armazenar fotografias de anúncios, imagens de perfil, anexos de mensagens e anexos de suporte.

RNF58 — Os ficheiros enviados pelos utilizadores devem ser armazenados em serviço adequado ao ambiente de produção.

RNF59 — O sistema deve validar ficheiros anexados por extensão, tipo de conteúdo e tamanho máximo permitido.

RNF60 — O sistema deve distinguir imagens de outros tipos de ficheiros quando aplicável.

RNF61 — O sistema deve guardar metadados dos ficheiros anexados, como nome, tipo, localização e tamanho.

RNF62 — A falha na difusão em tempo real de um anexo não deve implicar perda do ficheiro quando este já tiver sido guardado corretamente.

A.2.10 Operação em ambiente de produção

RNF63 — A aplicação deve estar preparada para execução em ambiente de produção.

RNF64 — A configuração da aplicação deve recorrer a variáveis de ambiente para dados sensíveis e valores dependentes do ambiente.

RNF65 — Credenciais, chaves secretas e ligações a serviços externos não devem estar expostas no código-fonte.

RNF66 — A aplicação deve separar responsabilidades entre aplicação web, base de dados, comunicação em tempo real, armazenamento de ficheiros e envio de emails.

RNF67 — O sistema deve permitir publicação controlada a partir do repositório de código.

RNF68 — Os ficheiros estáticos da aplicação devem ser servidos de forma adequada em produção.

RNF69 — O sistema deve prever a execução periódica de tarefas operacionais, como sincronização de alertas e expiração de anúncios.

RNF70 — O modo de depuração deve estar desativado em produção.

RNF71 — A aplicação deve manter configurações distintas para desenvolvimento local e ambiente de produção.

A.3 Síntese

A identificação e organização destes requisitos permitiu orientar o desenvolvimento da plataforma de forma estruturada. Os requisitos funcionais serviram de base à implementação dos principais módulos da aplicação, enquanto os requisitos não funcionais ajudaram a garantir preocupações transversais de segurança, privacidade, rastreabilidade, usabilidade, manutenção e operação em produção.

Desta forma, o levantamento de requisitos não funcionou apenas como uma etapa inicial do projeto, mas como um instrumento de acompanhamento e validação da solução desenvolvida ao longo do estágio.

Anexo B — Diagrama Entidade-Relação

O presente anexo apresenta o diagrama relacional da base de dados da plataforma, dividido por domínios funcionais para facilitar a leitura. Devido à dimensão do modelo, composto por 31 tabelas de negócio, o diagrama foi separado em várias figuras, mantendo as relações principais entre os módulos da aplicação.

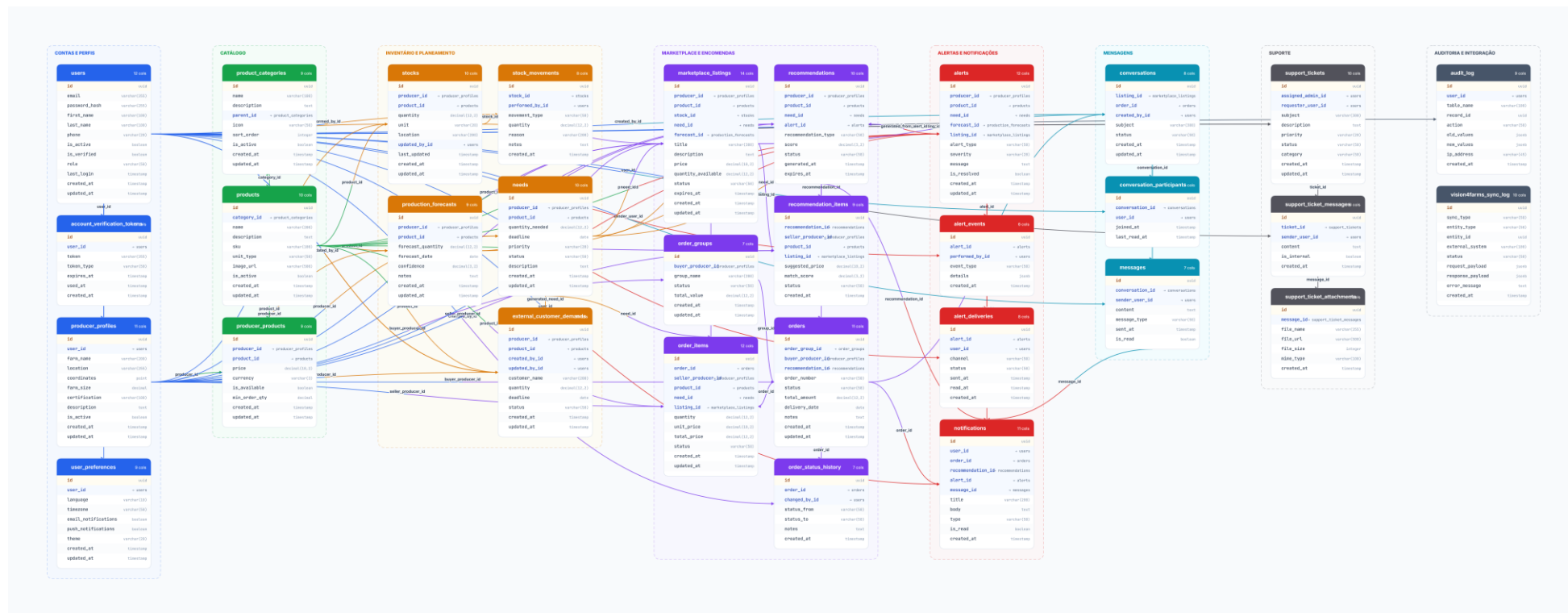


Figura B.1 — Visão geral do modelo relacional

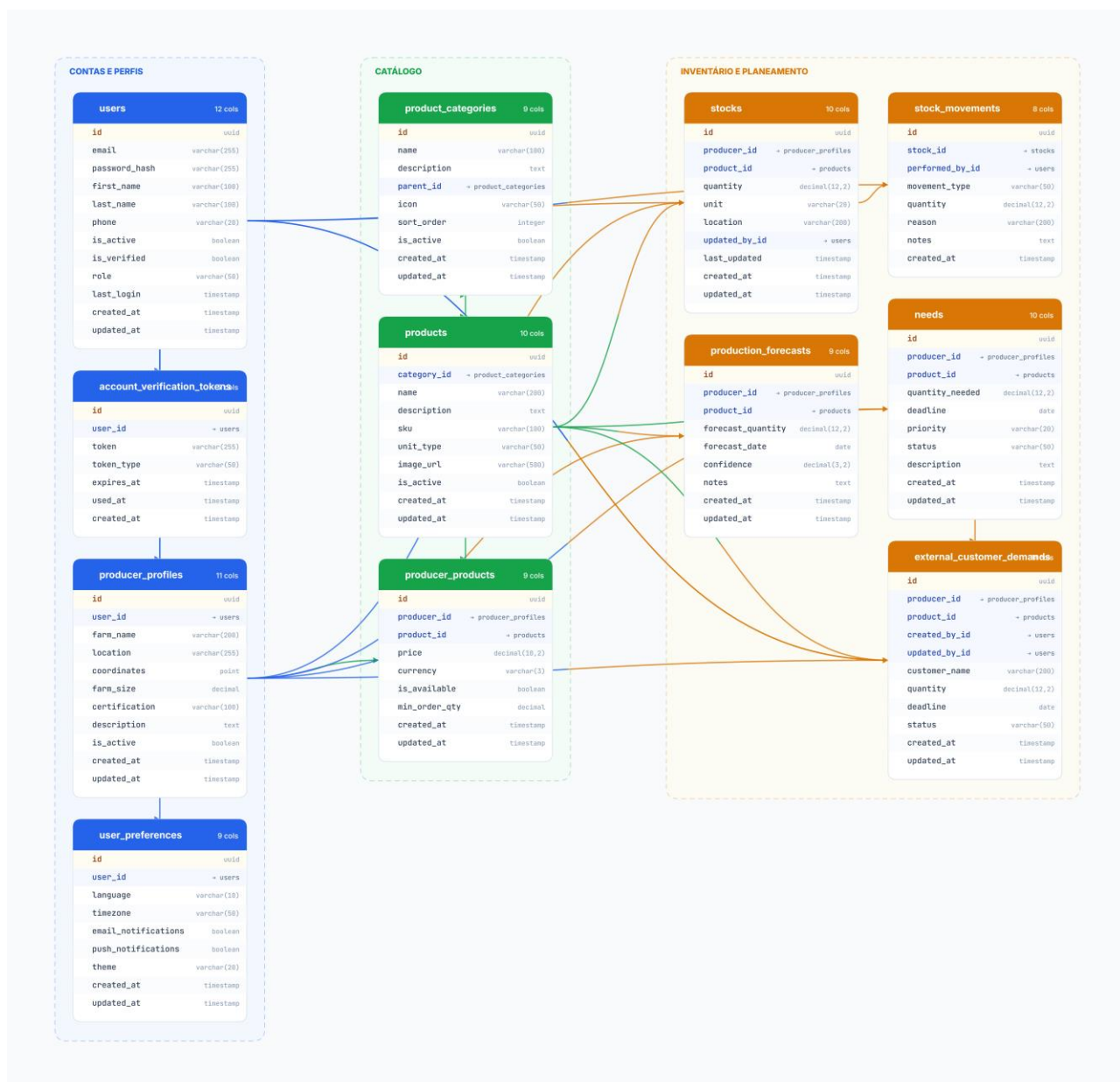


Figura B.2 — Relações entre contas, perfis, catálogo, inventário e necessidades

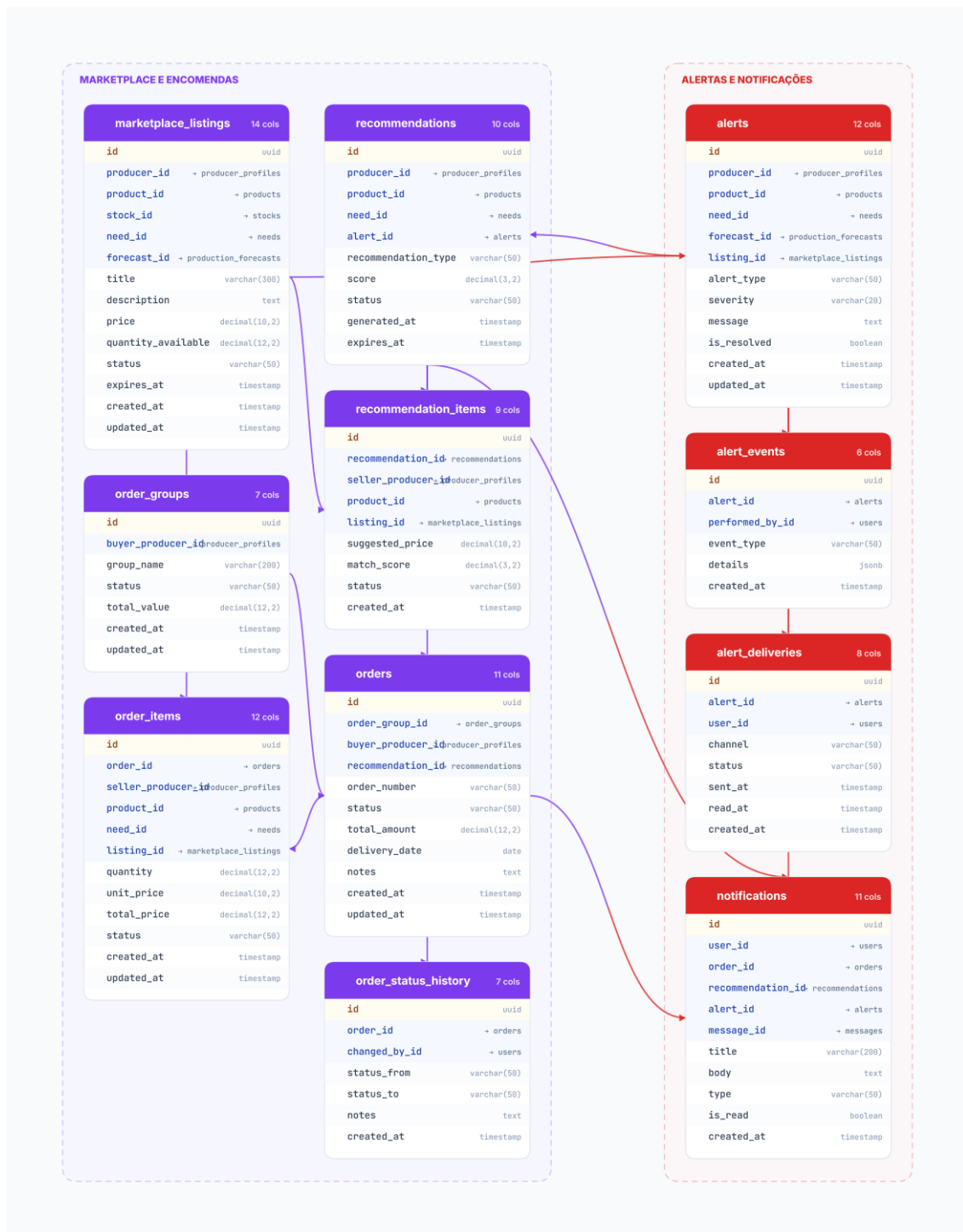


Figura B.3 — Relações entre marketplace, recomendações, encomendas, alertas e notificações

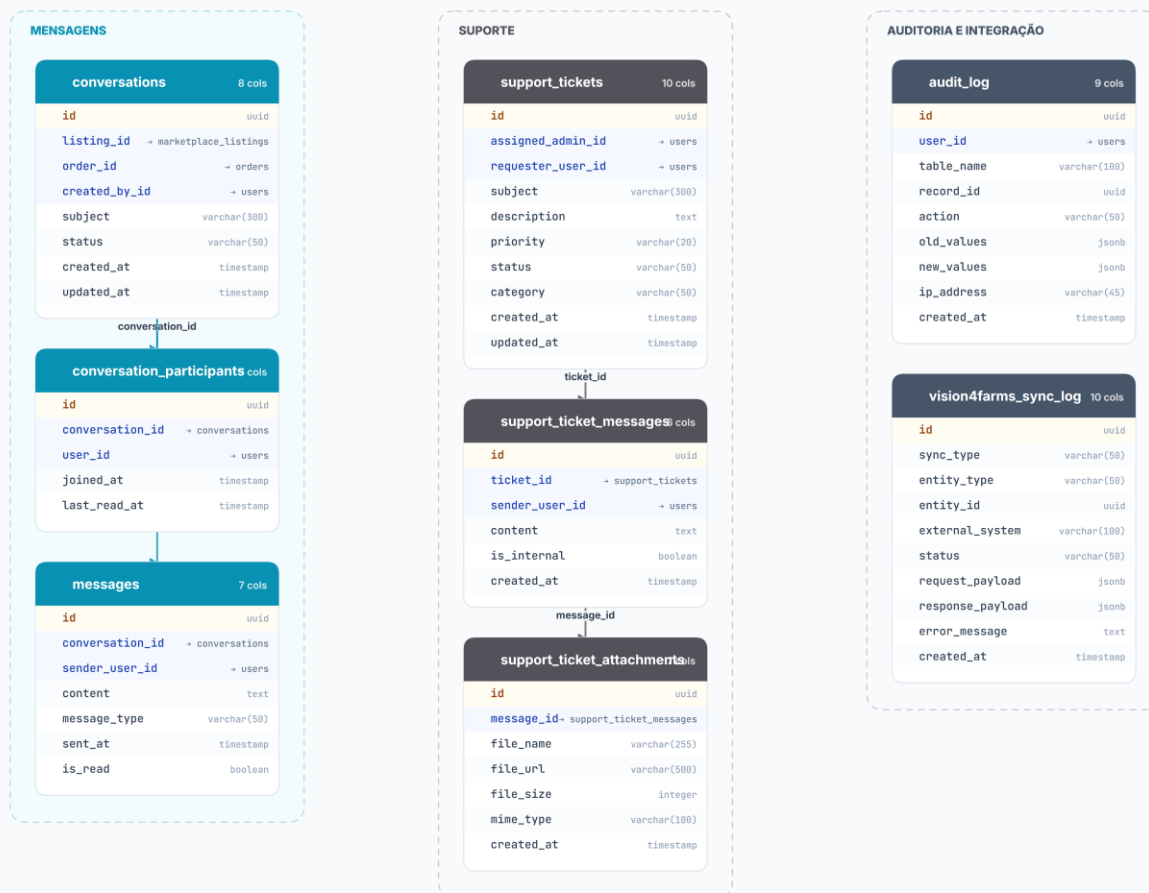


Figura B.4 — Relações dos módulos de comunicação, suporte, auditoria e integração

Anexo C — Tabelas

Tabela C.1 — Comparação entre soluções digitais agrícolas analisadas.

Solução	Foco principal	Pontos fortes	Limitações face ao projeto	Adequação a pequenos produtores
AGRIVI	Gestão agrícola profissional e operações agronómicas	Forte em rastreabilidade, planeamento, monitorização, integração com sensores, maquinaria e dados operacionais	Não apresenta foco claro em marketplace interno cooperativo ou redistribuição entre produtores da mesma rede	Média/baixa; posicionamento mais empresarial e preços não apresentados publicamente
Farmbrite	Gestão agrícola, inventário e venda online	Preços públicos, gestão de inventário, encomendas, alertas de baixo stock, loja online e funcionalidades de gestão da exploração	Orientada sobretudo para venda externa e gestão da própria exploração, não para trocas internas cooperativas	Média/alta; planos públicos a partir de valores mensais relativamente acessíveis
Levridge	Enterprise Resource Planning para agronegócio e cooperativas de maior dimensão	Forte em integração empresarial, contabilidade agrícola, vendas, patronage/equity, contratos, commodities e processos administrativos	Solução enterprise, mais pesada e menos orientada para pequenos produtores ou fluxos simples de marketplace interno	Baixa/média; mais adequada a organizações de maior dimensão
Plataformas B2C, como GrownBy	Venda direta entre produtor e consumidor final	Simplicidade comercial, relação direta com consumidor e facilidade de exposição dos produtos	Não resolvem a gestão interna de stocks, necessidades e redistribuição entre produtores	Média; úteis para venda direta, mas menos adequadas à lógica cooperativa interna
Solução desenvolvida	Cooperativa agrícola, marketplace interno e apoio à decisão	Integra stocks, necessidades, pré-venda, ofertas dirigidas, recomendações, encomendas e comunicação entre produtores	Não pretende substituir sistemas avançados de gestão agronómica, Enterprise Resource Planning completo ou soluções comerciais de grande escala	Alta para o contexto previsto; desenhada para uma utilização simples e cooperativa

Fonte: elaboração própria com base em AGRIVI (n.d.), Farmbrite (2024, 2025), Levridge (n.d.), Burgess & Sunmola (2022)

Tabela C.2 — Síntese das funcionalidades implementadas

Área funcional	Resultado obtido
Conta e segurança	Registo, confirmação por <i>email</i> , início de sessão, recuperação de palavra-passe, permissões, validação de sessão e auditoria.
Catálogo e perfil	Catálogo global de produtos e categorias, associação de produtos ao produtor e gestão de informação operacional do perfil.
Inventário	Gestão de <i>stock</i> , reservas, movimentos, previsões de produção, compromissos externos e cálculo temporal de disponibilidade.
Marketplace e necessidades	Publicação de anúncios de <i>stock</i> atual e pré-venda, procuras agregadas, respostas privadas e gestão de necessidades.
Recomendações	Sugestões de compra ou venda com base em défice temporal, margem vendável, necessidades abertas e anúncios disponíveis.
Encomendas	Criação de encomendas, grupos de encomendas, ciclo de estados, reservas, receção e atualização de cobertura de necessidades.
Alertas e notificações	Alertas operacionais, notificações informativas, deduplicação e indicadores de situações pendentes.
Mensagens e suporte	Conversas entre produtores, anexos, comunicação em tempo real, <i>tickets</i> de suporte e histórico conversacional.
Administração	Painel administrativo, gestão de utilizadores, catálogo, suporte, auditoria e indicadores globais de atividade.

Tabela C.3 — Principais resultados técnicos obtidos

Componente técnica	Resultado obtido
Arquitetura	Aplicação monolítica modular, organizada por domínios funcionais.
Base de dados	Modelo relacional em PostgreSQL, com gestão manual do <i>schema</i> .
Inventário	Cálculo centralizado de compromissos, défices e margem vendável.
Tempo real	Comunicação com <i>WebSocket</i> através de Django Channels, Redis e Daphne.
Ficheiros	Armazenamento de imagens e anexos em Cloudinary.
Email	Envio transaccional com Resend através de API HTTP.
Produção	Aplicação publicada no Railway, com configuração por variáveis de ambiente.
Segurança	Permissões, <i>rate limiting</i> , sessões validadas, auditoria e separação de segredos.

Tabela C.4 — Síntese dos fluxos funcionais validados

Fluxo validado	Resultado esperado
Registo e confirmação de conta	Conta criada, <i>email</i> enviado e acesso condicionado à validação.
Recuperação de palavra-passe	Token anterior invalidado, novo pedido emitido e <i>email</i> de segurança enviado após alteração.
Gestão de <i>stock</i>	Quantidades atualizadas, movimentos registados e disponibilidade recalculada.
Pedido externo de cliente	Cálculo de défice temporal e geração/atualização de necessidade quando aplicável.
Publicação no <i>marketplace</i>	Anúncio visível para outros produtores e corretamente associado a <i>stock</i> ou previsão.
Recomendação de compra	Sugestão criada quando existe défice temporal e ofertas compatíveis.
Recomendação de venda	Sugestão criada quando existe margem vendável ou necessidade compatível.
Encomenda	Reserva de quantidade, evolução por estados e atualização de <i>stock</i> na conclusão/cancelamento.
Mensagens	Envio, persistência, anexos e receção em tempo real.
Suporte	Criação de <i>ticket</i> , resposta administrativa, anexos e fecho explícito.
Produção	Aplicação funcional no Railway com base de dados, Redis, Cloudinary, Resend e WhiteNoise.

Tabela C.5 — Feedback considerado e ajustes realizados

Feedback ou necessidade identificada	Ajuste realizado
Algumas áreas apresentavam demasiada informação técnica.	Reorganização visual de páginas e simplificação de textos de apoio.
A distinção entre <i>stock</i> , reservado e compromissos externos podia gerar dúvidas.	Melhoria dos indicadores de inventário e reforço da explicação da margem vendável.
Era importante identificar rapidamente situações que exigiam ação.	Reorganização dos alertas por separadores, categorias e ações contextuais.
O <i>marketplace</i> precisava de distinguir melhor ofertas e procuras.	Separação visual entre anúncios de venda e procuras agregadas.
As encomendas precisavam de um acompanhamento mais claro.	Melhoria do detalhe da encomenda e da <i>timeline</i> operacional de estados.
A comunicação entre produtores deveria evitar canais externos.	Reforço do módulo de mensagens, anexos e comunicação em tempo real.
O suporte não deveria funcionar apenas como formulário isolado.	Implementação de <i>tickets</i> conversacionais com histórico e respostas administrativas.
O painel do produtor deveria orientar melhor a ação seguinte.	Introdução de ações recomendadas e indicadores operacionais contextualizados.